Software para Sistemas Ubíquos · Instituto de Informática, UFG · Prof. Dr. Otávio Calaça Xavier

Guia para apresentar o Marco 2

Permanência medida, não presença num instante. Um ESP32 no papel do celular do estudante ouve as três âncoras da sala; um servidor transforma esses heartbeats em minutos de permanência, avisa antes de punir e devolve a decisão ao aparelho. Primeiro o roteiro do dia, passo a passo e com prints anotados; depois a referência técnica completa.

Aluna **Alany Gabrielly** · 202105018 Responsabilidade **comunicação e resiliência**

Teste adversarial **sequência** , matrícula terminada em 8 Wokwi **aparelho ↗** · **CO_2 ↗**

Guia do dia · para você apresentar sozinha

Do computador ligado até a última pergunta

Sem slides. Você vai mostrar as pastas no VS Code, o circuito e o código no Wokwi, rodar o servidor no terminal e fazer cada teste ao vivo. Este guia diz, em ordem, o que abrir, o que digitar, o que apertar, o que vai aparecer e o que falar.

G1 Checklist do dia

	O QUÊ	COMO CONFERIR
☐	Rede que alcança o broker (porta 1883, ou 8884 com `--ws`)	No **PowerShell** (não no Git Bash): `Test-NetConnection broker.hivemq.com -Port 1883`. Se der `True`, o servidor é `python servidor.py`. Se der `False` (esperado na faculdade), é `python servidor.py --ws`. Se nem assim conectar, 4G do celular. Passo a passo na G3.
☐	Python com as bibliotecas	`pip install -r requirements.txt` dentro de `servidor/` (seção G3)
☐	VS Code aberto em `projeto-presenca`	Explorador mostrando `marco-02`
☐	Dois terminais no VS Code	Um para o servidor, outro para o validador, os testes e o gateway de CO_2
☐	Aba do Wokwi do aparelho	**wokwi.com/projects/476110187626906625**
☐	Aba do Wokwi do CO_2 (se for mostrar a Parte II)	**wokwi.com/projects/476001394839038977**
☐	Janela do navegador **visível**	Se o Wokwi ficar atrás de outra janela ou numa aba escondida, a simulação **congela**. Deixe o navegador e o VS Code lado a lado.
☐	Som ligado	O buzzer do aviso toca pelo alto-falante do computador

A ordem da apresentação

1 — 2 min

O problema e a fronteira (G2): permanência, não presença; quem mede e quem decide.

2 — 3 min

As pastas no VS Code (G4).

3 — 3 min

O circuito e o código no Wokwi (G6 e G7).

4 — 8–10 min

Os testes ao vivo (G9): normal, sala vizinha com avisos, adversarial, rede, orçamento esgotado.

5 — 2 min

Contrato e testes automáticos (G10 e G11).

6 — opcional

A fronteira de CO_2 (G12).

7 — resto

Perguntas (seção 13 da Parte I) e uma alteração ao vivo, se pedirem.

G2 O que foi feito, passo a passo

É a história que você conta. Cada passo tem o motivo.

1 — Escolha da fronteira

O Marco 2 pede dois componentes reais conversando. Escolhi o **celular do estudante** e o **servidor**, porque é por aí que passa a regra central do projeto: permanência medida. A âncora não entra, porque ela só transmite e nunca escuta ninguém (decisão fechada nº 3).

2 — O contrato primeiro

Antes do código, escrevi em `contrato/v1/` o formato de cada mensagem: `presenca.heartbeat.v1` (ida), `presenca.estado.v1` e `presenca.aviso.v1` (volta).

3 — A regra, sem rede

`servidor/permanencia.py` tem a trilateração, o polígono e a máquina de estados do `projeto-tecnico.md` §6, com 23 testes. Os tempos reais (30 s, 15 min…) são divididos por uma escala de 30 para caber numa demonstração.

4 — O servidor

`servidor/servidor.py` liga a regra ao MQTT: valida, deduplica, detecta fora de ordem e salto, refaz a sessão a cada heartbeat, devolve estado e avisos e acusa silêncio. Mais 11 testes.

5 — O aparelho

`aparelho/sketch.ino` no Wokwi: três potenciômetros como RSSI das âncoras, média, histerese −70/−85 dBm, heartbeat, fila para queda de rede, botões para as falhas, LEDs, OLED e buzzer.

6 **Rodei tudo junto**

Sete cenários no Wokwi com o servidor de verdade (Parte I, seção 8). Os prints e logs estão em `evidencias/presenca/`.

7 **Consertei o que apareceu**

Um atrasado contava como perdido; o "fora de ordem" do protótipo de CO_2 era, na verdade, uma duplicata; o acumulado oscilava meio segundo. Tudo corrigido e testado.

8 **A segunda fronteira**

A âncora de CO_2 (feita antes) continua: é a defesa contra a turma deixar os celulares na sala e ir embora.

G3 Preparar o computador (uma vez)

Passo 1 · Abrir um PowerShell

Todos os comandos deste guia são para o **PowerShell**. No VS Code, no painel do terminal (embaixo), clique na setinha ∨ ao lado do + e escolha **PowerShell**. Também serve o PowerShell do Windows, aberto pelo menu Iniciar.

Confira qual terminal abriu

Certo: a linha começa com `PS`, por exemplo `PS D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA>`.
Errado: aparece `MINGW64` e a linha termina em `$`. Isso é o Git Bash, que é o padrão do seu VS Code. Nele o `Test-NetConnection` dá `command not found`. Abra um PowerShell pela setinha ∨.

Passo 2 · Entrar na pasta do servidor

O PowerShell abre numa pasta qualquer, muitas vezes `C:\WINDOWS\System32`. Os comandos `pip install -r requirements.txt` e `python servidor.py` procuram os arquivos **na pasta atual**. Então, antes de tudo, entre na pasta com o `cd`:

```
PowerShell
cd "D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA\projeto-presenca\marco-02\servidor"
```

Confira o começo da linha. Tem que ser:

certo

```
PS D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA\projeto-presenca\marco-02\servidor>
```

Se aparecer “No such file or directory: 'requirements.txt'” ou “can't open file 'servidor.py'”

Você está na pasta errada. Se a linha começa com `PS C:\WINDOWS\System32>`, o `cd` não foi feito. Rode o `cd` acima e repita o comando.

Passo 3 · Instalar as bibliotecas (só na primeira vez)

PowerShell, já dentro da pasta servidor

```
pip install -r requirements.txt
```

Isso instala `paho-mqtt` (fala MQTT) e `jsonschema` (valida o contrato). O gateway de CO_2 usa as mesmas duas. Se aparecer `[notice] A new release of pip is available`, é só uma sugestão de atualização: pode ignorar.

Passo 4 · Testar a rede (sempre, em cada rede nova)

PowerShell

```
Test-NetConnection broker.hivemq.com -Port 1883
```

Este comando é **sempre igual**. Ele não conecta nada, só pergunta se a porta 1883 está liberada nesta rede. Olhe a última linha:

o que apareceu em casa, em 25/09/2026 (saída real)

```
ComputerName     : broker.hivemq.com
RemoteAddress    : 52.58.200.83
RemotePort       : 1883
InterfaceAlias   : Wi-Fi
SourceAddress    : 192.168.0.8
TcpTestSucceeded : True
```

Passo 5 · Escolher o comando do servidor

O TESTE DEU	PARA LIGAR O SERVIDOR, DIGITE	O QUE SIGNIFICA
`TcpTestSucceeded : True`	`python servidor.py`	A rede libera o MQTT comum. Foi o caso em casa.
`TcpTestSucceeded : False`	`python servidor.py --ws`	A rede bloqueia a 1883 (esperado na faculdade). O `--ws` manda o servidor ir pelo WebSocket, na porta 8884.
Com `--ws` também não aparece `[MQTT] conectado`	Conecte o notebook no 4G do celular e volte ao passo 4	A rede bloqueia os dois caminhos.

Onde vai o --ws

O `--ws` **não** vai no `Test-NetConnection` e **não** se digita sozinho. Ele vai no **fim do comando que liga o servidor**, separado por um espaço. É o mesmo comando de sempre com `--ws` no final:

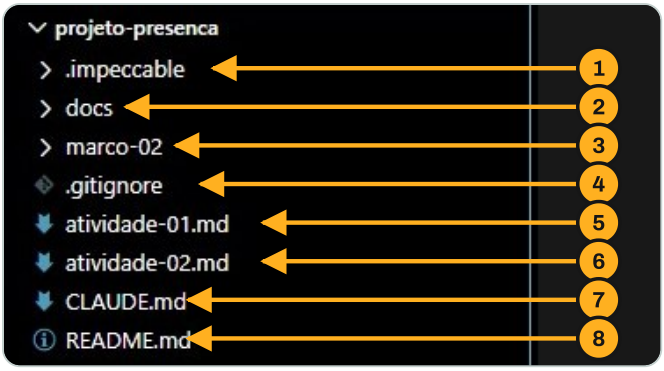
`python servidor.py` → `python servidor.py --ws`

O mesmo vale para o gateway de CO_2: `python consumidor.py --ws`.

Se deu `False` e você quiser conferir antes se o caminho do WebSocket está livre, troque só o número da porta no teste: `Test-NetConnection broker.hivemq.com -Port 8884`. Se esse der `True`, o `--ws` vai funcionar. O porquê de tudo isso está na seção G14.

G4 As pastas e os arquivos, no VS Code

Mostre na ordem abaixo, abrindo cada pasta no explorador. Cada número aponta para o que falar.



A raiz do projeto no VS Code.

1. **.impeccable/** Configuração da ferramenta de revisão de design usada na apresentação. Não faz parte do sistema; pode ignorar.
2. **docs/** Tem o `projeto-tecnico.md`, a especificação completa (o nosso SDD): parâmetros, protocolo, máquina de estados, alternativas rejeitadas. É de lá que saíram todos os números do Marco 2.
3. **marco-02/ Tudo o que você vai apresentar.** As próximas figuras abrem esta pasta.
4. **.gitignore** Lista o que o Git não guarda (logs, configurações locais, cache do Python).
5. **atividade-01.md** Entrega da Atividade 01: a análise do problema.
6. **atividade-02.md** Entrega da Atividade 02: a modelagem.
7. **CLAUDE.md** Notas de trabalho do projeto (decisões fechadas, parâmetros). Fica fora do Git de propósito.
8. **README.md** Apresentação do projeto Presença para quem abre o repositório.

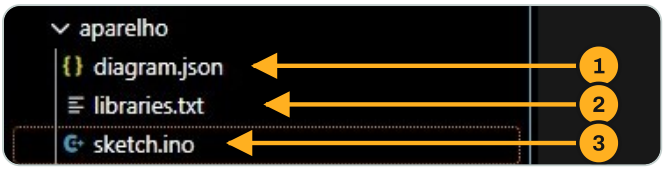


Dentro de `marco-02/`: uma pasta por peça.

1. **aparelho/ O que roda no Wokwi:** o ESP32 que faz o papel do celular do estudante. Produtor da fronteira principal.
2. **apresentacao/** Este guia (HTML e PDF) e o que o gera.
3. **co2-ancora/** O ESP32 da *segunda* fronteira: a âncora que mede CO_2 (Parte II).
4. **co2-gateway/** O Python da segunda fronteira: recebe o CO_2 e alerta incoerência. É o "gateway" que você vai rodar com `python consumidor.py`.
5. **contrato/ O idioma** que aparelho e servidor falam: os schemas JSON versionados e exemplos.
6. **evidencias/** Prints e logs dos testes que eu já rodei no Wokwi.
7. **servidor/ O que roda no seu notebook:** o servidor de permanência em Python. Consumidor da fronteira principal.
8. **GUIA.md** Guia em texto da fronteira de CO_2.
9. **README.md** Resumo do Marco 2 em uma página: o que é, como rodar, o que foi validado.

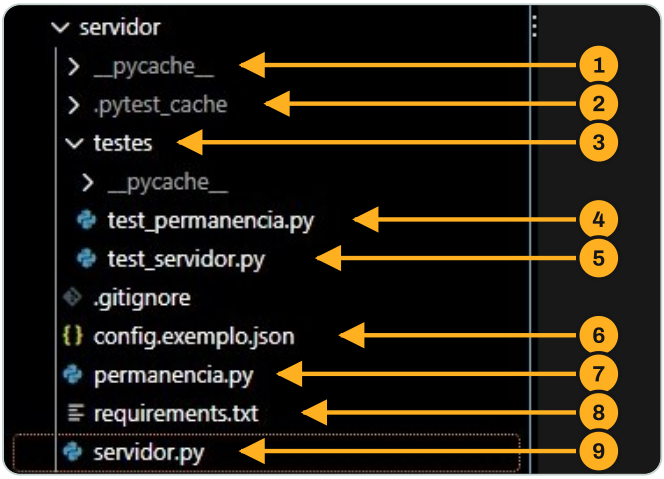
A frase para esta parte

"O `aparelho` é o produtor, roda no Wokwi. O `servidor` é o consumidor, roda aqui. O `contrato` é o que os dois combinaram. `evidencias` é a prova de que rodou."



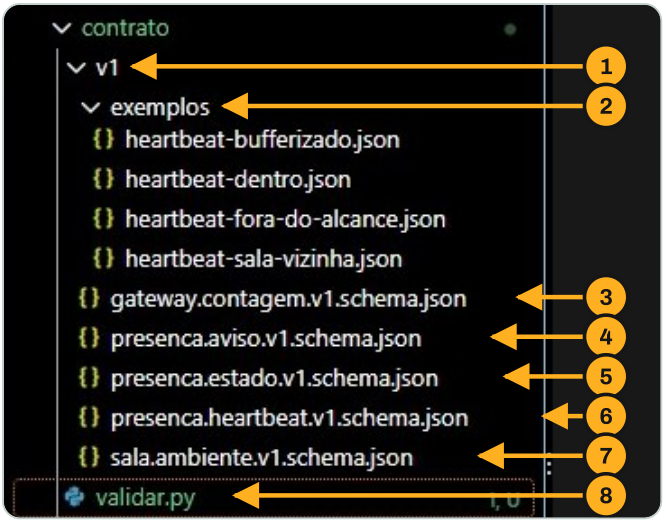
`aparelho/`: são exatamente os 3 arquivos do projeto no Wokwi.

1. **diagram.json** O circuito escrito em texto: cada peça (ESP32, 3 potenciômetros, 2 botões, 3 LEDs, 3 resistores, buzzer, OLED), onde fica na tela e cada fio. O Wokwi desenha o circuito a partir dele.
2. **libraries.txt** As bibliotecas que o Wokwi baixa: `PubSubClient` (MQTT), `Adafruit GFX` e `Adafruit SSD1306` (display).
3. **sketch.ino** O programa do ESP32, em C++ (~535 linhas). Lê o sinal, filtra, decide a região, manda o heartbeat, recebe o estado e acende LEDs, OLED e buzzer.



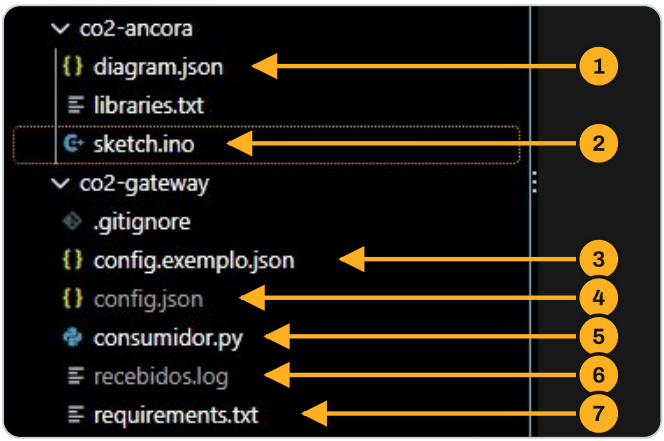
`servidor/`: o consumidor da fronteira principal.

1. **__pycache__/** Criada sozinha pelo Python. Ignore.
2. **.pytest_cache/** Criada sozinha pelo pytest. Ignore.
3. **testes/** Os 34 testes automáticos.
4. **test_permanencia.py** 23 testes da regra: trilateração, polígono, persistência, orçamento, requalificação, estorno, avisos, cálculo retroativo.
5. **test_servidor.py** 11 testes do servidor inteiro com um MQTT falso: contrato, duplicata, fora de ordem, salto, lacuna, reboot, atrasados.
6. **config.exemplo.json** Broker, tópicos, **parâmetros** (com a escala 1:30) e a **planta da sala** (polígono e posição das âncoras).
7. **permanencia.py** **A regra**: localização e máquina de estados. Não tem rede nem relógio; recebe o log e devolve o estado.
8. **requirements.txt** O que instalar: `paho-mqtt` e `jsonschema`.
9. **servidor.py** **O programa que você roda**: conecta no broker, valida, deduplica, chama a regra, publica estado e avisos, detecta silêncio.



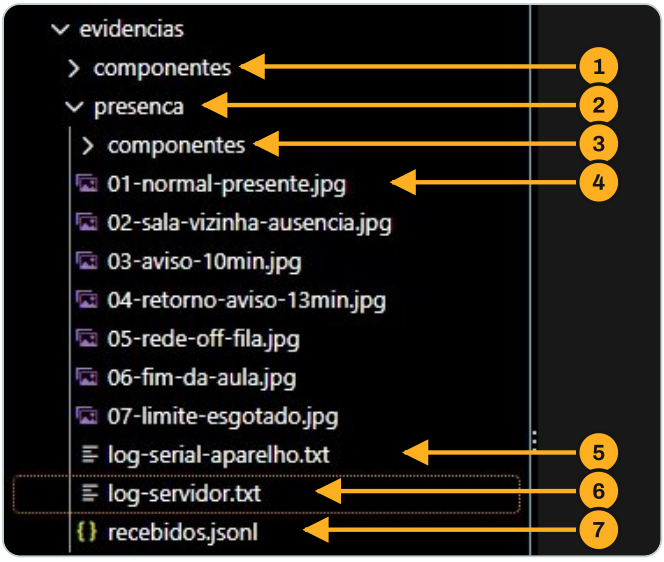
`contrato/`: o combinado entre os dois lados, em arquivo.

1. **v1/** A versão 1 do contrato. A versão faz parte do nome: uma mudança incompatível vira `v2` sem quebrar quem ainda usa a `v1`.
2. **exemplos/** Mensagens reais, gravadas durante os testes, uma por situação. Todas passam pela validação.
3. **gateway.contagem.v1** (CO_2) Comando do gateway para a âncora: quantos estudantes o rádio declara.
4. **presenca.aviso.v1** Servidor → aparelho: os avisos de 10 min, 13 min, limite esgotado e fim da aula.
5. **presenca.estado.v1** Servidor → aparelho: o estado decidido, acumulado, orçamento, posição.
6. **presenca.heartbeat.v1** **Aparelho → servidor**: o RSSI das 3 âncoras, a região, a sequência. O servidor valida cada heartbeat contra este arquivo.
7. **sala.ambiente.v1** (CO_2) Âncora → gateway: o CO_2 medido e o estado da verificação cruzada.
8. **validar.py** Script para você **mostrar o contrato funcionando**: valida todos os exemplos e mostra uma mensagem inválida sendo rejeitada.



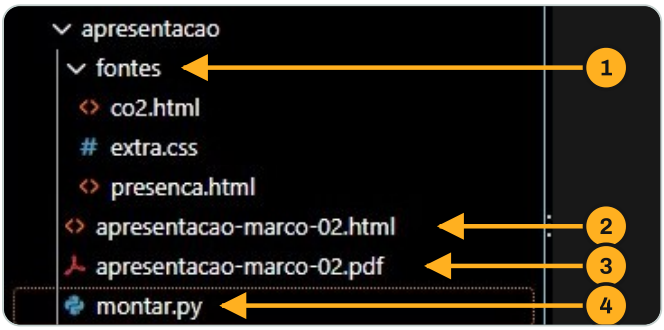
`co2-ancora/` e `co2-gateway/`: a segunda fronteira.

1. **co2-ancora/diagram.json** Circuito da âncora de CO_2: 1 potenciômetro (CO_2), 2 botões, 3 LEDs, OLED.
2. **co2-ancora/sketch.ino** Programa da âncora: mede CO_2, histerese, verificação cruzada com a contagem do rádio.
3. **config.exemplo.json** Modelo de configuração do gateway.
4. **config.json** A configuração que ele usa de fato (começa declarando 25 estudantes). Fica fora do Git.
5. **consumidor.py** O gateway de CO_2: `python consumidor.py`. Recebe, valida, deduplica e dispara o alerta de incoerência.
6. **recebidos.log** Criado ao rodar: uma linha por evento recebido.
7. **requirements.txt** O que instalar (o mesmo do servidor).



`evidencias/`: a prova de que rodou.

1. **componentes/** Fotos da âncora de CO_2 (Parte II).
2. **presenca/** Tudo da execução da fronteira principal no Wokwi, em 25/09/2026.
3. **presenca/componentes/** Recortes de cada peça e ampliações do OLED.
4. **01... a 07....jpg** Uma captura do Wokwi por cenário testado (normal, sala vizinha, avisos, rede fora, fim da aula, limite esgotado).
5. **log-serial-aparelho.txt** Tudo o que o ESP32 escreveu no monitor serial.
6. **log-servidor.txt** Tudo o que o servidor escreveu no terminal.
7. **recebidos.jsonl** Cada heartbeat recebido, com o carimbo do servidor e o que ele concluiu.



`apresentacao/`.

1. **fontes/** O texto deste guia, as imagens e os estilos. É aqui que se edita.
2. **.html** Este guia para abrir no navegador (funciona sem internet).
3. **.pdf** Este guia em PDF.
4. **montar.py** Junta as fontes e gera o HTML e o PDF: `python montar.py`.

G5 Ligar o servidor

No PowerShell (G3), depois do teste de rede. Digite **uma** das duas últimas linhas, conforme o resultado do `Test-NetConnection`: `True` → sem `--ws`; `False` → com `--ws`.

PowerShell · servidor

```
cd "D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA\projeto-presenca\marco-02\servidor"
python servidor.py       # rede normal (porta 1883)
python servidor.py --ws  # rede que bloqueia a 1883, como a da faculdade (G14)
```

Use **uma** das duas linhas. Com `--ws`, a linha de conexão vira `[MQTT] conectado a broker.hivemq.com:8884 (MQTT sobre WebSocket seguro)`; todo o resto é igual.

Tem que aparecer (saída real, sem `--ws`):

o que aparece

```
configuracao: config.exemplo.json

=== Presenca / servidor de permanencia ===
p painel | n nova sessao | q sair
[MQTT] conectado a broker.hivemq.com:1883
[MQTT] assinando ufg/ssu/presenca/alany/sala-204/heartbeat

=== sessao s31a2 aberta ===
escala 1:30  aula 100 s  persistencia 1 s  orcamento 30 s  avisos em 20 s e 26 s  requalificacao 60 s
```

TECLA + ENTER	O QUE FAZ	QUANDO USAR
`n`	Abre uma **sessão nova** (aula nova, do zero)	Antes de cada demonstração, logo depois de dar ▶ no Wokwi
`p`	Mostra o painel: estado, acumulado, contadores, transições	Quando quiser resumir o que aconteceu
`q`	Encerra	No fim (ou `Ctrl` + `C`)

Cada heartbeat vira uma linha. Como ler uma:

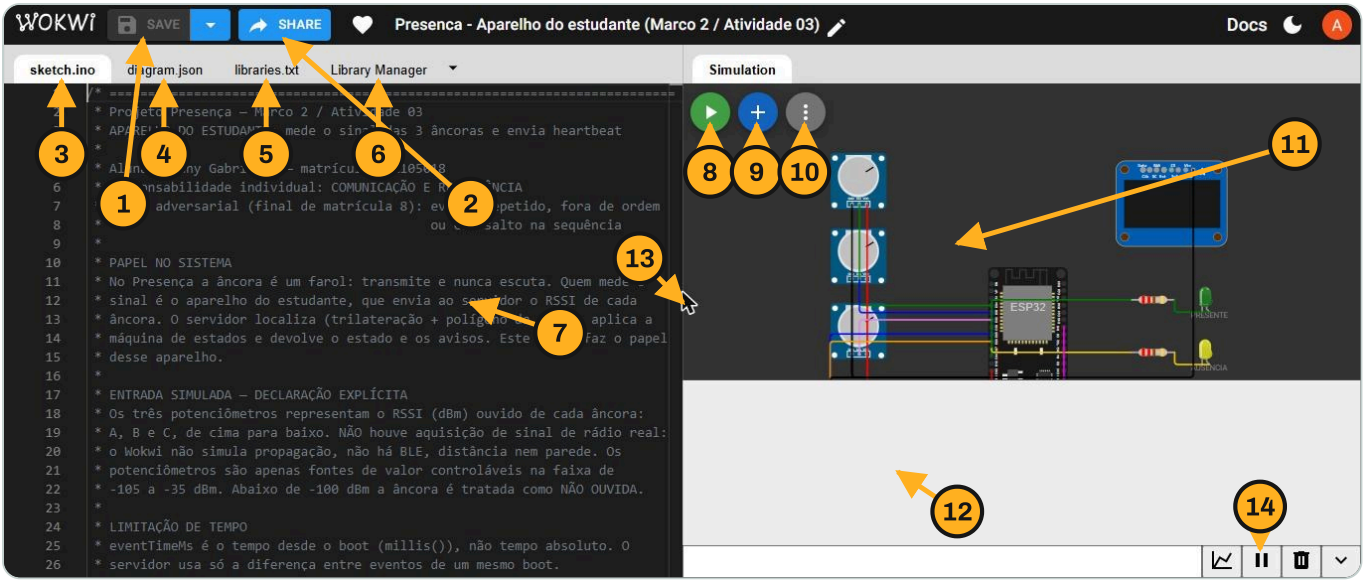
uma linha do servidor, parte por parte

```
01:31:02 t=  5.5s    seq=10  A -62 B -62 C -59  DENTRO_ALCANCE (  6.0,  4.4) dentro -> PRESENTE       acum  4.1
|        |         |     |              |         |         |         |                  |
hora     tempo da aula  sequência  RSSI das 3 âncoras  região (aparelho)  posição (x,y)  classe     estado decidido
```

A cada 30 s ele imprime o painel sozinho. Quando o estado muda, aparece uma linha `>>> X -> Y em t=… (motivo)`. Avisos aparecem numa faixa vermelha.

G6 Ligar o Wokwi e se localizar na tela

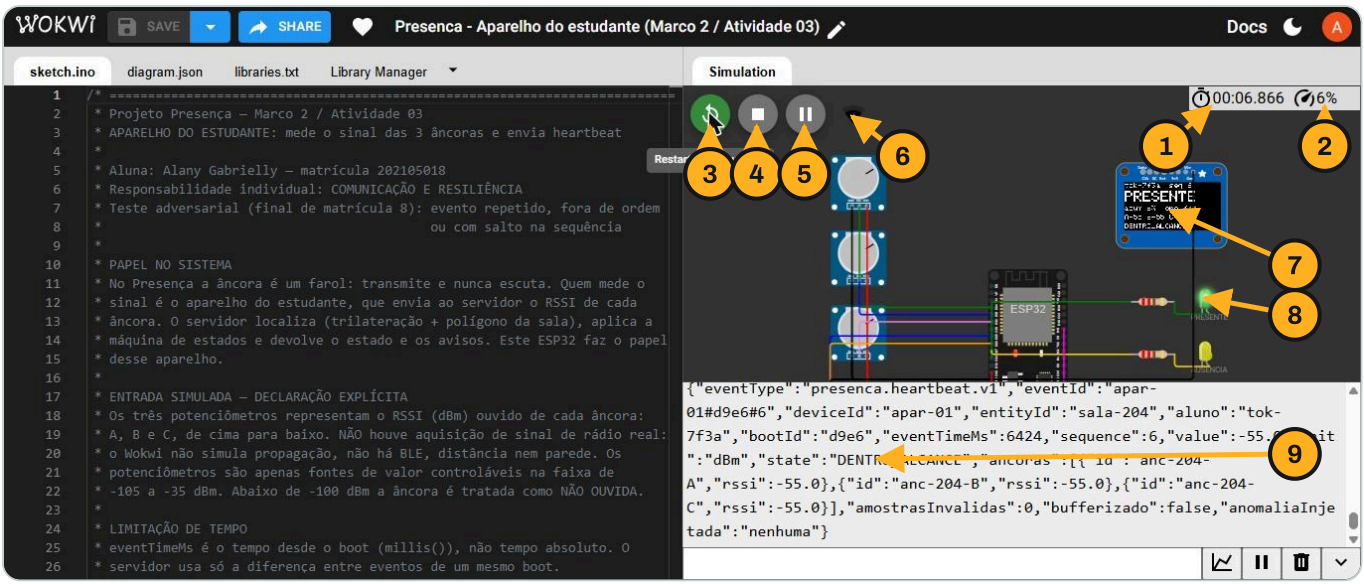
Abra **wokwi.com/projects/476110187626906625**. Não precisa estar logada para rodar.



O que aparece ao abrir **wokwi.com/projects/476110187626906625**.

1. **SAVE** Salva o projeto. **No dia, não precisa**: o projeto já está salvo. Se mexer em algo e salvar, a mudança fica pública.
2. **SHARE** Copia o link do projeto.
3. **sketch.ino** O programa do ESP32. Clique aqui para mostrar o código.
4. **diagram.json** O circuito em texto.
5. **libraries.txt** As bibliotecas.
6. **Library Manager** Onde se adicionam bibliotecas. Não precisa mexer.
7. **Editor** O código. Dá para rolar e mostrar cada parte (seção 8 deste guia).
8. **▶ Play Liga a simulação.** Na primeira vez compila (pode demorar até 1 min na fila do plano gratuito).
9. **+** Adiciona peças. Não mexa.
10. **⋮** Menu da simulação.
11. **Circuito** As peças. Roda do mouse aproxima e afasta; arrastar o fundo move a vista.
12. **Monitor serial** Tudo o que o ESP32 escreve: os heartbeats em JSON, as mudanças de região, os avisos, a rede. Mostre para a banca.
13. **Divisória** Arraste para dar mais espaço ao circuito ou ao código. A divisória entre circuito e monitor também se arrasta.
14. **Botões do monitor** Gráfico, pausar a rolagem, limpar e recolher.

Clique no **▶ Play** (8). Na primeira vez aparece “Compiling project...” e às vezes uma fila (“Build Servers Busy”): **não é erro**, espere. Em cerca de 10 s de simulação o Wi-Fi conecta, o servidor responde e o OLED mostra `PRESENTE`.



Depois do ▶: a simulação rodando.

1. **Relógio da simulação** Tempo *simulado* desde o boot. Anda mais devagar que o real.
2. **Velocidade** Quanto do tempo real a simulação consegue acompanhar (fica entre 10 % e 80 %). É normal.
3. ↻ **Reiniciar** Reinicia o ESP32 (novo boot). **Não** devolve os potenciômetros à posição inicial.
4. ■ **Parar** Desliga a simulação.
5. ❚❚ **Pausar** Congela. Não use durante um teste: o servidor vai acusar silêncio.
6. **Wi-Fi** Aparece quando o ESP32 conectou na rede `Wokwi-GUEST`.
7. **OLED** O que o estudante vê: estado decidido pelo servidor, acumulado, orçamento, RSSI das 3 âncoras, região.
8. **LED verde** PRESENTE, decidido pelo servidor.
9. **Heartbeat no serial** O JSON que foi enviado. Aponte o `eventId` (aparelho#boot#sequência).

Como mexer nos potenciômetros e nos botões

PEÇA	COMO
Potenciômetro	**Clique nele** e use o teclado: `Home` = 0 (tudo à esquerda), `End` = 1023, `PageUp` / `PageDown` = ±102, setas = ±1. Também dá para arrastar o botão com o mouse, mas o teclado é exato. O RSSI aparece no OLED (linha `A-56 B-56 C-56`).
Botão	Clique e **segure por 1 segundo**. A simulação anda mais devagar que o real; um clique rápido pode não ser visto.

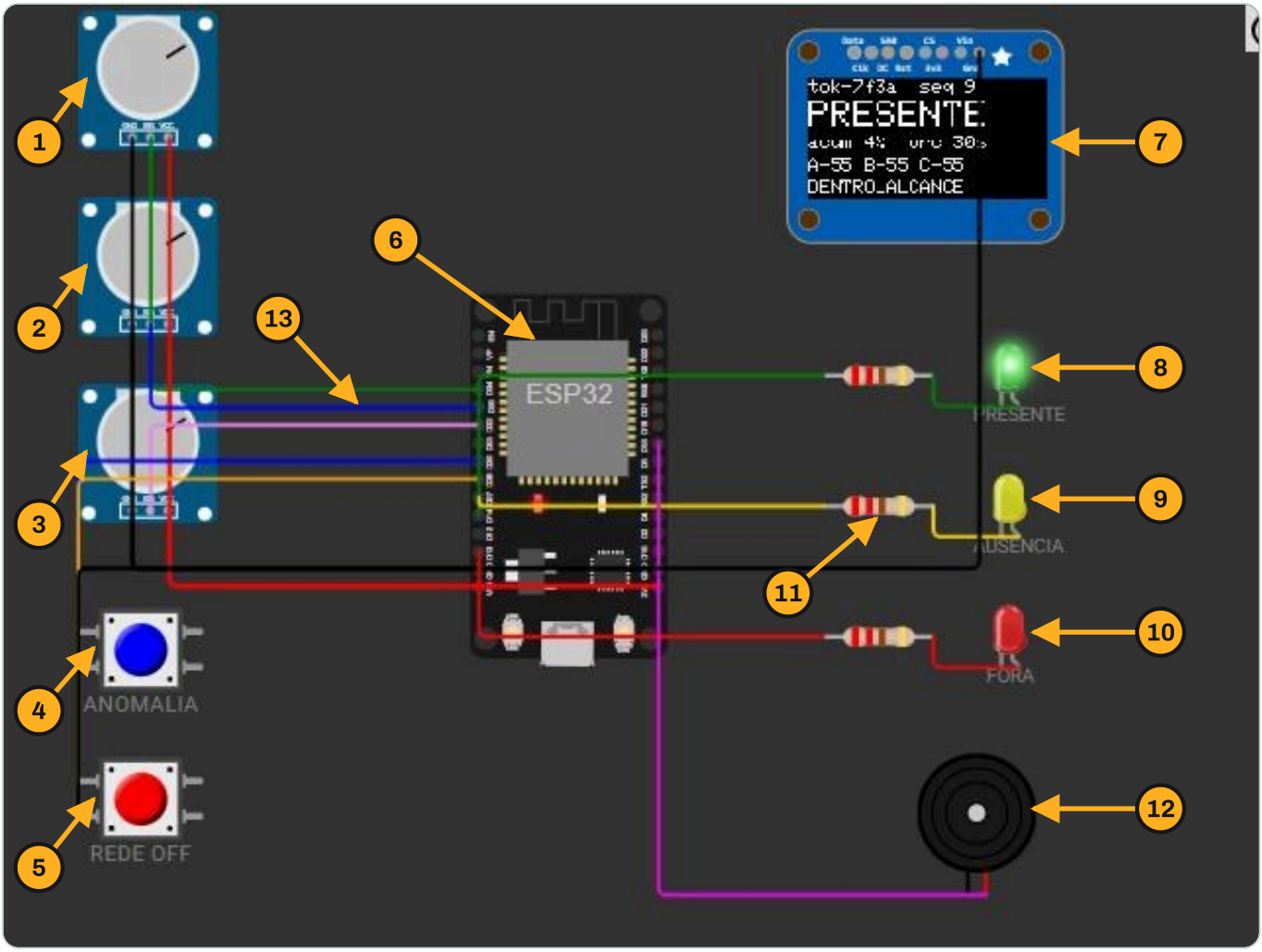
CENÁRIO	A	B	C	OLED MOSTRA	SERVIDOR DECIDE
Dentro da sala (posição inicial)	Home, PgUp ×7	Home, PgUp ×7	Home, PgUp ×7	A-56 B-56 C-56	(6,0; 2,8) → dentro
Corredor, colado à sala	Home, PgUp ×5	Home, PgUp ×8	Home, PgUp ×7	A-70 B-49 C-56	(13,9; 12,4) → fora, com sinal forte
Fora do prédio	Home	Home	Home	A-- B-- C--	nenhuma âncora ouvida → fora

Duas pegadinhas

1. O ↻ Reiniciar **não** devolve os potenciômetros à posição inicial. Para voltar, use a receita "dentro da sala" ou recarregue a página (F5).
2. Se você trocar de aba ou minimizar o navegador, a simulação congela e o servidor escreve `SILENCIO`. Isso não é defeito: é a detecção de silêncio funcionando.

G7 O circuito no Wokwi

Para mostrar o circuito grande, arraste a divisória (13 da figura da tela) para a esquerda e use a roda do mouse sobre o circuito.



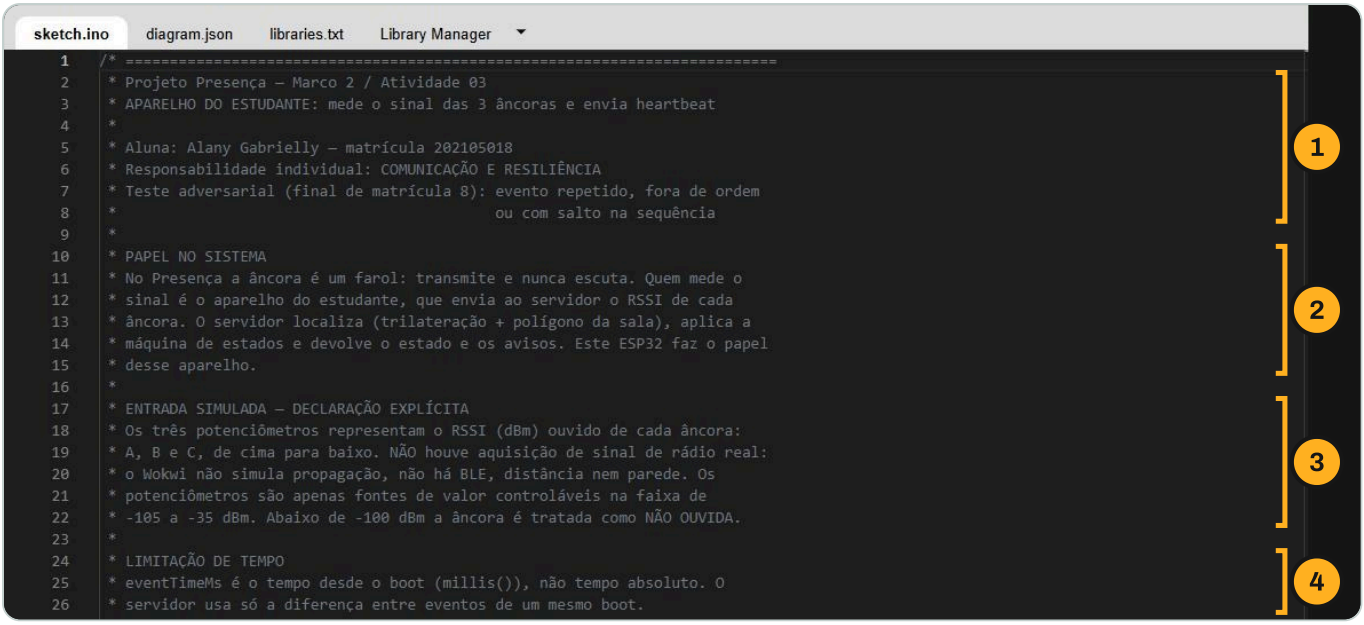
O circuito do aparelho, peça por peça (vista ampliada, simulação rodando).

1. **Potenciômetro A** RSSI da âncora A (parede oeste), no pino `GPIO34`.
2. **Potenciômetro B** RSSI da âncora B (parede leste), no `GPIO35`.
3. **Potenciômetro C** RSSI da âncora C (parede norte), no `GPIO32`.
4. **Botão azul ANOMALIA** `GPIO25`. Cada aperto arma a próxima anomalia: duplicata → fora de ordem → salto.
5. **Botão vermelho REDE OFF** `GPIO26`. Derruba e restabelece a rede simulada.
6. **ESP32** O "celular". Wi-Fi, MQTT, a lógica do `sketch.ino`.
7. **OLED SSD1306** Display I^2C (SDA 21, SCL 22).
8. **LED verde** `GPIO14`: PRESENTE.
9. **LED amarelo** `GPIO27`: AUSÊNCIA TOLERADA; pisca quando chega um aviso.
10. **LED vermelho** `GPIO13`: FORA.
11. **Resistores de 220 Ω** Um por LED, para limitar a corrente (~6 mA).
12. **Buzzer** `GPIO18`: toca quando chega um aviso (a notificação do celular).
13. **Fios dos potenciômetros** Verde, azul e violeta levam o sinal de cada potenciômetro ao ESP32; vermelho é 3,3 V e preto é GND.

G8 O código no Wokwi, parte por parte

Clique em **sketch.ino** e role. Cada print abaixo é uma tela; os números marcam os trechos e as linhas estão à esquerda. Para ir direto a uma linha: clique no código e aperte `Ctrl` + `G` … (ou `Ctrl` + `F` e procure o nome da função).

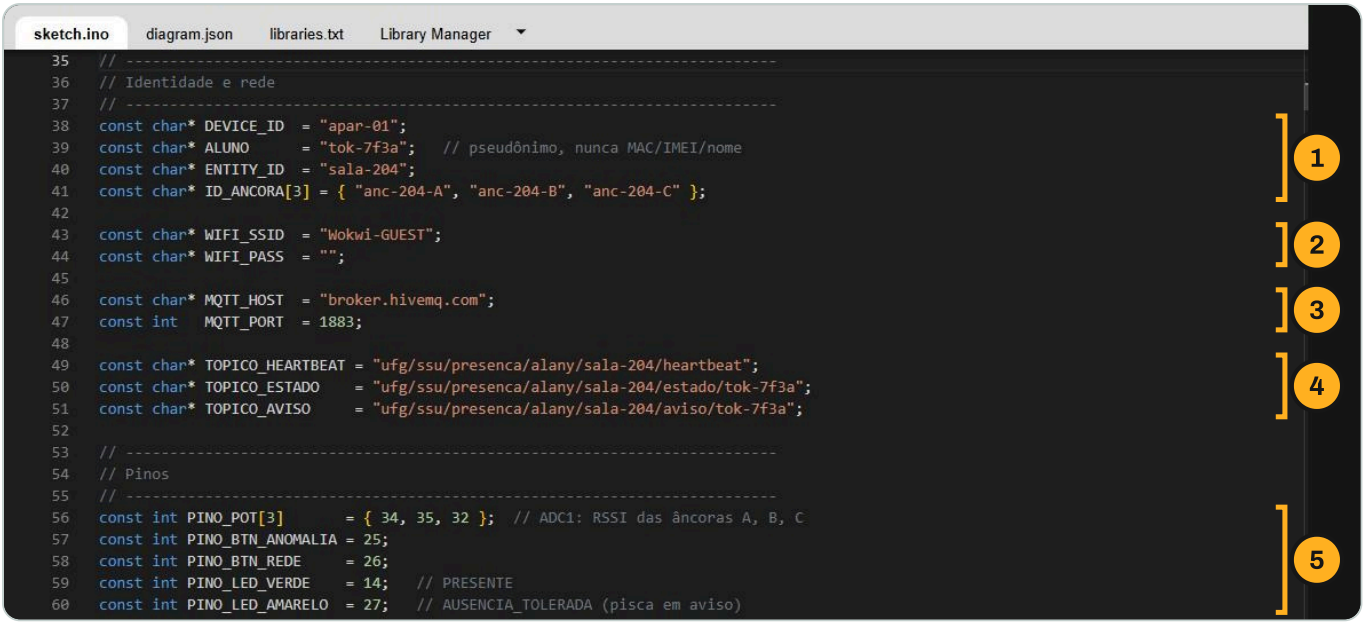
PARA EXPLICAR…	PROCURE	LINHA
Parâmetros e histerese	`LIMIAR_ENTRADA`	70
Âncora não ouvida	`void amostrar`	151
Regra local (histerese)	`atualizarRegiao`	195
O heartbeat (contrato)	`montarJson`	218
Teste adversarial	`publicarHeartbeat`	281
Fila da queda de rede	`enfileirar`	247
Aviso idempotente	`aoReceberAviso`	367
Comportamento seguro	`void atuar`	396
bootId	`esp_random`	497




```
/* ========================================================================
 * Projeto Presença — Marco 2 / Atividade 03
 * APARELHO DO ESTUDANTE: mede o sinal das 3 âncoras e envia heartbeat
 *
 * Aluna: Alany Gabrielly — matrícula 202105018
 * Responsabilidade individual: COMUNICAÇÃO E RESILIÊNCIA
 * Teste adversarial (final de matrícula 8): evento repetido, fora de ordem
 *                                           ou com salto na sequência
 *
 * PAPEL NO SISTEMA
 * No Presença a âncora é um farol: transmite e nunca escuta. Quem mede o
 * sinal é o aparelho do estudante, que envia ao servidor o RSSI de cada
 * âncora. O servidor localiza (trilateração + polígono da sala), aplica a
 * máquina de estados e devolve o estado e os avisos. Este ESP32 faz o papel
 * desse aparelho.
 *
 * ENTRADA SIMULADA — DECLARAÇÃO EXPLÍCITA
 * Os três potenciômetros representam o RSSI (dBm) ouvido de cada âncora:
 * A, B e C, de cima para baixo. NÃO houve aquisição de sinal de rádio real:
 * o Wokwi não simula propagação, não há BLE, distância nem parede. Os
 * potenciômetros são apenas fontes de valor controláveis na faixa de
 * -105 a -35 dBm. Abaixo de -100 dBm a âncora é tratada como NÃO OUVIDA.
 *
 * LIMITAÇÃO DE TEMPO
 * eventTimeMs é o tempo desde o boot (millis()), não tempo absoluto. O
 * servidor usa só a diferença entre eventos de um mesmo boot.
```

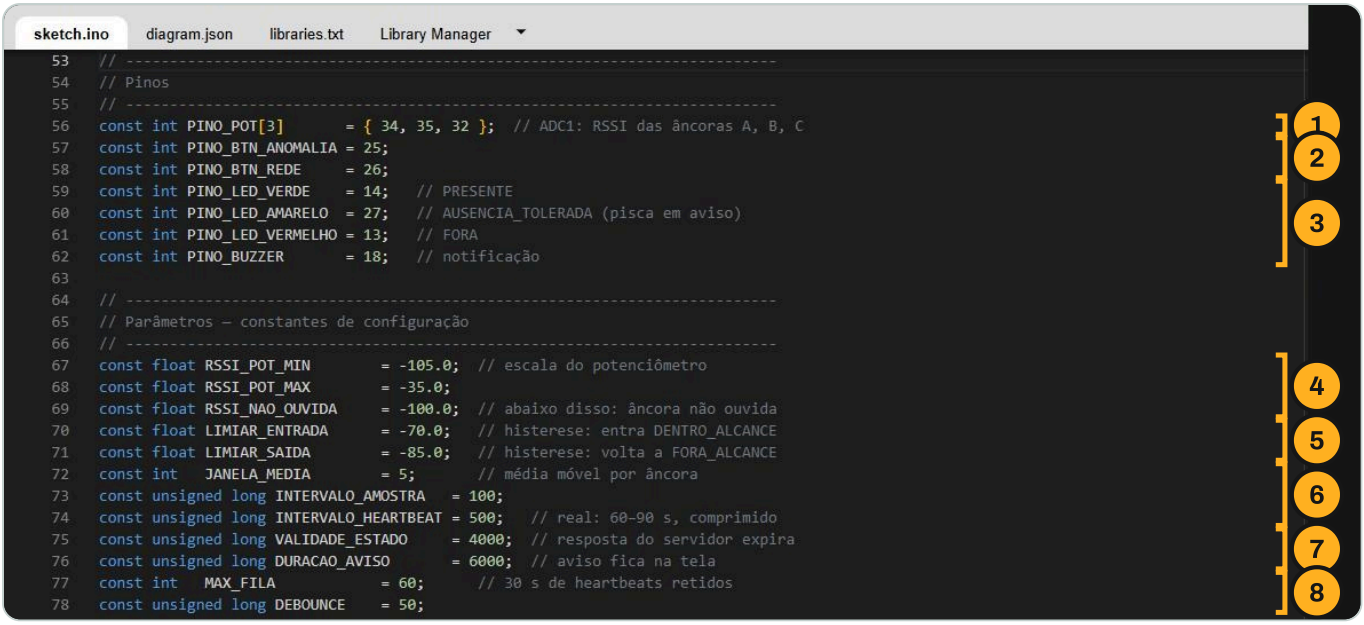
Linhas 1–26: o cabeçalho.

1. **Identificação** Projeto, aluna, responsabilidade e o teste adversarial da matrícula final 8.
2. **Papel no sistema** Por que o ESP32 é o celular: a âncora só transmite, quem mede é o aparelho.
3. **Declaração da entrada simulada** Exigida pela Atividade 03: potenciômetro não é rádio. Faixa −105 a −35 dBm; abaixo de −100 é “não ouvida”.
4. **Limitação de tempo** `eventTimeMs` é tempo desde o boot, não relógio real.



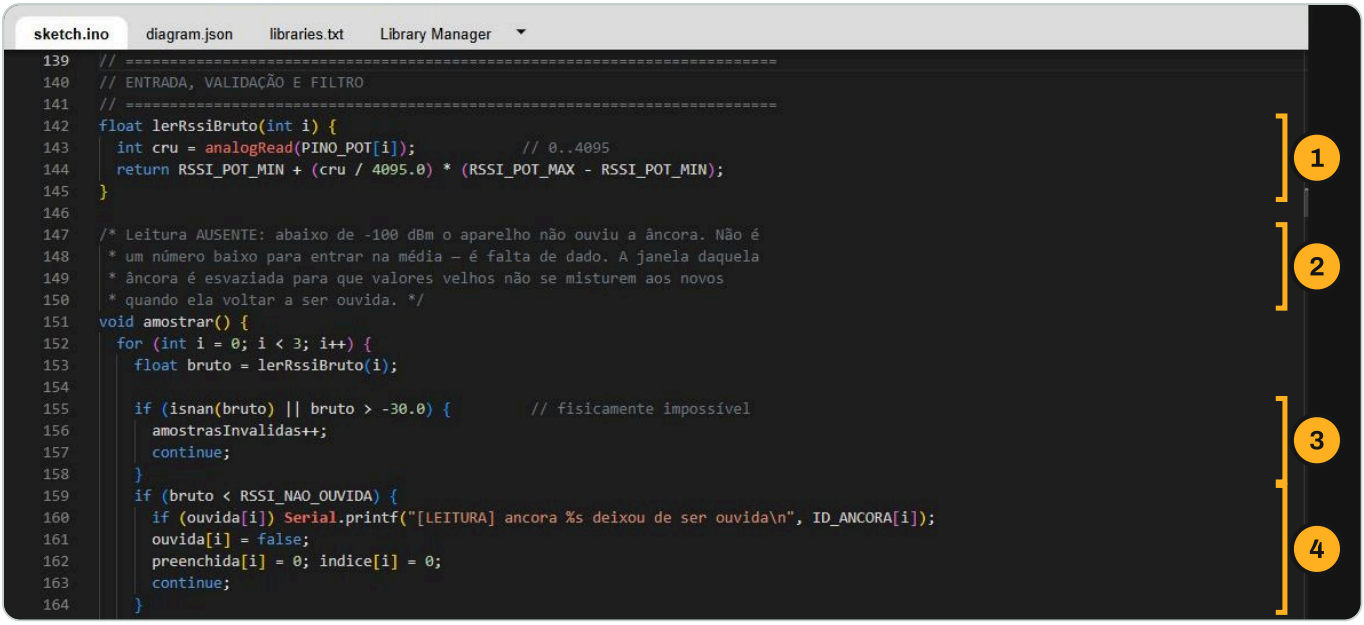
Linhas 35–60: identidade, rede e tópicos.

1. **Identidade** `apar-01` é o aparelho; `tok-7f3a` é o *pseudônimo* do aluno (nunca MAC ou nome); os ids das 3 âncoras.
2. **Wi-Fi** A rede do próprio Wokwi, com saída para a internet.
3. **Broker MQTT** O servidor público `broker.hivemq.com`, porta 1883.
4. **Tópicos** Onde publica o heartbeat e onde escuta o estado e o aviso — só os do próprio aluno.
5. **Pinos** Onde cada peça está ligada.



Linhas 53–78: pinos e parâmetros. Todos os números estão aqui, nenhum espalhado pelo código.

1. **Potenciômetros** Nos pinos 34, 35 e 32 (ADC1, que funciona com Wi-Fi ligado).
2. **Botões** 25 (anomalia) e 26 (rede).
3. **LEDs e buzzer** 14 verde, 27 amarelo, 13 vermelho, 18 buzzer.
4. **Escala do potenciômetro** 0–4095 vira −105 a −35 dBm; abaixo de −100 a âncora não foi ouvida.
5. **Histerese** Entra no alcance acima de −70 dBm, sai só abaixo de −85. *Bom lugar para uma alteração ao vivo.*
6. **Tempos** Média de 5 amostras, uma amostra a cada 100 ms, heartbeat a cada 500 ms (no real, 60–90 s).
7. **Validade e aviso** Resposta do servidor vale 4 s; aviso fica 6 s na tela.
8. **Fila e debounce** Até 60 heartbeats guardados sem rede; botão ignora repiques de 50 ms.



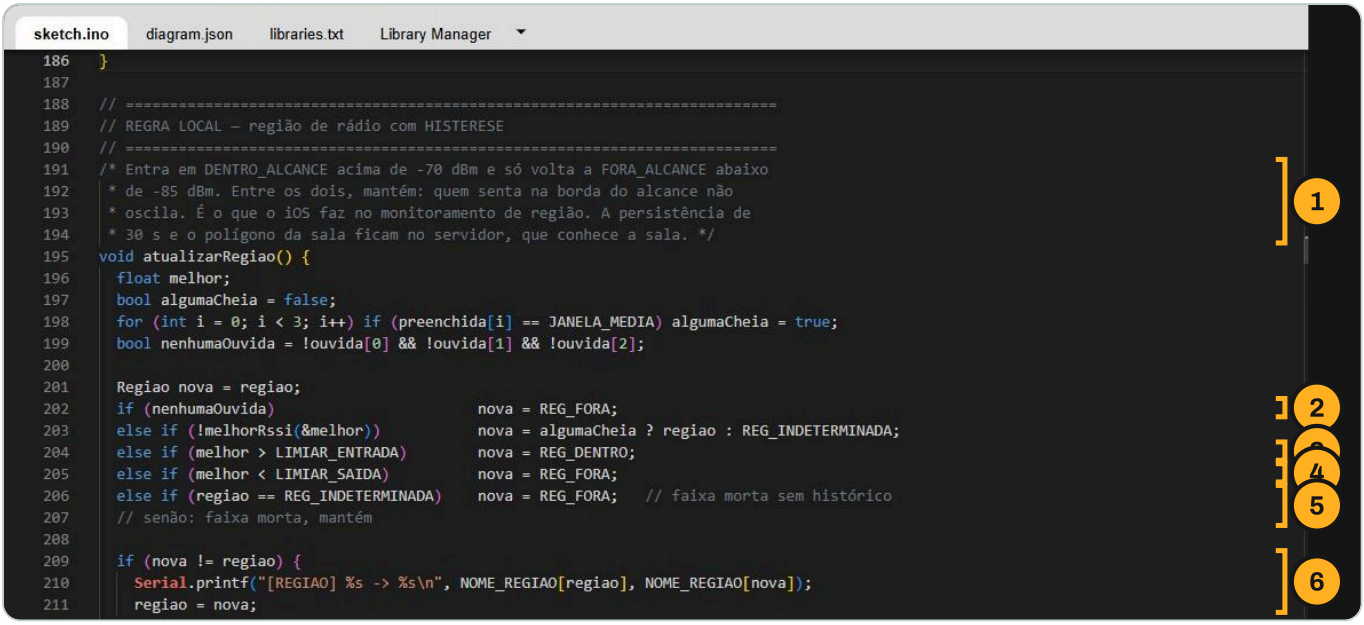
Linhas 139–164: leitura e validação.

1. **lerRssiBruto** Lê o ADC (0–4095) e converte para dBm.
2. **Comentário** Por que "não ouvida" não é um número baixo, é falta de dado.
3. **Leitura impossível** Acima de −30 dBm conta como inválida e é descartada.
4. **Âncora não ouvida** Abaixo de −100 dBm: marca como não ouvida e *esvazia a janela* daquela âncora.



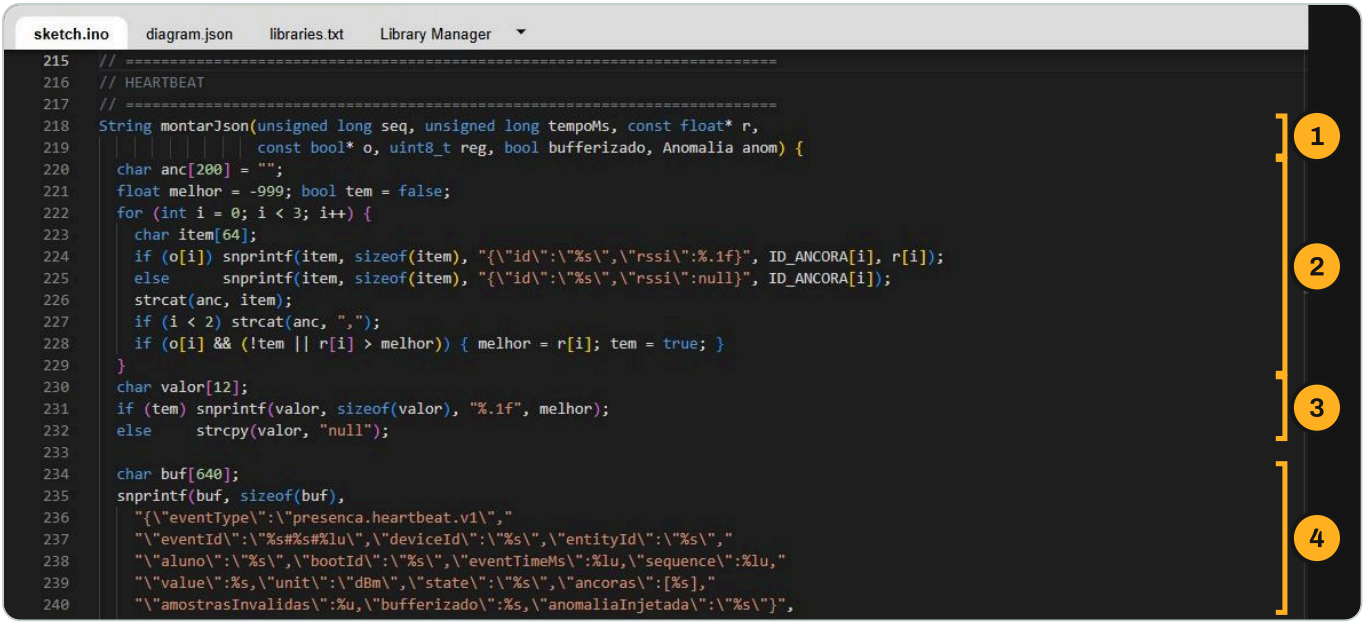
Linhas 166–186: filtro.

1. **Média móvel** Guarda as 5 últimas leituras de cada âncora e tira a média: suaviza o ruído.
2. **melhorRssi** O maior sinal entre as âncoras ouvidas. É ele que vai na regra de região.



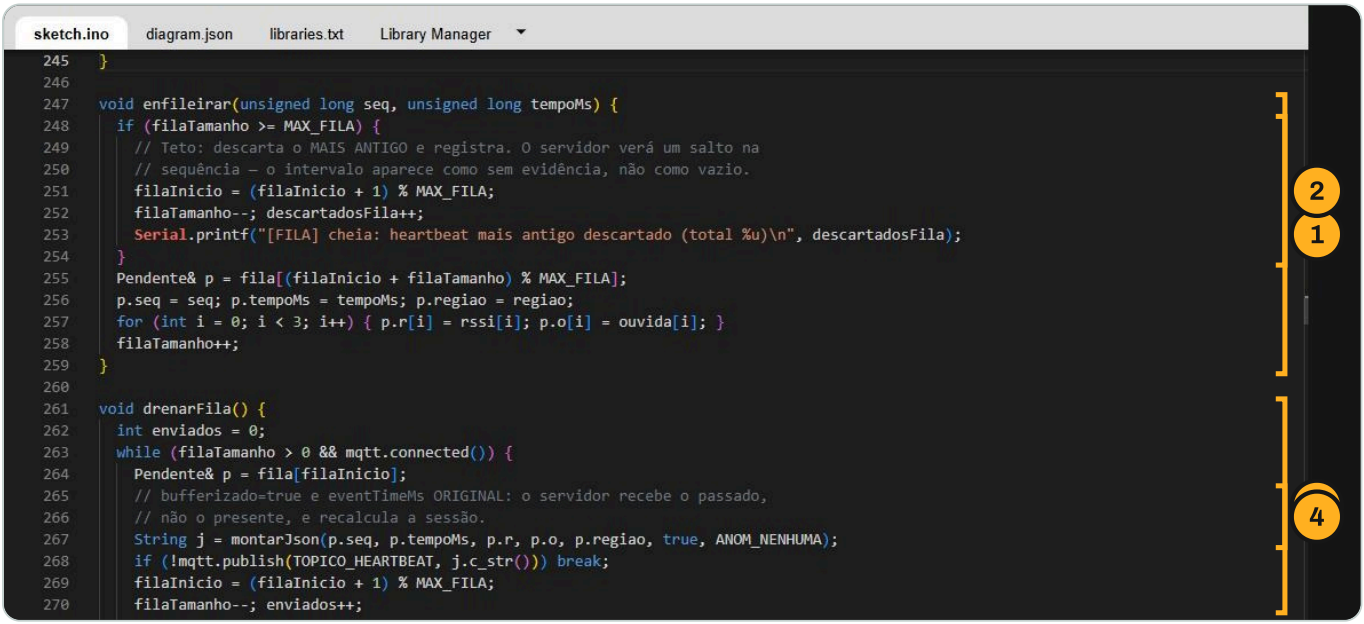
Linhas 186–211: a regra local, com histerese.

1. **Explicação** A regra local do aparelho.
2. **Nenhuma ouvida** Fora do alcance, sem discussão.
3. **Entrada** Melhor sinal acima de −70 dBm → `DENTRO_ALCANCE`.
4. **Saída** Abaixo de −85 dBm → `FORA_ALCANCE`.
5. **Faixa morta** Entre −85 e −70 mantém o que estava: **isso é a histerese**.
6. **Serial** Escreve `[REGIAO] X -> Y` quando muda.



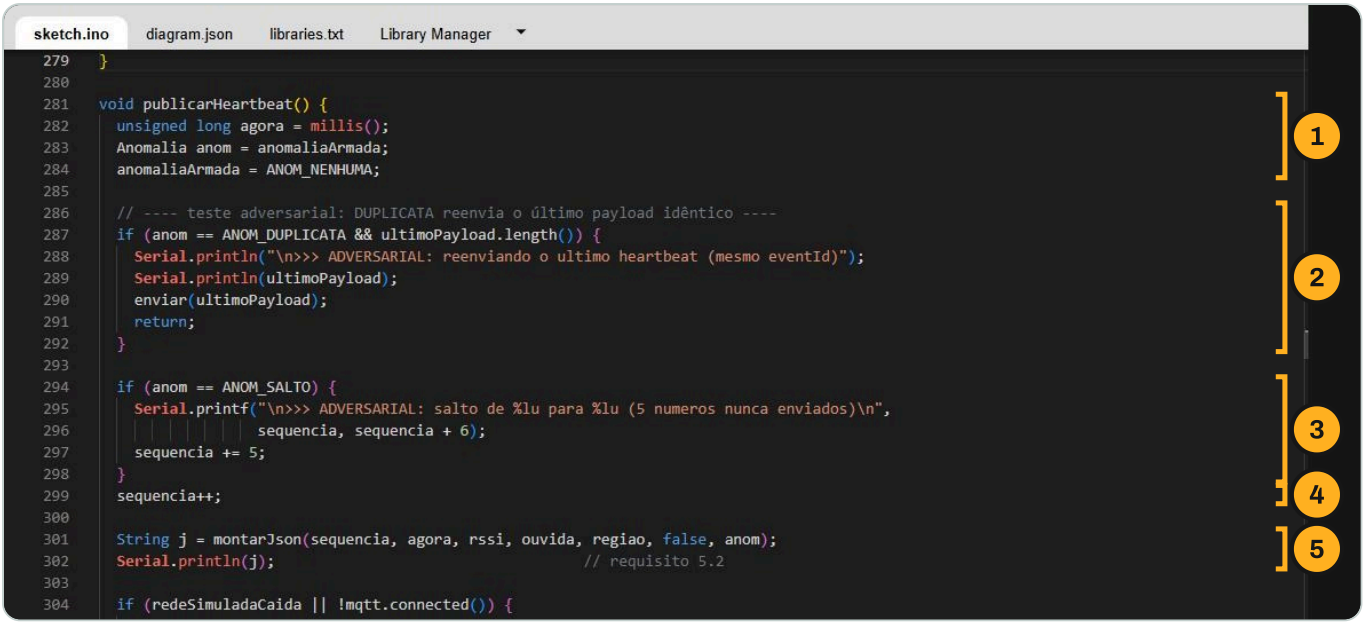
Linhas 215–240: o heartbeat.

1. **montarJson** Monta o texto do heartbeat.
2. **Âncoras** Uma entrada por âncora; `null` quando não foi ouvida.
3. **value** O melhor sinal, ou `null`.
4. **O contrato** Os campos exatamente como no `presenca.heartbeat.v1.schema.json`. Repare no `eventId` = aparelho#boot#sequência.



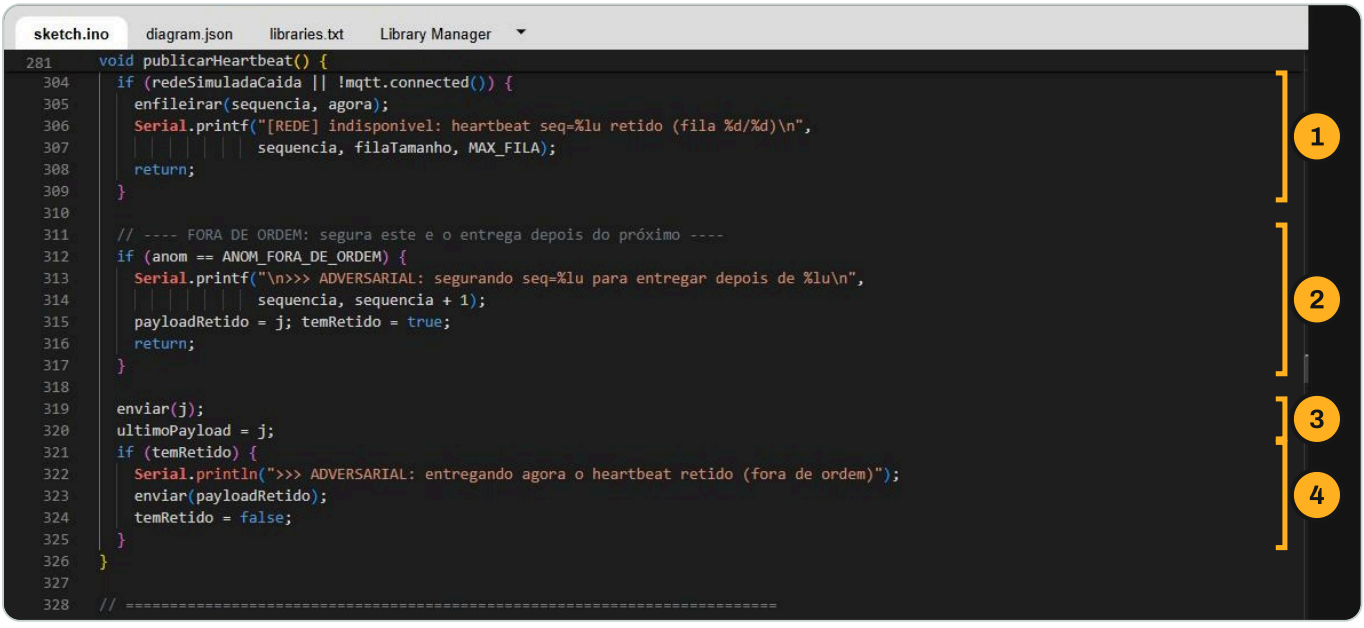
Linhas 245–270: a fila da queda de rede.

1. **enfileirar** Sem rede, guarda o heartbeat (sequência, tempo original, RSSI, região).
2. **Fila cheia** Descarta o *mais antigo* e avisa: o servidor verá um salto.
3. **drenarFila** Quando a rede volta, envia tudo o que estava guardado.
4. **bufferizado** Vai com `bufferizado: true` e o tempo original: o servidor coloca no passado.



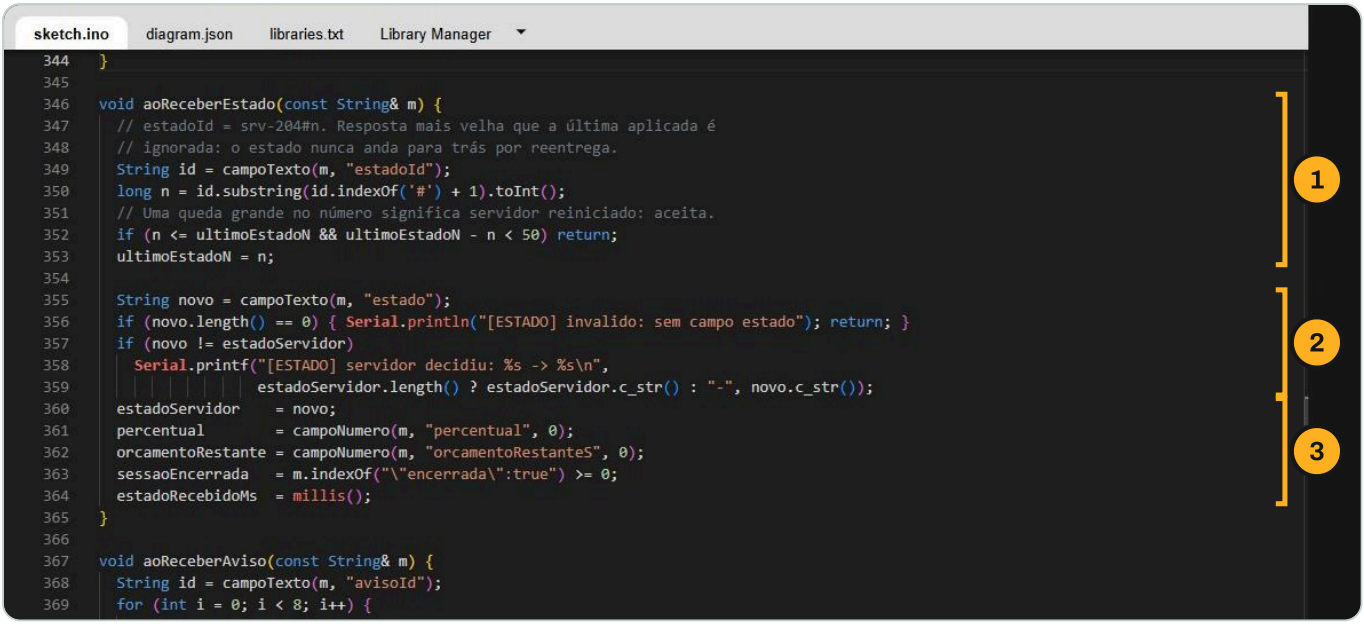
Linhas 279–302: envio e teste adversarial (1).

1. **publicarHeartbeat** Roda a cada 500 ms. Lê qual anomalia está armada.
2. **Duplicata** Reenvia o último heartbeat idêntico (mesmo `eventId`).
3. **Salto** Pula 5 números de sequência.
4. **Sequência** Normal: soma 1.
5. **Monta e escreve** O JSON vai para o monitor serial (exigência da Atividade 03, item 5.2).



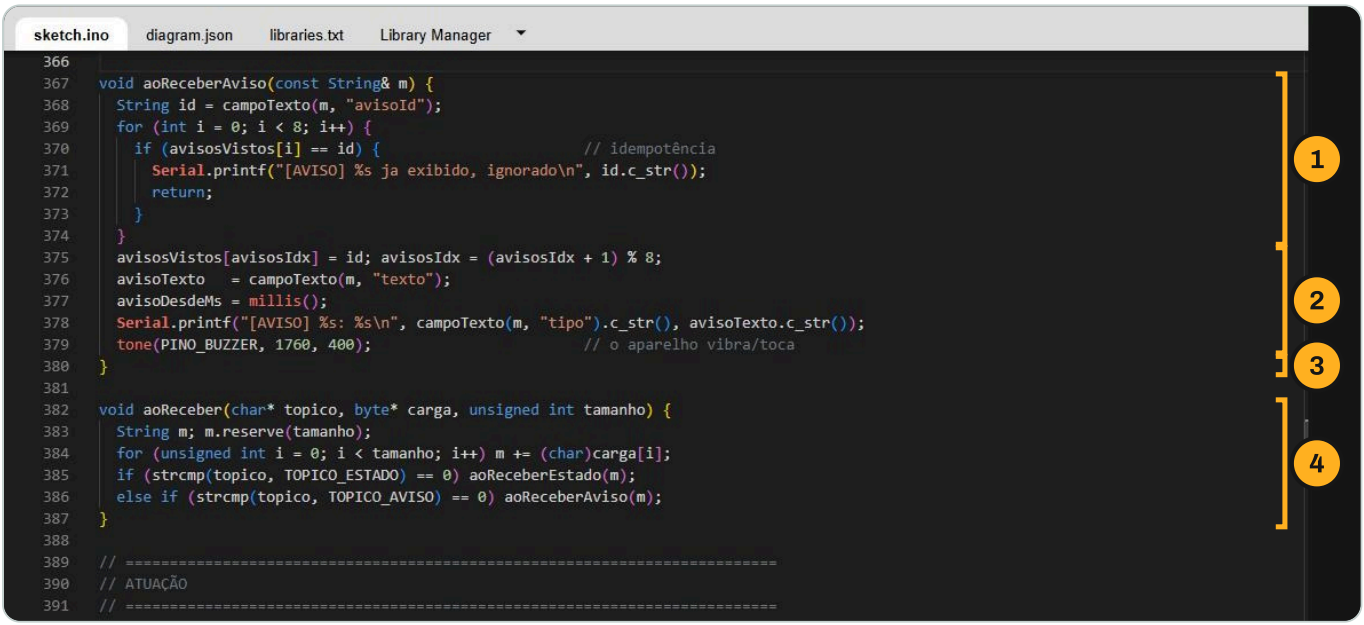
Linhas 304–325: envio e teste adversarial (2).

1. **Sem rede** Guarda na fila e escreve `[REDE] indisponivel … retido`.
2. **Fora de ordem** Segura este heartbeat...
3. **Envio normal** ...envia o próximo...
4. **Entrega o retido** ...e só então entrega o que estava segurando: o N chega depois do N+1.



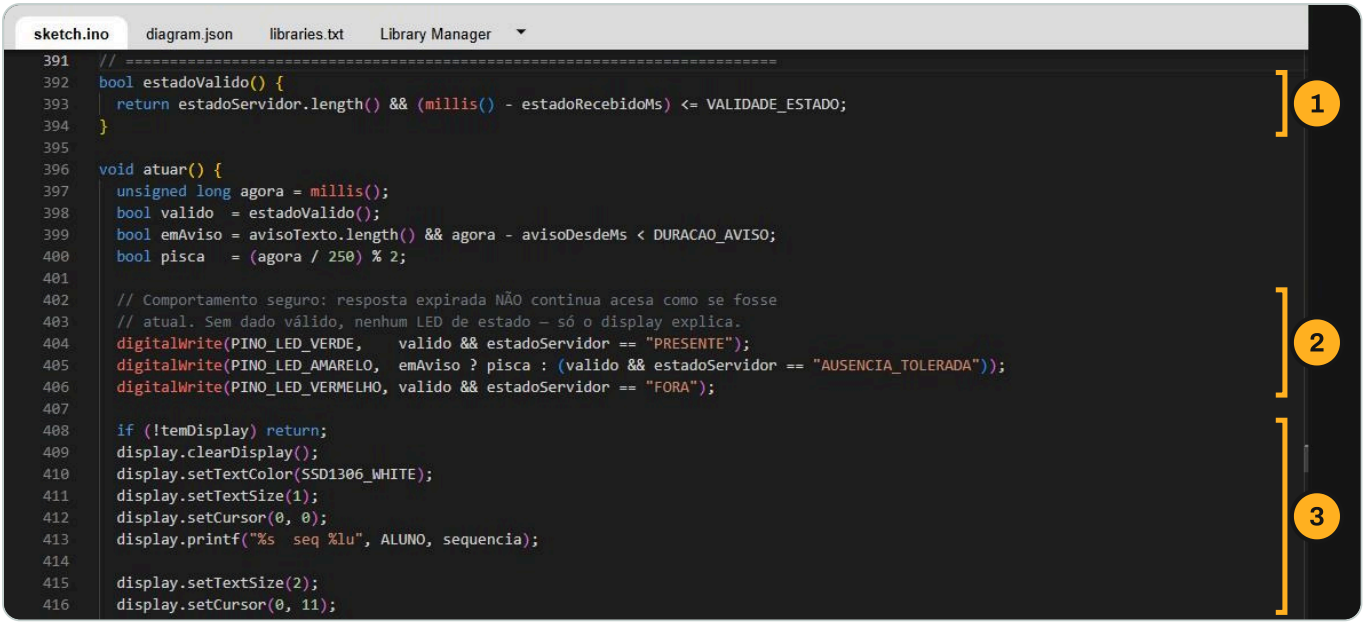
Linhas 344–365: recebendo o estado.

1. **Resposta velha não vale** Cada estado tem número; um número menor que o último é ignorado.
2. **Estado do servidor** Lê o estado e escreve `[ESTADO] servidor decidiu: X -> Y`.
3. **Guarda** Percentual, orçamento, se a aula acabou e *quando* chegou (para saber se ainda vale).



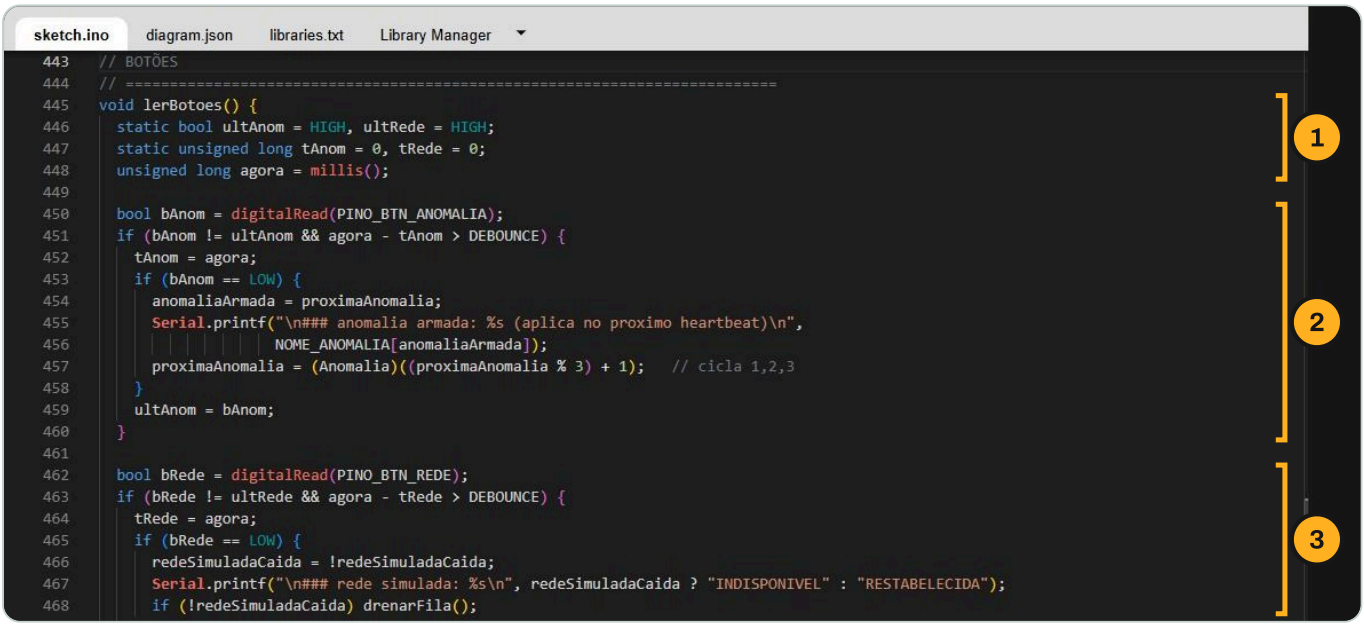
Linhas 366–387: recebendo o aviso.

1. **Idempotência** Guarda os 8 últimos avisos; se o mesmo chegar de novo, ignora.
2. **Mostra** Guarda o texto para o OLED e escreve `[AVISO] …`.
3. **Buzzer** Toca 400 ms: a notificação.
4. **aoReceber** Separa o que chegou: estado ou aviso.



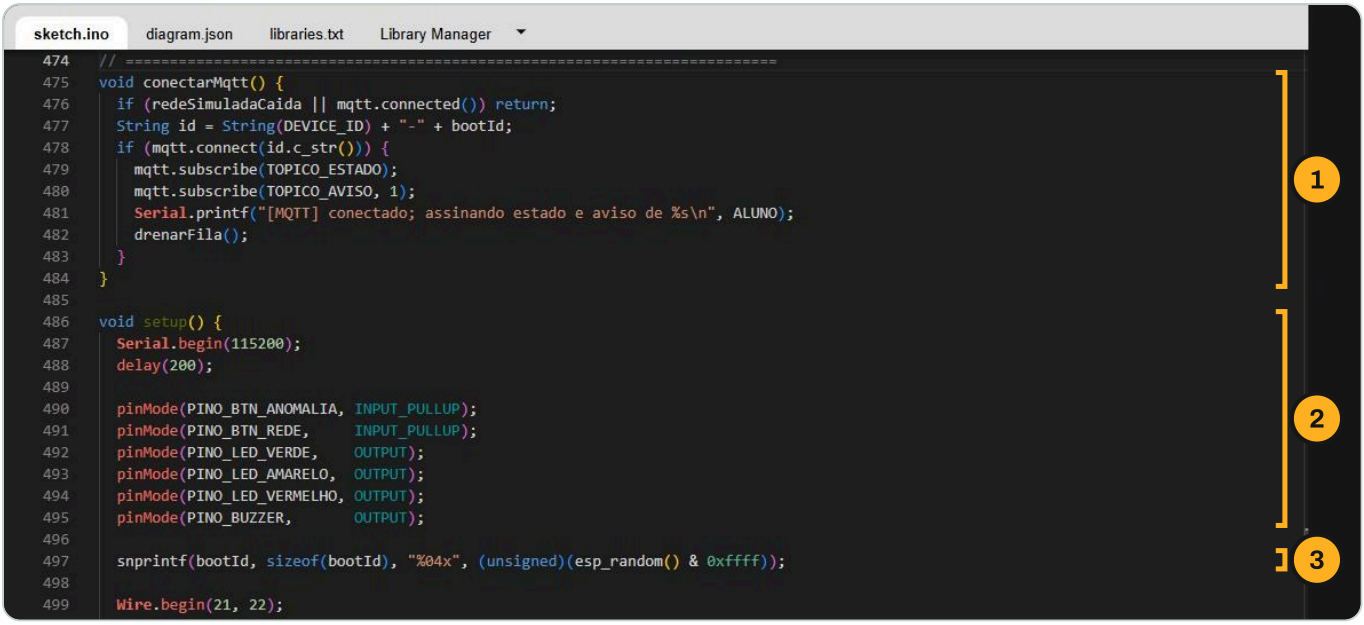
Linhas 391–416: a atuação.

1. **estadoValido** Só vale se chegou há menos de 4 s.
2. **Comportamento seguro** Cada LED só acende se o estado é recente; resposta velha apaga tudo.
3. **Display** Escreve pseudônimo, estado, acumulado, RSSI, região ou o aviso.



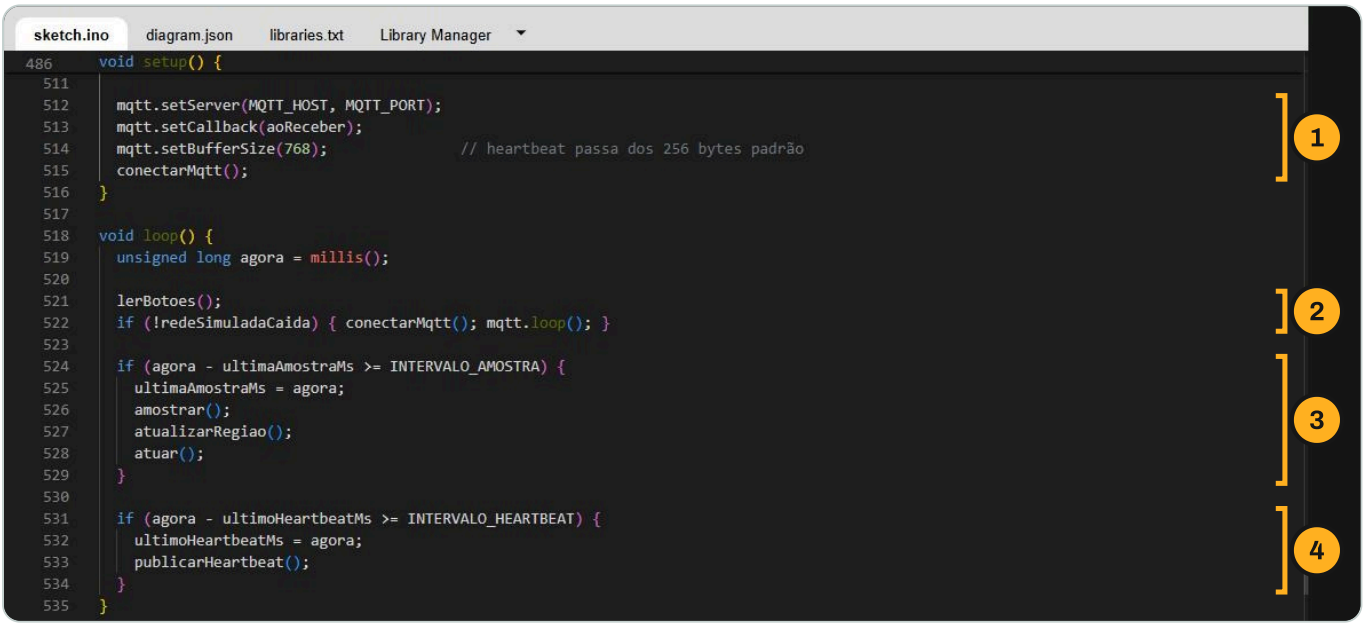
Linhas 443–468: os botões, com debounce.

1. **lerBotoes** Guarda o último estado de cada botão e quando mudou.
2. **Azul** Na borda de descida (apertou), arma a anomalia e passa para a próxima do ciclo.
3. **Vermelho** Alterna a rede; ao voltar, drena a fila.



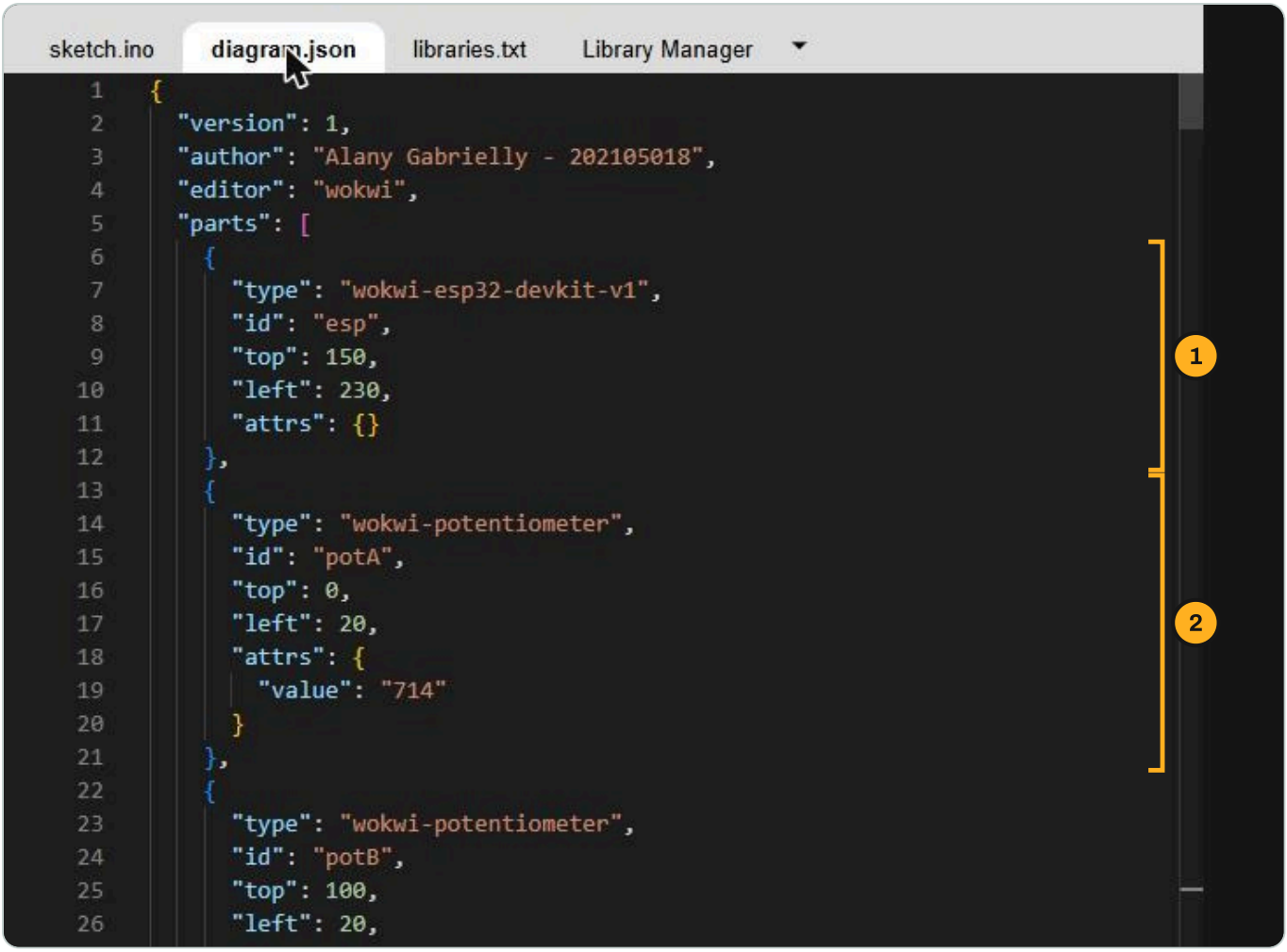
Linhas 474–497: inicialização.

1. **conectarMqtt** Conecta no broker e assina estado e aviso do próprio aluno.
2. **setup: pinos** Botões com pull-up interno; LEDs e buzzer como saída.
3. **bootId** Número aleatório a cada ligação: o reboot não vira duplicata.



Linhas 511–535: o laço principal.

1. **MQTT** Broker, função que recebe e buffer de 768 bytes (o heartbeat passa de 256).
2. **Todo ciclo** Lê os botões e mantém o MQTT vivo.
3. **A cada 100 ms** Amostra, atualiza a região, atua.
4. **A cada 500 ms** Publica o heartbeat.

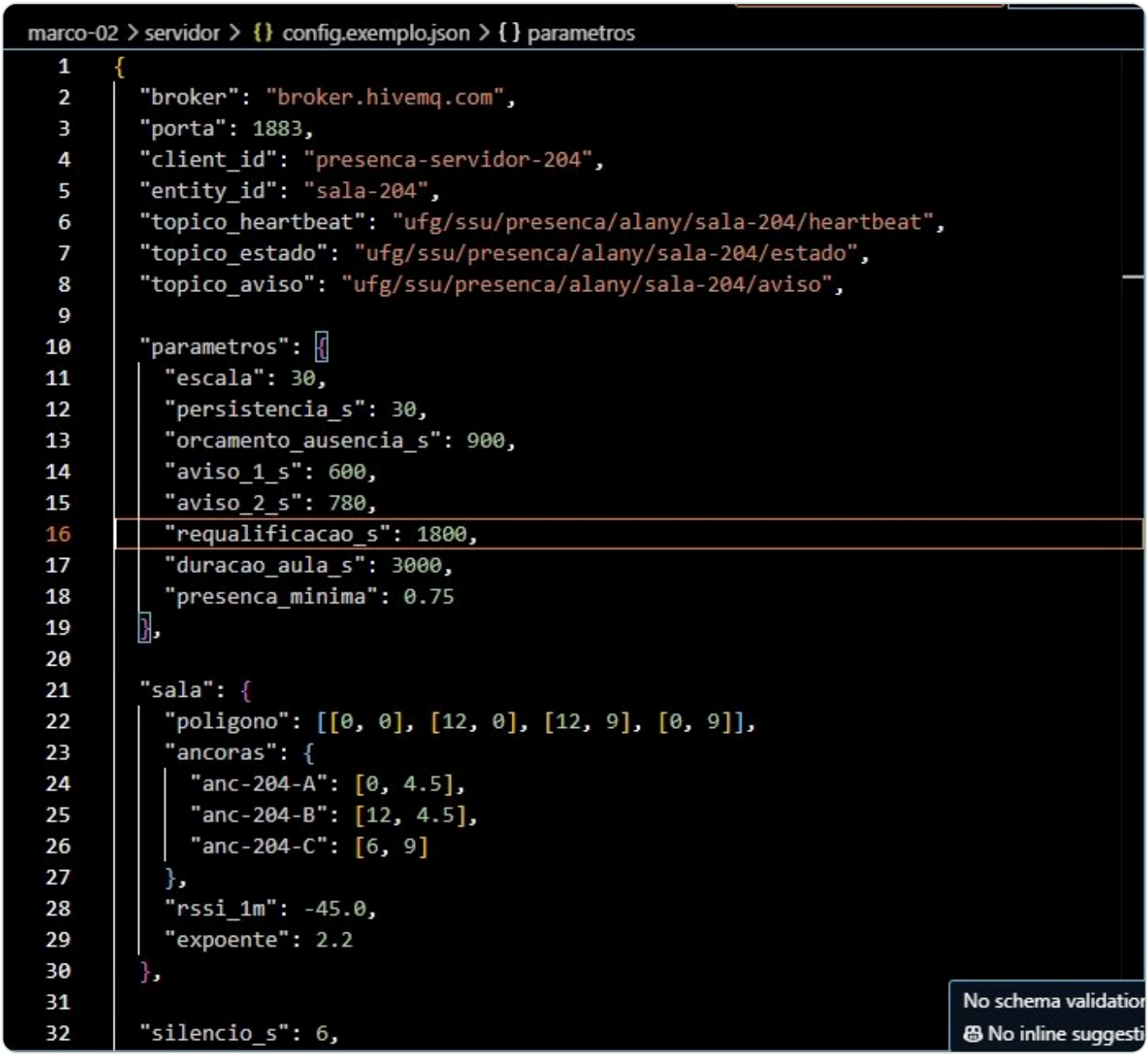


A aba `diagram.json`: o circuito em texto.

1. **Uma peça** Tipo (`wokwi-esp32-devkit-v1`), id, posição na tela.
2. **Potenciômetro A** Com `value: 714`: começa em −56 dBm, dentro da sala.

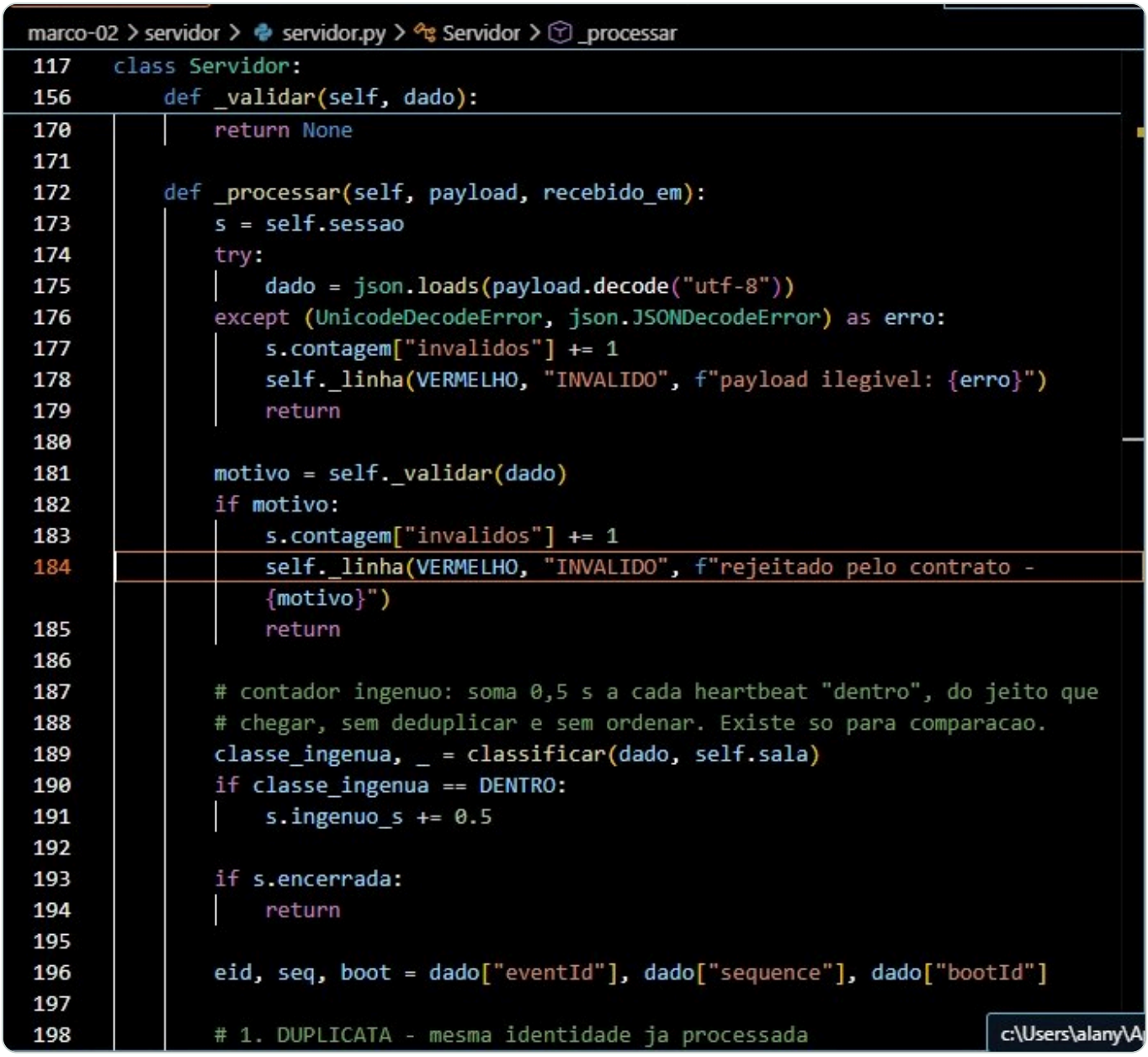
O código do servidor, no VS Code

Abra os arquivos pelo explorador. As legendas citam as linhas do print.



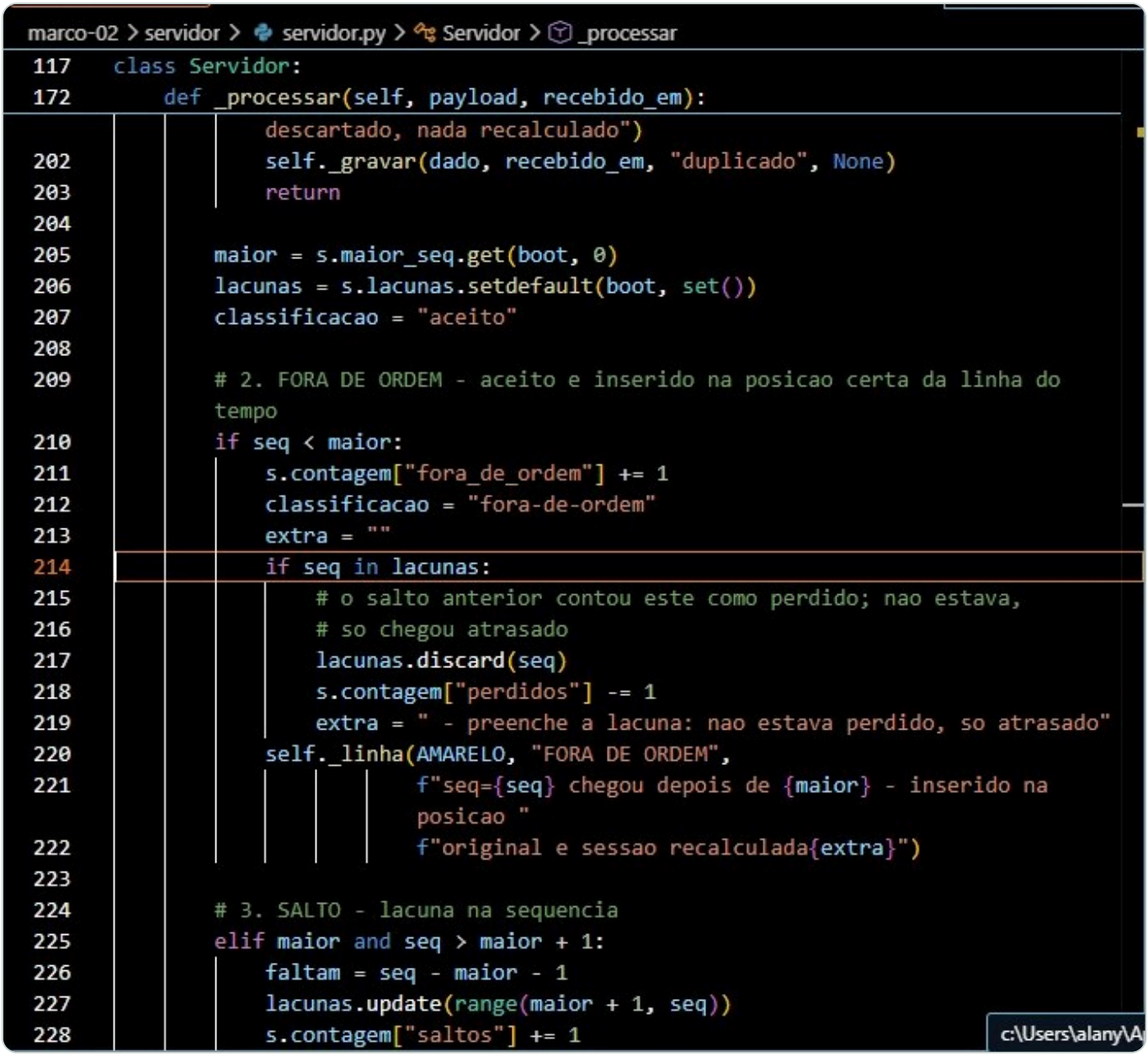
`servidor/config.exemplo.json`

1. **Linhas 2–8** Broker e os três tópicos (heartbeat, estado, aviso).
2. **Linhas 10–19** **Os parâmetros** em valores reais (30 s, 900 s = 15 min...) e a `escala` 30 que divide todos. Mudar `escala` é a alteração ao vivo mais fácil.
3. **Linhas 21–30** A planta: polígono 12×9 m, posição das 3 âncoras e o modelo de propagação.



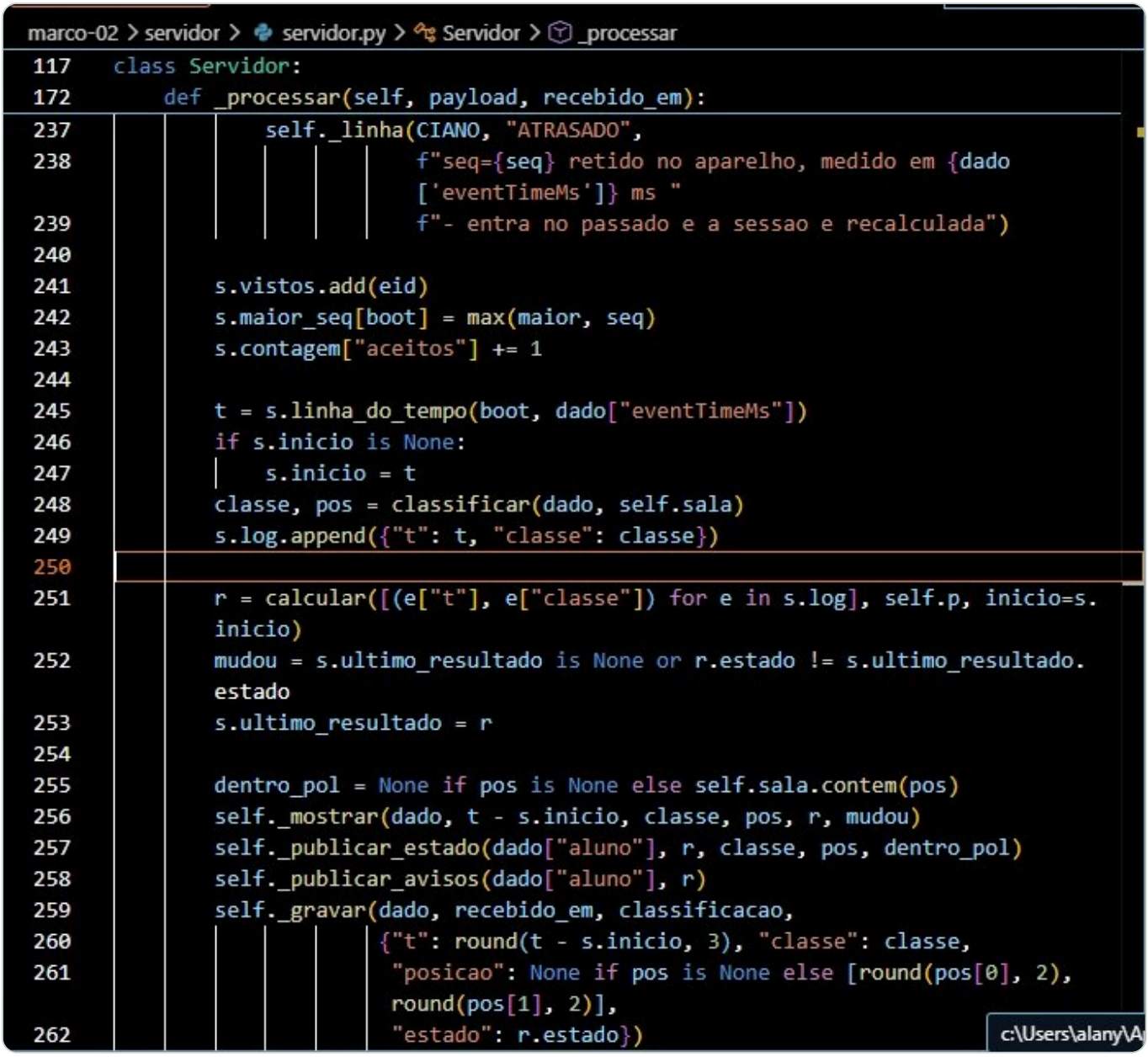
`servidor.py` — início do processamento

1. **Linhas 172–179** Chegou uma mensagem: tenta ler o JSON.
2. **Linhas 181–185** Valida contra o contrato; se falhar, escreve `INVALIDO` e descarta.
3. **Linhas 187–191** O contador ingênuo, só para comparação.
4. **Linha 196** Pega `eventId`, sequência e boot.



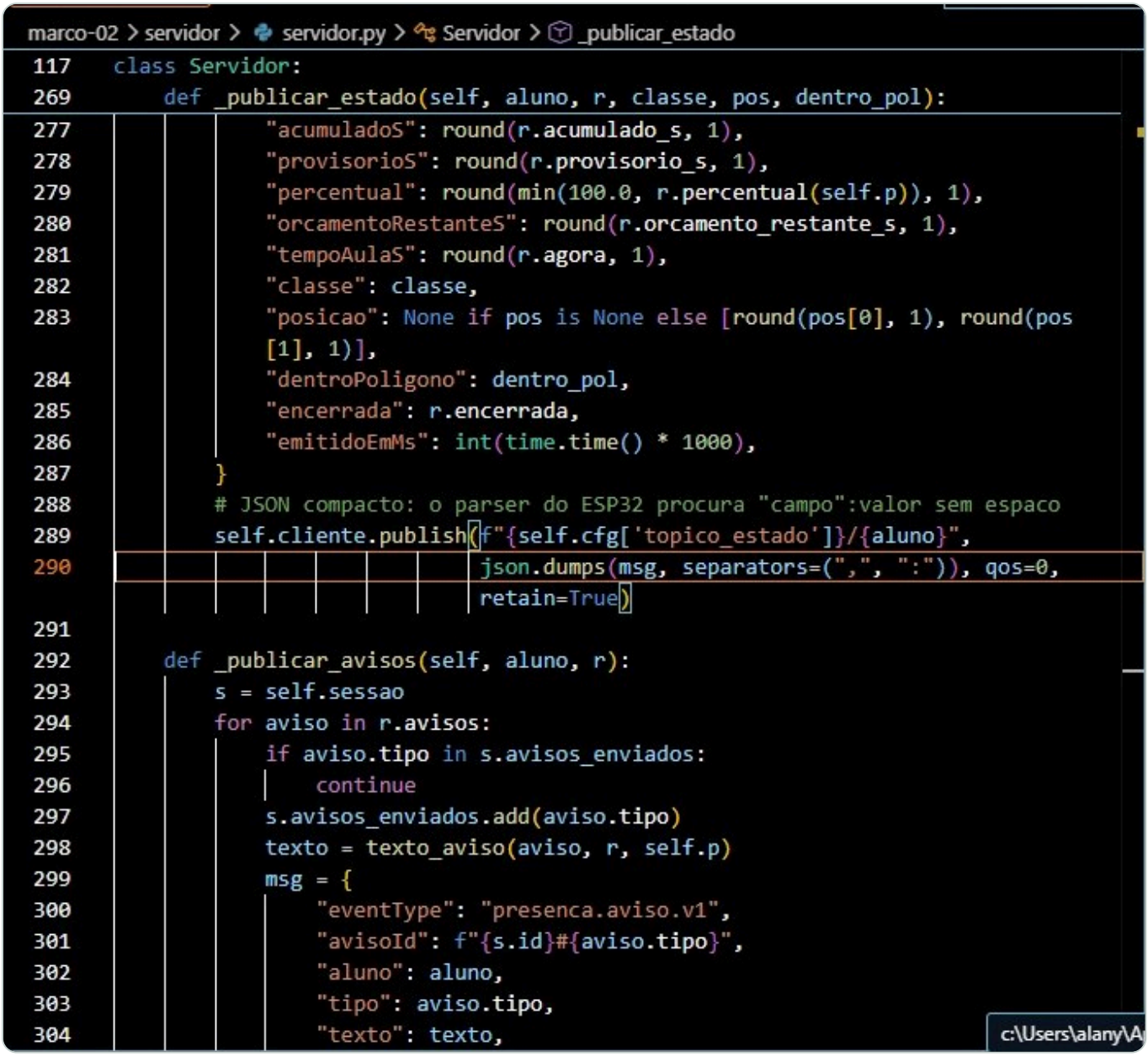
`servidor.py` — as três anomalias

1. **Linhas 198–203 Duplicata:** `eventId` já visto → descarta.
2. **Linhas 209–222 Fora de ordem:** sequência menor que a maior vista → aceita e insere no lugar certo; se preenche uma lacuna, desconta o perdido.
3. **Linhas 224–228 Salto:** sequência pulou → conta os perdidos e guarda a lacuna.



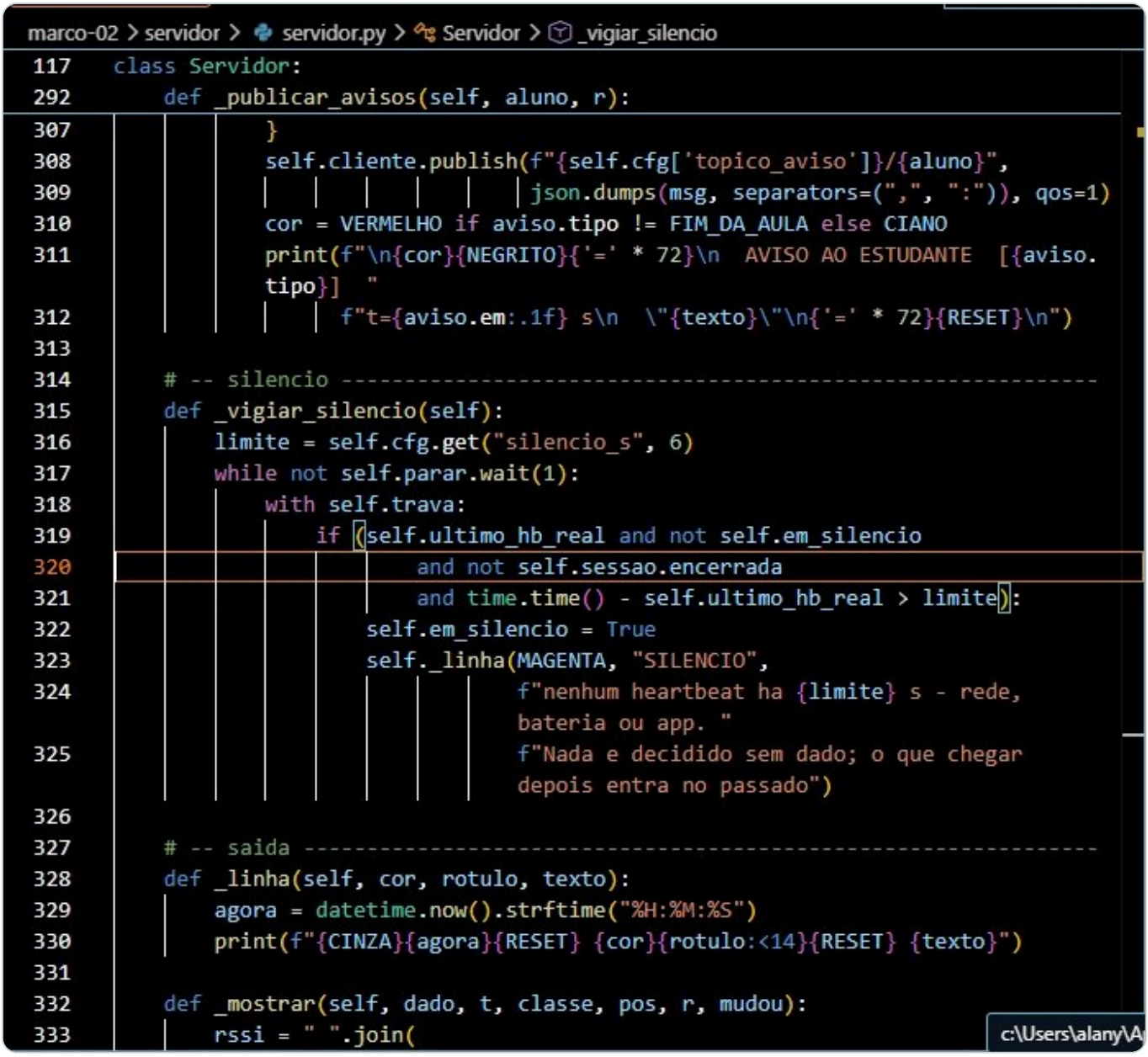
`servidor.py` — do heartbeat à decisão

1. **Linhas 234–239** Atrasado (`bufferizado`): avisa que entra no passado.
2. **Linhas 241–248** Registra, põe na linha do tempo, classifica (dentro/fora/neutro) e guarda no log.
3. **Linha 251** **Refaz a sessão inteira** com `calcular()` .
4. **Linhas 256–258** Mostra no terminal, publica o estado e os avisos.



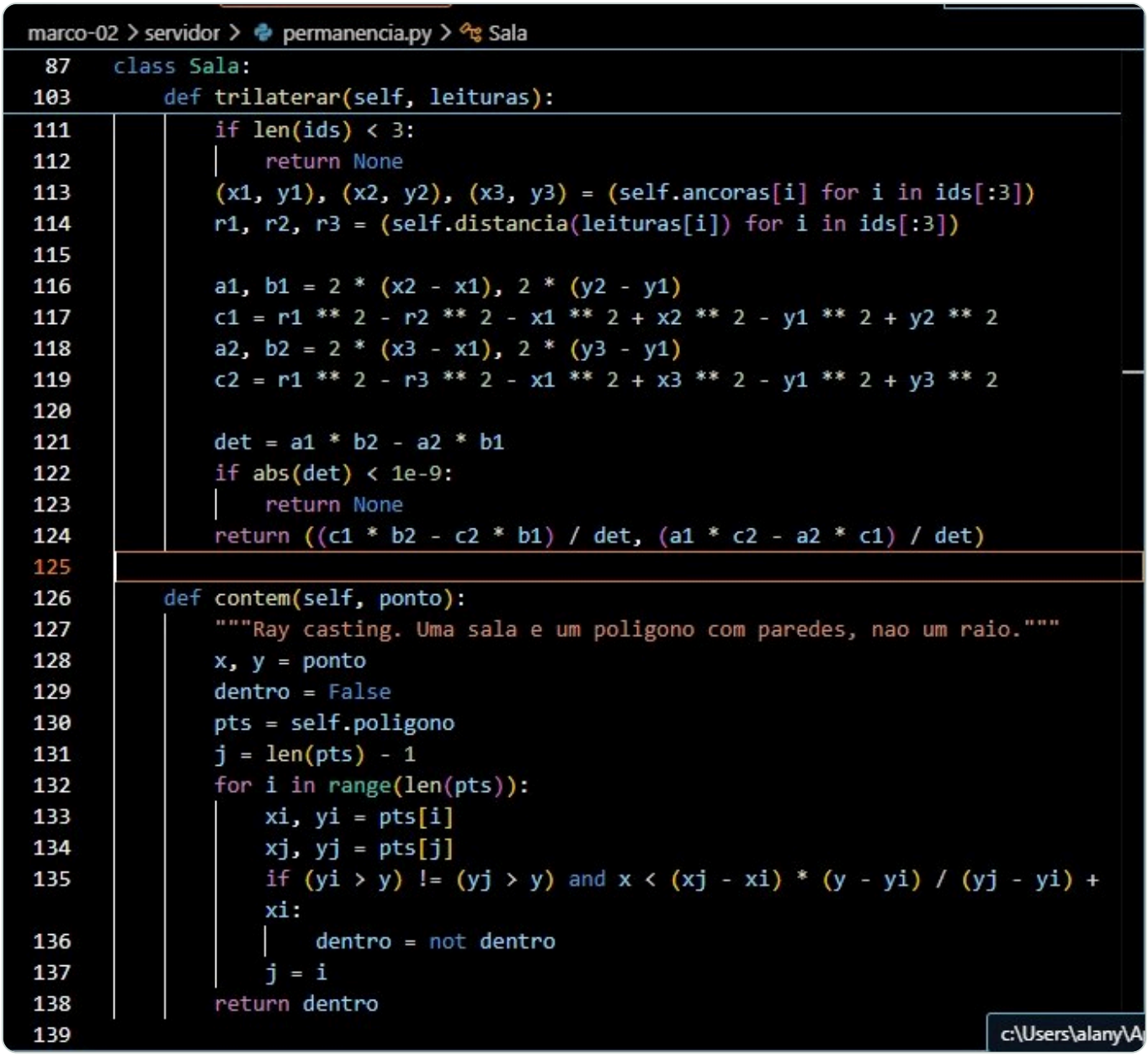
`servidor.py` — respostas ao aparelho

1. **Linhas 269–290** Monta o `presenca.estado.v1` e publica com `retain` no tópico do aluno.
2. **Linhas 292–310** Para cada aviso novo: monta o `presenca.aviso.v1`, publica com QoS 1 e imprime a faixa vermelha no terminal.



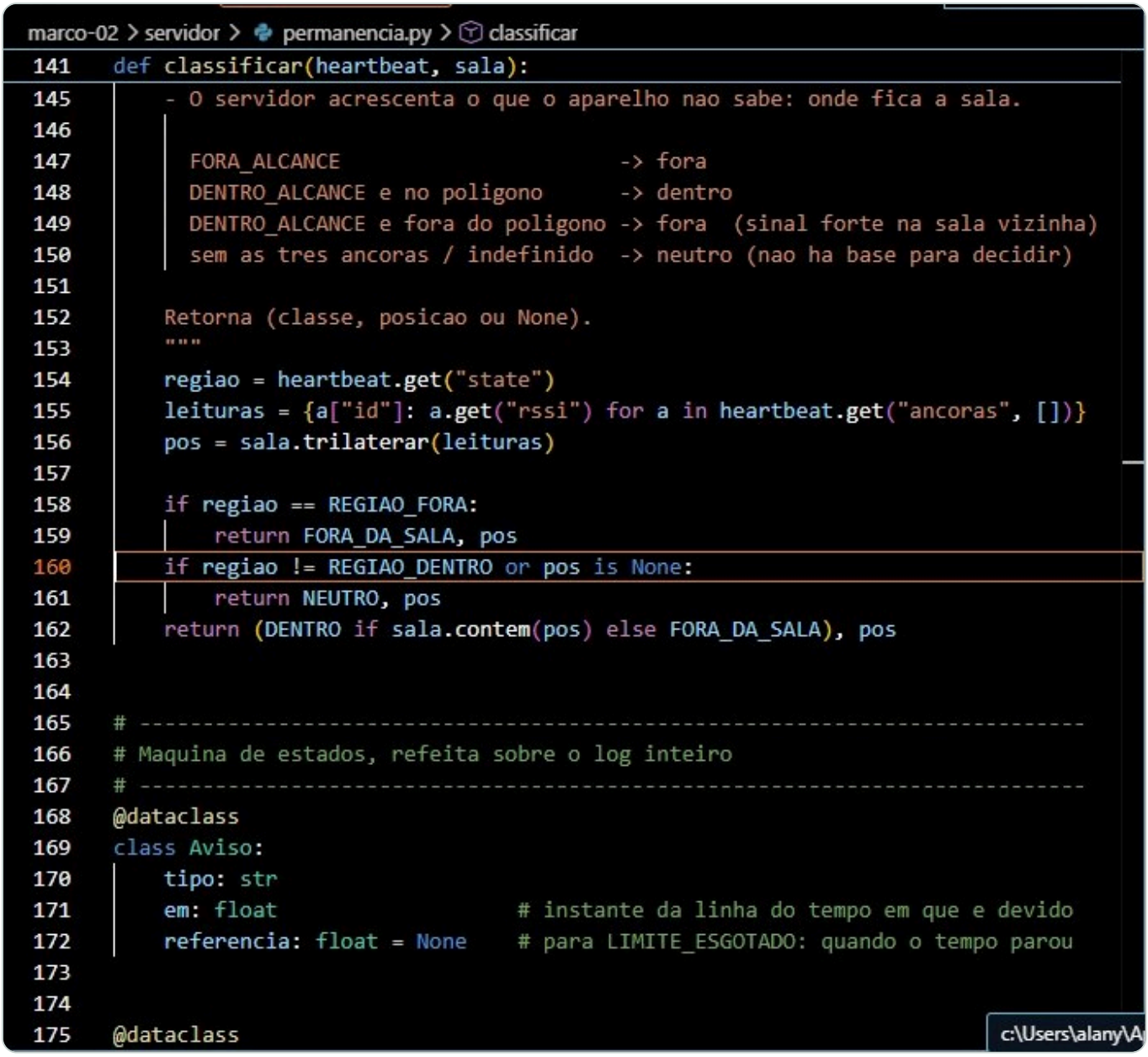
`servidor.py` — silêncio

1 **Linhas 315–325** A cada segundo olha o relógio real; sem heartbeat há 6 s, escreve `SILENCIO` e não decide nada.



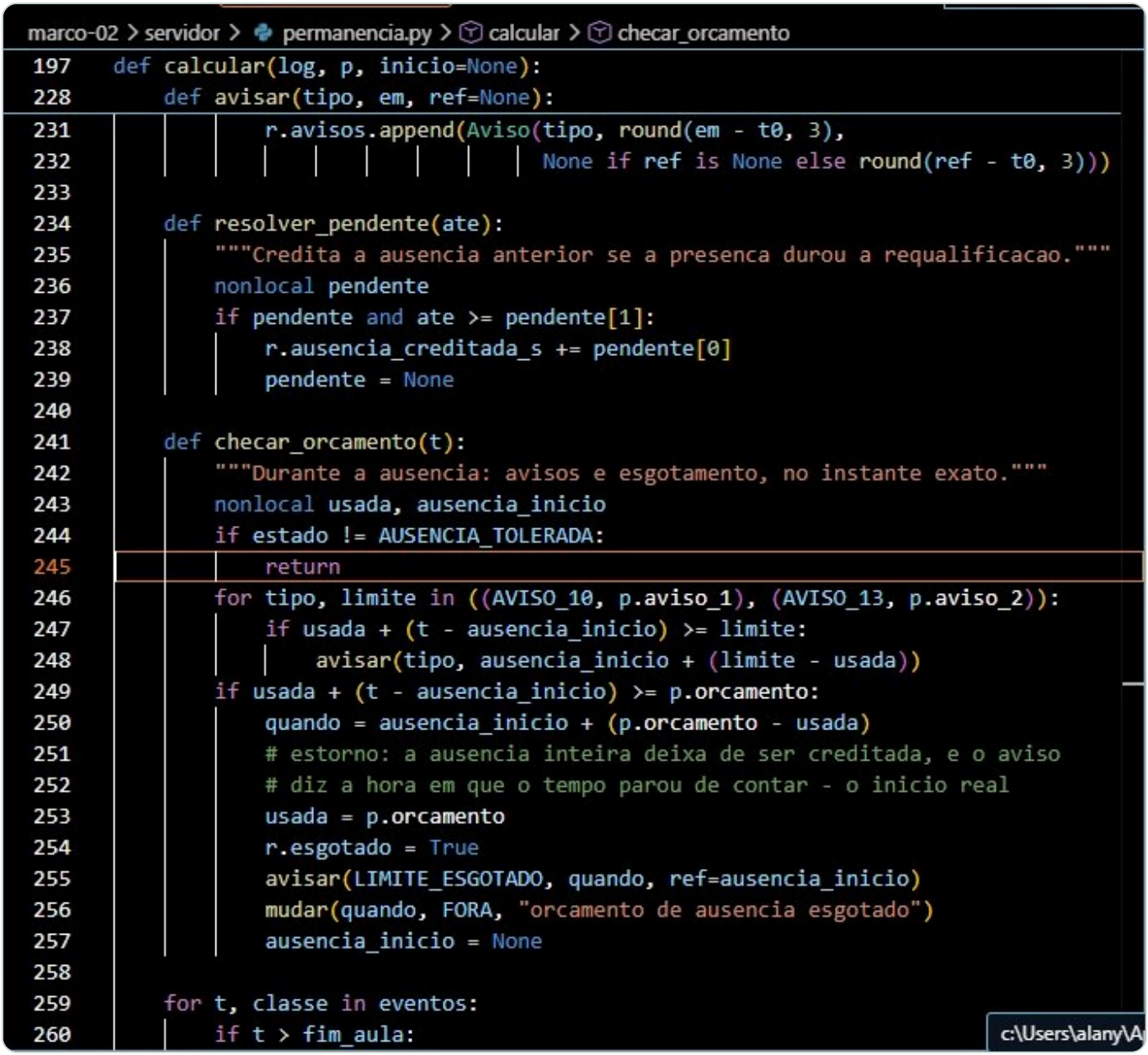
`permanencia.py` — trilateração e polígono

1. **Linhas 103–124 Trilateração**: RSSI vira distância; três círculos viram um sistema linear; resolve (x, y).
2. **Linhas 126–138 Polígono**: conta quantas paredes uma semirreta atravessa (ímpar = dentro).



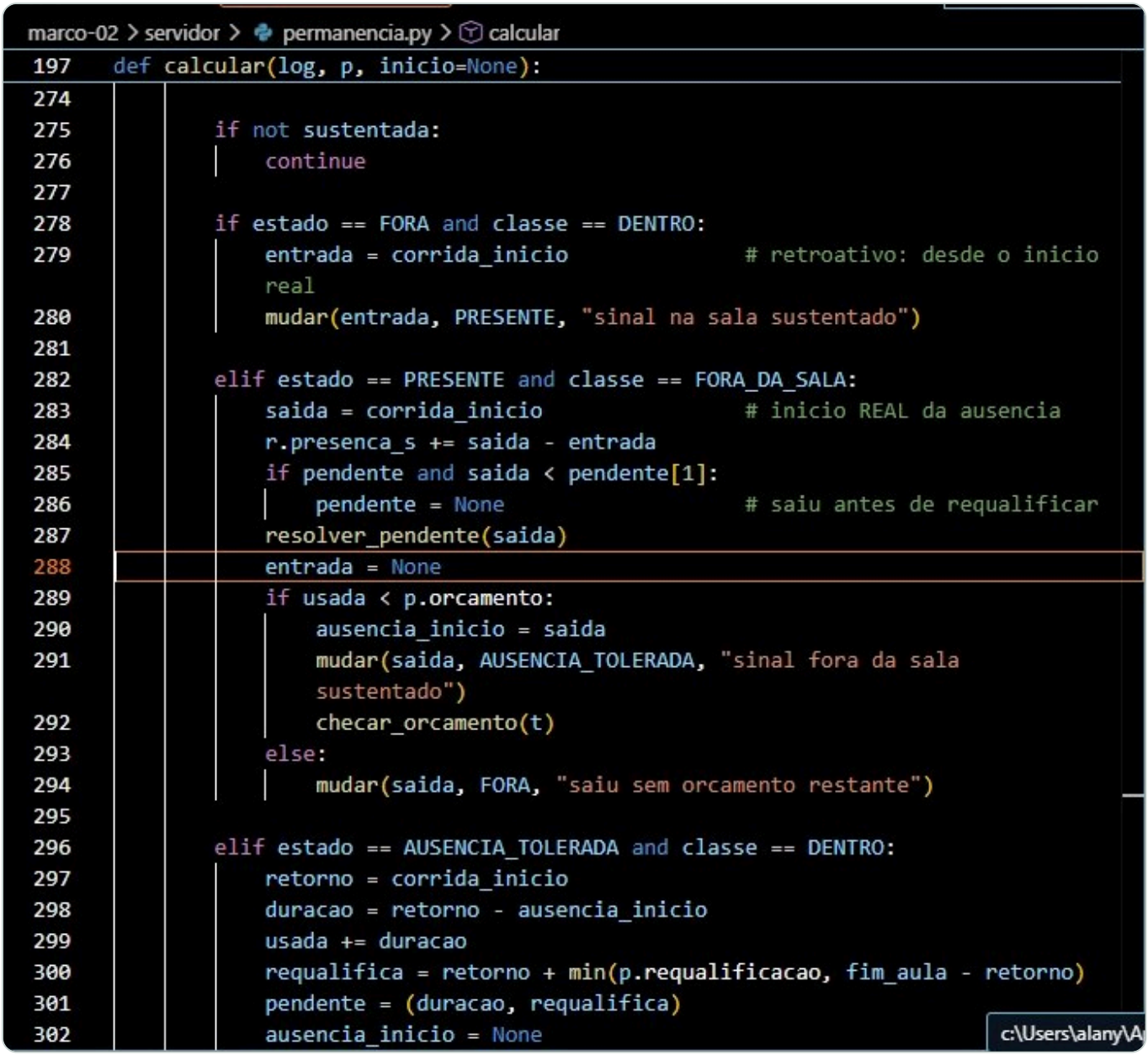
`permanencia.py` — o que um heartbeat diz

1. **Linhas 141–153** A tabela da decisão, escrita no comentário.
2. **Linhas 155–162** Fora do alcance → fora; sem 3 âncoras → neutro; senão, o polígono decide.



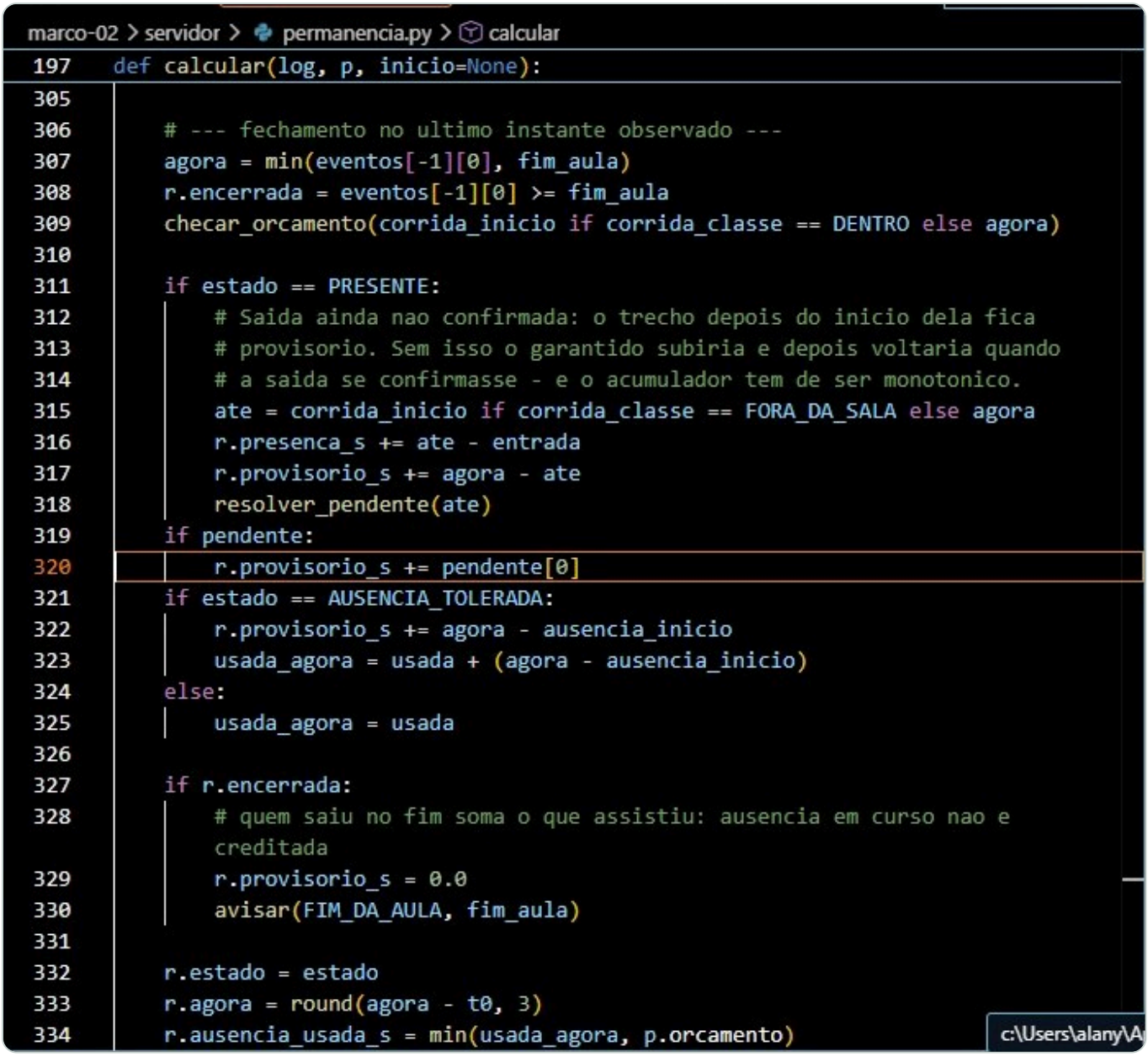
`permanencia.py` — orçamento

1. **Linhas 234–239** Credita a ausência anterior só se a presença durou a requalificação.
2. **Linhas 241–257** Durante a ausência: avisos de 20 s e 26 s e, ao esgotar, **estorno** e FORA.



`permanencia.py` — a máquina de estados

1. **Linhas 278–280** FORA → PRESENTE, datado no início real da condição.
2. **Linhas 282–294** PRESENTE → AUSÊNCIA (se há orçamento) ou → FORA (se não há).
3. **Linhas 296–304** AUSÊNCIA → PRESENTE: consome o orçamento e deixa a ausência pendente de requalificação.



`permanencia.py` — fechamento

1. **Linhas 306–318** No último instante: o que ainda não se confirmou vira *provisório*; o garantido nunca cai.
2. **Linhas 327–330** Fim da aula: ausência em curso não é creditada; emite o aviso final.

G9 Os testes ao vivo, um por um

A aula dura 100 s *simulados*, o que dá uns 4 a 8 minutos reais. Faça os testes 1 a 4 numa sessão só, na ordem. O teste 5 usa uma sessão nova.

Antes do teste 1

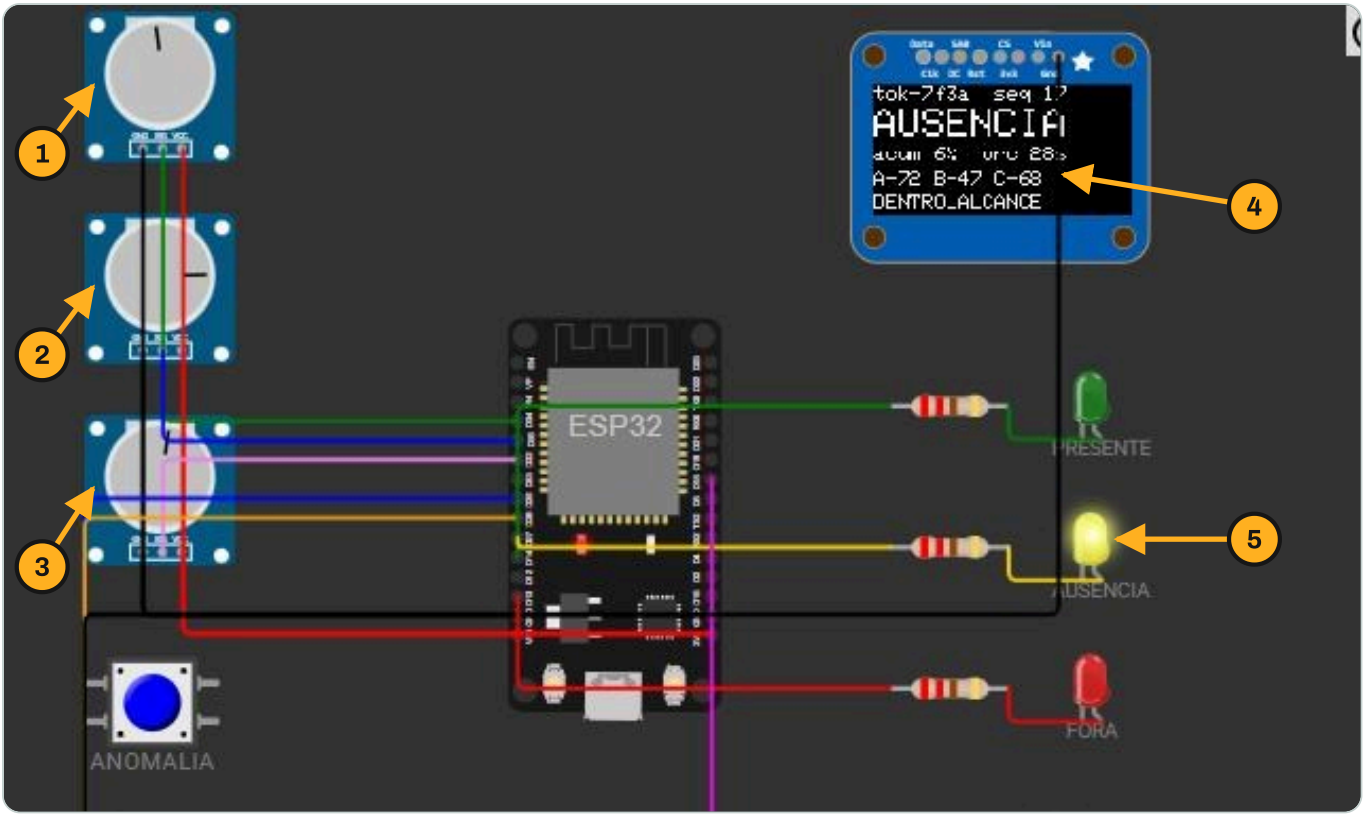
Potenciômetros na posição “dentro da sala”. ▶ no Wokwi. Quando o serial mostrar `[MQTT] conectado`, digite `n` + Enter no servidor.

1 Estado normal

Faça	Nada. Só espere 3 ou 4 heartbeats.
No Wokwi	OLED `PRESENTE`, LED verde, `A-56 B-56 C-56`, `DENTRO_ALCANCE`. No serial, os heartbeats em JSON e `[ESTADO] servidor decidiu: FORA -> PRESENTE`.
No servidor	`>>> FORA -> PRESENTE em t=… (sinal na sala sustentado)` e o acumulado subindo.
Fale	“O aparelho mede as três âncoras e decide a região com histerese. O servidor trilatera, vê que o ponto está dentro do polígono e confirma a presença depois da persistência. A presença conta desde o começo da condição, não desde a confirmação: o cálculo é retroativo.” Aponte o `eventId` no serial e a mesma sequência no servidor.

2 Decisão e atuação: corredor, avisos e volta

Faça	Potenciômetros na posição **corredor**: A = Home + PgUp×5, B = Home + PgUp×8, C = Home + PgUp×7. Espere o buzzer duas vezes (≈20 s e ≈26 s de ausência). **Logo depois do segundo aviso**, volte os três para "dentro" (Home + PgUp×7).
No Wokwi	A região continua `DENTRO_ALCANCE` (sinal forte!), mas o estado vira `AUSENCIA` com LED amarelo. Nos avisos, buzzer, LED amarelo piscando e o texto no OLED. Na volta, LED verde.
No servidor	Posição `(13.9, 12.4) fora`; `PRESENTE -> AUSENCIA_TOLERADA`; as faixas vermelhas `AVISO_10_MIN` e `AVISO_13_MIN`; depois `AUSENCIA_TOLERADA -> PRESENTE (retornou dentro do orcamento)`. O orçamento para de cair.
Fale	"O sinal é forte, então um sistema com raio em metros contaria presença. Mas os três círculos se cruzam do lado de fora da parede: o polígono diz não. O aviso chega enquanto ainda dá para voltar. Eu voltei, então não fui punida. É isso que fecha o laço ciber-físico: a notificação muda a posição, que é o que o sistema mede."
Se passar do tempo	Se não voltar a tempo, vira o teste 5 (orçamento esgotado). Tudo bem: explique o estorno.



Sinal forte vindo de fora da sala: o servidor decide AUSÊNCIA.

1. **A** Mais fraco (a âncora A está longe, do outro lado da sala).
2. **B** Forte: o estudante está perto da parede leste, mas do lado de fora.
3. **C** Médio.
4. **OLED** `AUSENCIA`, com a região ainda `DENTRO_ALCANCE`: o aparelho acha que está no alcance, mas o servidor trilaterou e o ponto caiu fora do polígono.
5. **LED amarelo** AUSÊNCIA TOLERADA: o tempo ainda conta, mas consome o orçamento.

3 Adversarial (matrícula final 8): repetido, fora de ordem, salto

Faça	Botão **azul**, segurando 1 s. Espere uns 5 s. De novo. Espere. De novo. (Três apertos.)
No serial	`### anomalia armada: duplicata` → `>>> ADVERSARIAL: reenviando o ultimo heartbeat`; depois `fora-de-ordem` → `segurando seq=N` e `entregando agora o heartbeat retido`; depois `salto` → `salto de N para N+6`.
No servidor	`DUPLICADO … descartado, nada recalculado`; `FORA DE ORDEM … inserido na posicao original … preenche a lacuna`; `SALTO … 5 heartbeat(s) perdido(s)`. O estado não muda.
Fale	As três perguntas da atividade: **por que o ingênuo erra** (somaria a duplicata duas vezes, voltaria no tempo com o atrasado, trataria o buraco como observado); **o mecanismo** (identidade `aparelho#boot#sequência`, lacunas e recálculo sobre o log ordenado); **o que foi observado** (as três linhas acima, sem mudar o estado).

4 Queda de rede

Faça	Botão **vermelho** (1 s). Espere o servidor escrever `SILENCIO` e mais uns 10 s. Botão vermelho de novo.
No Wokwi	OLED `REDE OFF` e `fila N/60 retidos`; **todos os LEDs apagam.** No serial, `[REDE] indisponivel: heartbeat seq=… retido`. Na volta, `[RETRY] N heartbeat(s) retidos entregues com atraso`.
No servidor	`SILENCIO … Nada e decidido sem dado`; depois `RETOMADO` e várias linhas `ATRASADO … entra no passado e a sessao e recalculada`. O acumulado continua sem buraco.
Fale	“Silêncio não é ausência: pode ser rede, bateria ou o app. O servidor não decide nada sem dado. O aparelho guarda tudo com o horário original e, quando a rede volta, o servidor encaixa no passado e refaz a sessão. E os LEDs apagam porque mostrar um ‘presente’ velho seria mentir.”

5 Orçamento esgotado (sessão nova)

Faça	Potenciômetros “dentro”, `n` no servidor, espere o `PRESENTE`. Depois os três em **Home** (fora do prédio) e deixe assim por mais de 30 s de simulação.
No Wokwi	`A-- B-- C--`, `FORA_ALCANCE`, `AUSENCIA`; os dois avisos; depois `FORA`, LED vermelho e “O tempo parou de ser contabilizado em t=...”.
No servidor	`LIMITE_ESGOTADO`, `orc 0.0s` e o acumulado **igual ao que era antes de sair**.
Fale	“O orçamento é de 15 minutos por aula, não por saída. Acabou sem volta: a ausência inteira é estornada, e o aviso diz a hora em que o tempo parou de contar, que é quando a pessoa saiu. É esse horário que o estudante usaria para contestar.”

No fim da aula (t = 100 s)

O aparelho mostra “Voce acumulou N% desta aula.” e o servidor imprime o **RELATÓRIO FINAL**, com presença, ausência creditada, aprovado ou não, e o contador ingênuo para comparar. Use `p` a qualquer momento para mostrar o painel.

G10 Mostrar o contrato funcionando

Num segundo PowerShell (o primeiro está ocupado com o servidor):

PowerShell (segundo terminal)

```
cd "D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA\projeto-presenca\marco-02\contrato"
python validar.py
```

o que aparece (saída real)

```
Validando 10 mensagem(ns) contra contrato/v1/

  OK  ambiente-bufferizado.json        sala.ambiente.v1
  OK  ambiente-coerente.json           sala.ambiente.v1
  OK  ambiente-incoerente.json         sala.ambiente.v1
  OK  aviso-10min.json                 presenca.aviso.v1
  OK  contagem-comando.json            gateway.contagem.v1
  OK  estado-ausencia.json             presenca.estado.v1
  OK  heartbeat-bufferizado.json       presenca.heartbeat.v1
  OK  heartbeat-dentro.json            presenca.heartbeat.v1
  OK  heartbeat-fora-do-alcance.json   presenca.heartbeat.v1
  OK  heartbeat-sala-vizinha.json      presenca.heartbeat.v1

Agora de proposito: o mesmo heartbeat com um campo que o contrato nao preve

  NAO heartbeat-com-bateria (inventado)  presenca.heartbeat.v1 -> (raiz): Additional properties are not allowed

  -> o servidor faz o mesmo: responde INVALIDO e descarta. Para aceitar
     'bateria' seria preciso criar presenca.heartbeat.v2.
```

Fale: “O contrato não é documentação: o servidor lê este arquivo e rejeita o que não bate. Uma mudança incompatível vira v2.” Abra `presenca.heartbeat.v1.schema.json` e mostre `additionalProperties: false` e o `pattern` do `eventId`.

G11 Mostrar os testes automáticos

PowerShell (segundo terminal)

```
cd "D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA\projeto-presenca\marco-02\servidor"
python -m pytest testes -v
```

Aparecem os 34 nomes com `PASSED` e, no fim, `34 passed`. Com `-q` no lugar de `-v` sai só o resumo. Destaque três nomes: `test_ordem_de_chegada_nao_altera_o_resultado` (embaralha o log inteiro e dá o mesmo resultado), `test_sinal_forte_na_sala_vizinha_nao_conta` e `test_orcamento_e_por_sessao_nao_por_evento`.

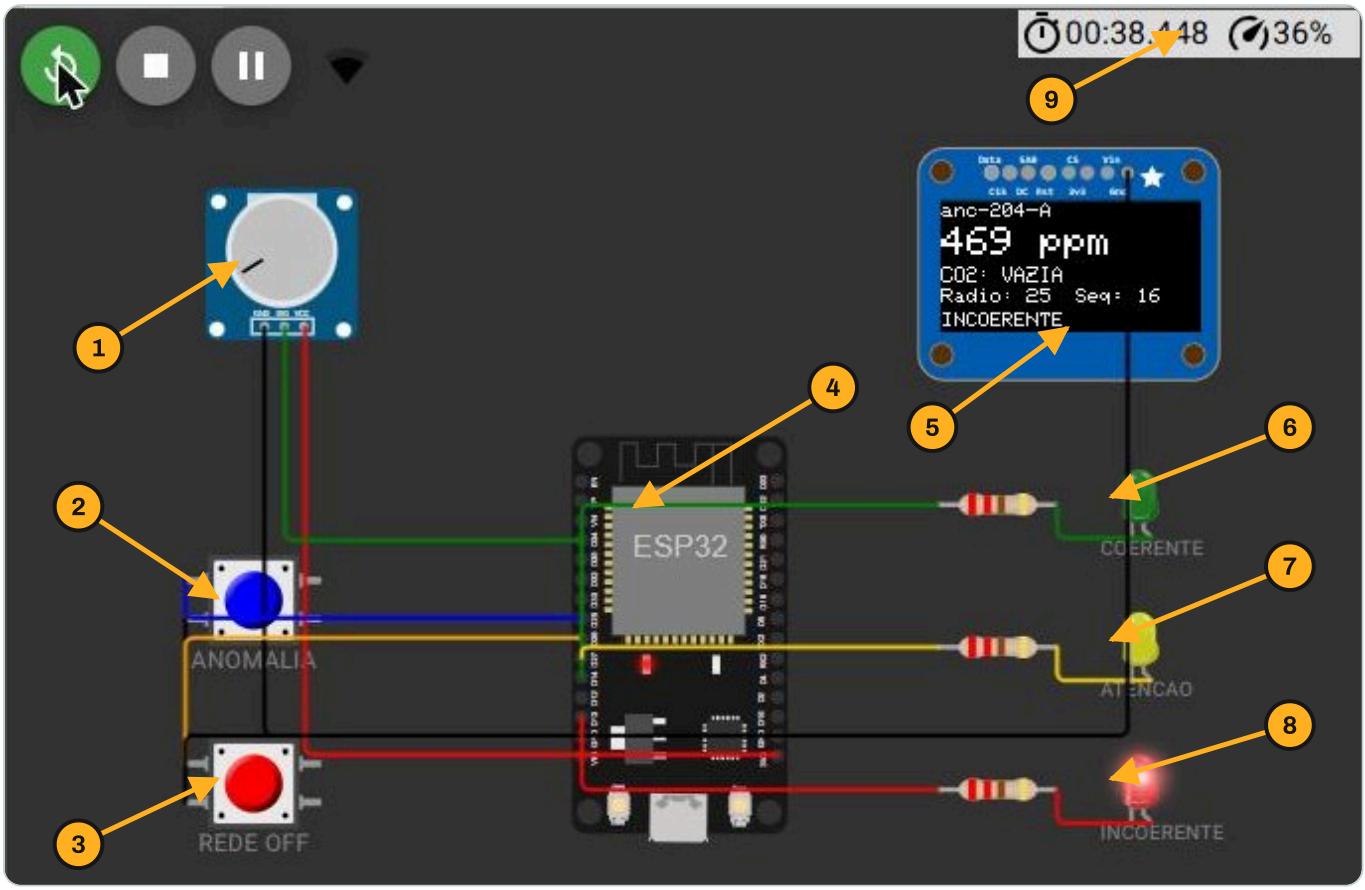
G12 A fronteira de CO_2 (o “gateway”)

Pode rodar ao mesmo tempo que o servidor de presença: usam tópicos diferentes.

PowerShell (segundo terminal) · gateway de CO_2

```
cd "D:\SOFTWARE PARA COMPUTAÇÃO UBÍQUA\projeto-presenca\marco-02\co2-gateway"
python consumidor.py      # ou: python consumidor.py --ws
```

Abra **wokwi.com/projects/476001394839038977** e dê ▶.



O projeto de CO_2 (**wokwi.com/projects/476001394839038977**) acusando incoerência.

1. **Potenciômetro** CO_2 simulado, de 380 a 2200 ppm.
2. **Botão azul** Anomalia de sequência (duplicata → fora de ordem → salto).
3. **Botão vermelho** Rede fora / de volta.
4. **ESP32** A âncora da sala.
5. **OLED** ppm, ocupação pelo CO_2, contagem do rádio, sequência, estado.
6. **LED verde** COERENTE.
7. **LED amarelo** ATENÇÃO (dados insuficientes ou transição).
8. **LED vermelho** INCOERENTE: o rádio diz 25 estudantes, o CO_2 diz sala vazia.
9. **Relógio** Tempo simulado.

NO GATEWAY, DIGITE	EFEITO
um número (ex. `0` ou `25`) + Enter	Quantos estudantes o rádio declara. Vai para a âncora.
`p`	Painel
`r`	Zera os contadores (use depois de reiniciar a simulação)
`q`	Sai

1 **Normal**

Digite `0`. Potenciômetro em qualquer posição: LED verde, `COERENTE`.

2 **Decisão**

Digite `25` e deixe o potenciômetro baixo (CO_2 de sala vazia). Em ~10 s: LED vermelho e o **ALERTA DE INCOERÊNCIA** no gateway.

3 **Adversarial**

Botão azul duas vezes: `DUPLICADO` e `FORA DE ORDEM`.

gateway de CO_2 · saída real de 25/09/2026

```
02:19:26 DUPLICADO     anc-204-A#4 ja processado - descartado sem efeito no estado
02:19:42 SALTO         esperado 6, recebido 7 - 1 evento(s) perdido(s)
02:19:43 FORA DE ORDEM  seq=6 chegou depois de 7 (1 atras) - aceito, nao retrocede o estado
02:19:59 seq=10         468.9 ppm  co2=VAZIA       radio=25  INCOERENTE
======================================================================
 ALERTA DE INCOERENCIA DE OCUPACAO
 O radio declara 25 estudantes,
 mas o CO2 em 469 ppm indica sala VAZIA.
 Sessao marcada para revisao do professor.
======================================================================
```

Esta saída é de antes da última correção. Agora, quando o atrasado chega, o gateway também escreve `preenche a lacuna, nao estava perdido`.

G13 Se algo der errado

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
Servidor não mostra `[MQTT] conectado`	Porta 1883 bloqueada	`Ctrl` + `C` e rode `python servidor.py --ws`. Se ainda não conectar, 4G do celular. Confira com `Test-NetConnection` (G3).
`No such file or directory: 'requirements.txt'` ou `can't open file 'servidor.py'`	O PowerShell está em outra pasta (ex. `C:\WINDOWS\System32`)	Faça o `cd` da G3, passo 2, e repita.
`Test-NetConnection: command not found`	O terminal aberto é o Git Bash (`MINGW64`, `$`)	Abra um PowerShell: setinha ˅ ao lado do + no terminal do VS Code (G3, passo 1).
Com `--ws`, a primeira resposta demora uns segundos	O WebSocket passa por mais etapas (TLS, abertura do canal)	Normal. Depois fica fluido.
Wokwi mostra “Build Servers Busy” ou fica em “Compiling”	Fila do plano gratuito	Espere até 1–2 min. Não é erro de código.
Relógio do Wokwi parado	Janela ou aba escondida	Traga o navegador para frente. O servidor terá escrito `SILENCIO`, e está certo.
OLED mostra `SEM RESP.` e LEDs apagados	Servidor parado ou sem rede	Rode `python servidor.py`. É o comportamento seguro em ação.
Clique no botão não fez nada	Clique rápido demais para o tempo simulado	Segure 1 s.
Estado não é o esperado depois de reiniciar	Potenciômetros ficaram onde estavam; sessão antiga	Refaça a receita “dentro da sala” e digite `n` no servidor.

`ModuleNotFoundError: paho`	Bibliotecas não instaladas	`pip install -r requirements.txt`
A aula acabou no meio da demonstração	100 s de simulação passaram	Digite `n`: sessão nova.

Alterações ao vivo, se a banca pedir

PEDIDO	ONDE	O QUE ACONTECE
“Deixe a tolerância maior”	`servidor/config.exemplo.json`, `"escala": 30` → `15`; reinicie o servidor	Todos os tempos dobram (orçamento 60 s, avisos em 40 s e 52 s)
“Mude a histerese”	`sketch.ino` linha 70, `LIMIAR_ENTRADA = -70.0` → `-60.0`; ▶ de novo	O corredor (B −49) continua dentro do alcance; a posição “dentro” (−56) também
“Mostre o contrato rejeitando”	`python validar.py`	A mensagem com `bateria` é recusada
“E se a sala fosse maior?”	`config.exemplo.json`, `"poligono"`	O corredor pode passar a ser dentro

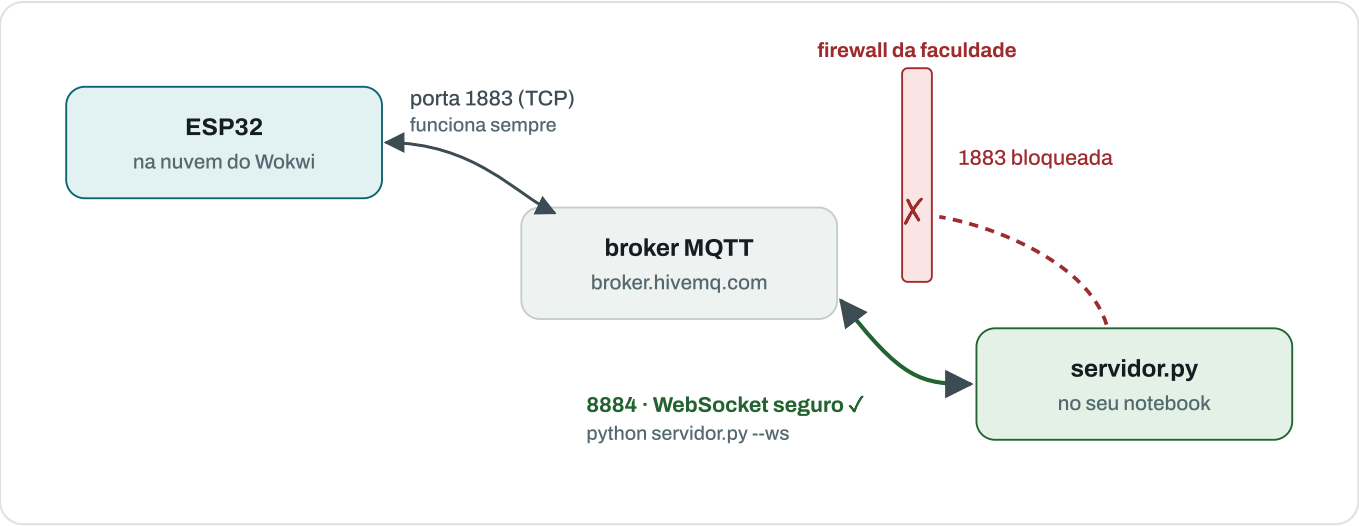
Não clique em SAVE no Wokwi depois de uma alteração ao vivo, a menos que queira que ela fique no projeto público.

G14 A rede: porta 1883, MQTT e WebSocket

Por que o servidor tem duas formas de conectar, e o que responder se perguntarem.

O caminho de uma mensagem

O ESP32 e o servidor nunca falam direto um com o outro. Os dois conversam com um **broker** MQTT, o `broker.hivemq.com`, que funciona como um correio na internet: o ESP32 deixa o heartbeat num tópico, o broker entrega a quem assinou esse tópico (o servidor), e a resposta faz o caminho inverso.



O que é uma porta

Um computador na internet tem um endereço, e cada serviço dentro dele atende numa **porta**, que é um número. Por convenção, sites usam a 443 (HTTPS) e o MQTT comum usa a **1883**. O **firewall** é o porteiro da rede: decide para quais portas os computadores de dentro podem se conectar. Redes de faculdade e de empresa costumam liberar a web e bloquear o resto, e a 1883 cai no "resto". Foi o que se viu na rede da faculdade: três brokers diferentes deram tempo esgotado na 1883, enquanto a 443 passava.

O que é WebSocket

WebSocket é um jeito de abrir uma conexão de mão dupla que fica aberta. Ela **começa como um pedido comum de página web** (HTTP) e depois vira um canal por onde os dois lados mandam mensagens quando quiserem. Foi criado para páginas web receberem dados em tempo real, como em chats e placares ao vivo. Como nasce parecendo tráfego web, os firewalls costumam deixá-lo passar.

A ideia aqui é **embrulhar**: as mesmas mensagens MQTT, com os mesmos tópicos e o mesmo JSON, viajam dentro de um WebSocket. O MQTT não muda; muda só o envelope por onde ele passa.

	MQTT COMUM	MQTT SOBRE WEBSOCKET SEGURO
Comando	`python servidor.py`	`python servidor.py --ws`
Endereço	`broker.hivemq.com:1883`	`wss://broker.hivemq.com:8884/mqtt`
Camadas	MQTT → TCP	MQTT → WebSocket → TLS → TCP
Criptografia	Não	Sim (TLS, o mesmo cadeado do HTTPS)
Passa no firewall da faculdade?	Não (testado)	Costuma passar (não testado lá)
Tópicos, JSON, contrato	Idênticos nos dois	

O **wss** do endereço quer dizer "WebSocket seguro": o WebSocket vai dentro de TLS, a criptografia do cadeado do navegador. O `/mqtt` no fim é o caminho que o broker usa para esse serviço.

E o ESP32?

Não muda nada. Ele roda nos servidores do Wokwi, que ficam na internet, fora da rede da faculdade, e continua na 1883. O broker aceita os dois transportes ao mesmo tempo e entrega as mensagens de um para o outro. Testei assim em 25/09/2026, com o servidor em WebSocket e o aparelho em TCP: os heartbeats chegaram ao servidor e as respostas voltaram ao aparelho.

Onde está no código

Em `servidor/servidor.py`, na função `criar_cliente`. Com `--ws`, ela cria o cliente MQTT com `transport="websockets"`, liga a criptografia com `tls_set()`, define o caminho com `ws_set_options(path="/mqtt")` e usa a porta `porta_ws` (8884) do `config.exemplo.json`. O gateway de CO_2 tem a mesma função. Para deixar o WebSocket como padrão, sem precisar do `--ws`, troque `"transporte": "tcp"` por `"websockets"` no config.

O que ainda não se sabe

A 8884 não foi testada *dentro* da rede da faculdade: os testes de hoje foram em casa. Por isso a ordem no dia é: `Test-NetConnection broker.hivemq.com -Port 1883` ; se der False, o mesmo com `-Port 8884` ; se der True, use `--ws` ; se der False, 4G do celular.

Se perguntarem

P. Por que não usar WebSocket sempre?

O MQTT direto é mais simples e mais leve, e é o que o ESP32 usa. O WebSocket entra para atravessar uma rede que bloqueia o MQTT, sem mexer no protocolo nem no contrato.

P. A mensagem muda?

Não. É o mesmo JSON, nos mesmos tópicos, validado pelo mesmo schema. Muda só o transporte.

P. E num sistema real?

O aplicativo do celular estaria na rede da faculdade, como o notebook. Por isso faria sentido ele já usar um transporte seguro que passe em qualquer rede (o projeto técnico prevê HTTPS). O `--ws` mostra que trocar o transporte não mexe em nada acima dele.

Parte I · fronteira principal

Aparelho do estudante ↔ servidor de permanência

O núcleo do sistema Presença rodando de verdade: um ESP32 que ouve as três âncoras e um servidor que transforma esses heartbeats em minutos de permanência, avisa antes de punir e devolve a decisão ao aparelho.

01 O que este marco entrega

A chamada tradicional mede **presença**: um estado binário, num instante arbitrário. O Presença mede **permanência**, o tempo acumulado ao longo da aula. Três minutos de sensor ruim custam três minutos, e não a aula inteira.

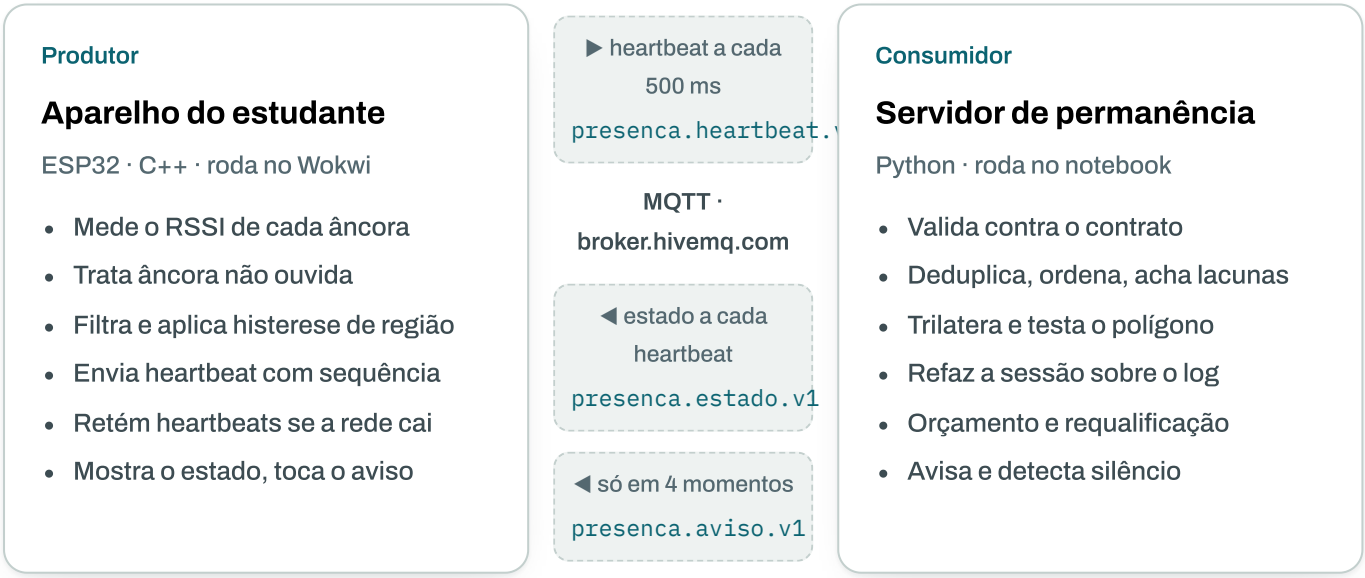
O Marco 2 pede uma fronteira real entre *dois* componentes. A escolhida é a que carrega o projeto: o **aparelho do estudante**, que mede o sinal das âncoras, falando com o **servidor**, que decide quanto tempo aquilo vale. Tudo o que o projeto técnico chama de regra inviolável passa por essa fronteira: histerese, trilateração, polígono, persistência, acumulador monotônico, orçamento por sessão, requalificação e aviso antes da punição.

7	**34**	**98,6 %**	**3**
cenários executados no Wokwi, com captura e log	testes automatizados passando	permanência da sessão gravada, calculada sobre 188 heartbeats	defeitos encontrados rodando, todos corrigidos

O ciclo completo exigido pela Atividade 03

ETAPA	ONDE ACONTECE
Fenômeno	Intensidade do sinal das âncoras A, B e C no ponto onde o estudante está
Entrada	Três potenciômetros em `GPIO34`, `GPIO35` e `GPIO32`, convertidos para −105… −35 dBm
Validação	Âncora não ouvida abaixo de −100 dBm (dado ausente, não número baixo), média móvel de 5 amostras por âncora
Evento	`presenca.heartbeat.v1` em JSON, no serial e no MQTT, a cada 500 ms
Estado e regra	No aparelho, região com histerese −70/−85 dBm. No servidor, polígono, persistência, máquina de estados, orçamento e requalificação
Decisão	PRESENTE · AUSENCIA_TOLERADA · FORA
Atuação	LEDs e OLED mostram o estado; buzzer e OLED entregam os avisos de 10 min, 13 min e limite esgotado

02 A fronteira



A âncora da sala **não** aparece nesta fronteira, e isso é deliberado: pela decisão fechada nº 3, ela só transmite e nunca escuta aparelhos. Quem mede é o aparelho. Por isso o ESP32 deste marco faz o papel do celular, e as âncoras existem apenas como o sinal que ele ouve (os potenciômetros).

Quem decide o quê

DECISÃO	ONDE	POR QUÊ
A âncora foi ouvida? Qual o sinal filtrado?	Aparelho	É medição; só o aparelho tem o rádio
Região de rádio: dentro ou fora do alcance	Aparelho	É o que o iOS faz no monitoramento de região. O sistema operacional decide entrada e saída e acorda o app
O ponto está *dentro da sala*?	Servidor	Só o servidor conhece a planta. Rádio atravessa parede, e um sinal forte na sala vizinha passa pela histerese mas não pelo polígono
Estado, acumulado, orçamento, avisos	Servidor	Precisa do log inteiro da sessão e é recalculado retroativamente
Qual horário vale	Servidor	O aparelho só informa tempo relativo ao boot; o servidor grava o próprio carimbo de recepção

Uma volta completa pela fronteira

1 Aparelho · a cada 100 ms

Lê os três ADCs, converte para dBm, descarta âncora abaixo de −100 dBm e atualiza a média móvel de cada uma.

2 Aparelho

Aplica a histerese sobre o melhor sinal: `DENTRO_ALCANCE` acima de −70 dBm, `FORA_ALCANCE` abaixo de −85 dBm. Entre os dois, mantém.

3 Aparelho · a cada 500 ms

Monta o heartbeat com `eventId = apar-01#bootId#seq` e publica. Se a rede está fora, guarda na fila.

4 Servidor

Valida contra `presenca.heartbeat.v1.schema.json`. Um campo a mais ou a menos é rejeitado.

5 Servidor

Confere a sequência: duplicata é descartada, fora de ordem é inserido no lugar certo, salto abre uma lacuna.

6 **Servidor**

Converte RSSI em distância, trilatera, testa o polígono e classifica o heartbeat como dentro, fora ou neutro.

7 **Servidor**

Refaz a sessão inteira a partir do log e obtém estado, acumulado garantido, provisório e orçamento restante.

8 **Servidor → aparelho**

Publica `presenca.estado.v1` com retain, no tópico do próprio aluno. Se um limite de aviso foi cruzado, publica também `presenca.aviso.v1`.

9 **Aparelho**

Acende o LED do estado, escreve no OLED e, se houver aviso novo, toca o buzzer. Aviso repetido é ignorado.

03 Todos os arquivos

```
marco-02/
├── README.md                    ← a entrega: a fronteira em uma página
├── GUIA.md                      ← guia da segunda fronteira (CO₂)
│
├── contrato/v1/                 ← O IDIOMA dos componentes (versão no nome)
│   ├── presenca.heartbeat.v1.schema.json   aparelho → servidor
│   ├── presenca.estado.v1.schema.json      servidor → aparelho
│   ├── presenca.aviso.v1.schema.json       servidor → aparelho
│   ├── sala.ambiente.v1.schema.json        (CO₂) âncora → gateway
│   ├── gateway.contagem.v1.schema.json     (CO₂) gateway → âncora
│   └── exemplos/                mensagens reais, todas válidas pelo schema
│
├── aparelho/                    ← O QUE RODA NO ESP32 (Wokwi)
│   ├── sketch.ino                   ~535 linhas de C++
│   ├── diagram.json                 o circuito em texto
│   └── libraries.txt                PubSubClient, Adafruit GFX, SSD1306
│
├── servidor/                    ← O QUE RODA NO NOTEBOOK
│   ├── permanencia.py               a regra: localização + máquina de estados, se
│   ├── servidor.py                  MQTT, contrato, sequência, avisos, silêncio
│   ├── config.exemplo.json          broker, tópicos, parâmetros, planta da sala
│   ├── requirements.txt             paho-mqtt, jsonschema
│   └── testes/                      34 testes (pytest)
│
├── co2-ancora/  co2-gateway/    ← a segunda fronteira (Parte II)
│
├── evidencias/presenca/         ← capturas e logs da execução no Wokwi
│   ├── 01…07-*.jpg                  uma captura por cenário
│   ├── log-serial-aparelho.txt      tudo o que o ESP32 escreveu
│   ├── log-servidor.txt             tudo o que o servidor escreveu
│   └── recebidos.jsonl              cada heartbeat com carimbo e classificação
│
└── apresentacao/                ← este documento (HTML e PDF) e suas fontes
```

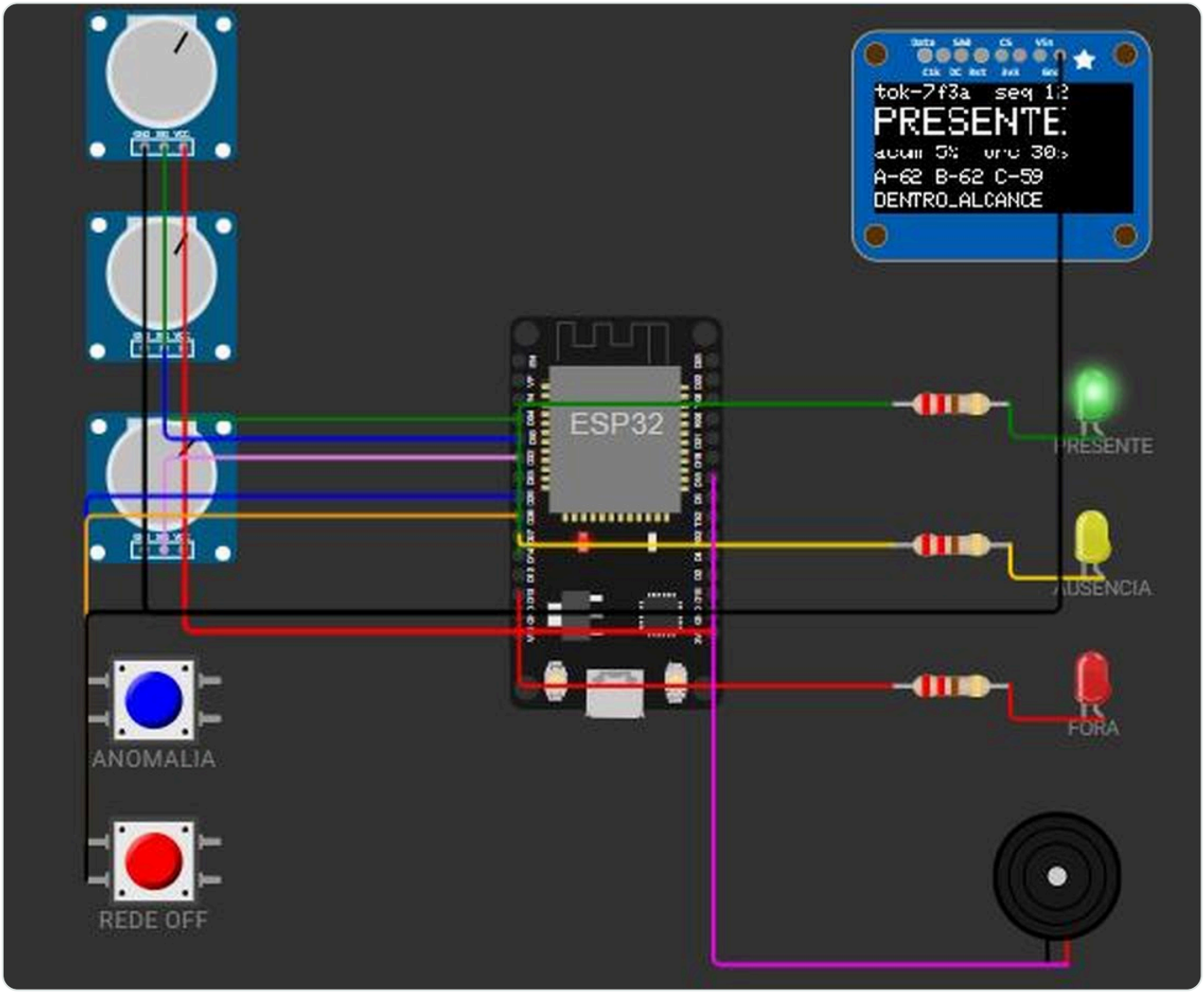
Por que a lógica mora num arquivo sem rede

`permanencia.py` não importa MQTT nem lê relógio. Recebe uma lista de heartbeats e devolve o resultado. Isso tem duas consequências. Dá para testar cada regra do projeto técnico em milissegundos, sem Wokwi nem broker. E o cálculo retroativo sai de graça: como a função só olha para o log, basta chamá-la de novo quando chega um dado atrasado.

Arquivos que nascem em execução

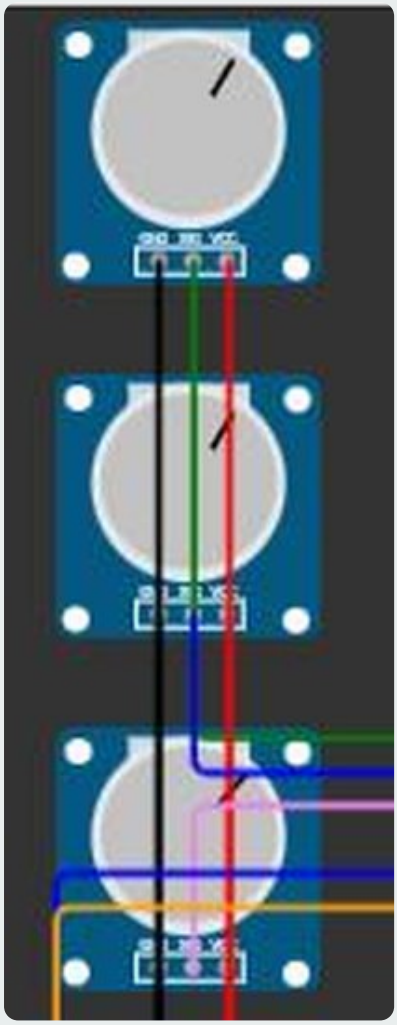
ARQUIVO	QUANDO APARECE	VERSIONADO?
`servidor/config.json`	Ao copiar o exemplo para mudar broker ou escala	Não
`servidor/recebidos.log`	Toda execução; uma linha JSON por heartbeat, com o carimbo do servidor, a classificação e o que foi derivado (t, classe, posição, estado)	Não, mas uma cópia está em `evidencias/`

04 O circuito do aparelho, peça por peça



Circuito completo em execução. O OLED mostra `PRESENTE` e o LED verde está aceso: o servidor confirmou a presença.

PEÇA	PINO DO ESP32	PAPEL
Potenciômetro A (de cima)	`GPIO34`	RSSI da âncora `anc-204-A`, parede oeste
Potenciômetro B (meio)	`GPIO35`	RSSI da âncora `anc-204-B`, parede leste
Potenciômetro C (de baixo)	`GPIO32`	RSSI da âncora `anc-204-C`, parede norte
Botão azul ANOMALIA	`GPIO25`	Arma duplicata → fora de ordem → salto
Botão vermelho REDE OFF	`GPIO26`	Liga e desliga a rede simulada
LED verde + 220 Ω	`GPIO14`	`PRESENTE`
LED amarelo + 220 Ω	`GPIO27`	`AUSENCIA_TOLERADA`; pisca durante um aviso
LED vermelho + 220 Ω	`GPIO13`	`FORA`
Buzzer	`GPIO18`	Toca 400 ms a 1760 Hz em cada aviso novo
OLED SSD1306 (I²C 0x3C)	`SDA 21` `SCL 22`	Estado, acumulado, orçamento, RSSI, região e texto do aviso



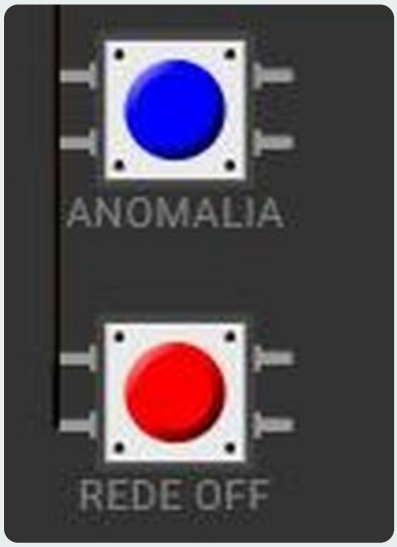
Três potenciômetros: o sinal das três âncoras

O que são. Resistores variáveis com três pinos: VCC (3,3 V), GND e o cursor (SIG), que entrega uma tensão entre 0 e 3,3 V conforme o giro. O ADC de 12 bits do ESP32 lê isso como 0 a 4095.

O que representam. Cada um é o RSSI que o celular mediria de uma âncora. O código converte linearmente: `-105 + (leitura / 4095) × 70`, ou seja, −105 dBm no mínimo e −35 dBm no máximo.

Por que três. Porque o projeto usa trilateração: com três distâncias dá para calcular um ponto, e com um ponto dá para perguntar se ele está *dentro da sala*. Um potenciômetro só daria um raio, e raio é exatamente o que o projeto rejeita.

Por que ADC1. Os pinos 32, 34 e 35 pertencem ao ADC1. O ADC2 fica indisponível enquanto o Wi-Fi está ligado, e este aparelho vive conectado.

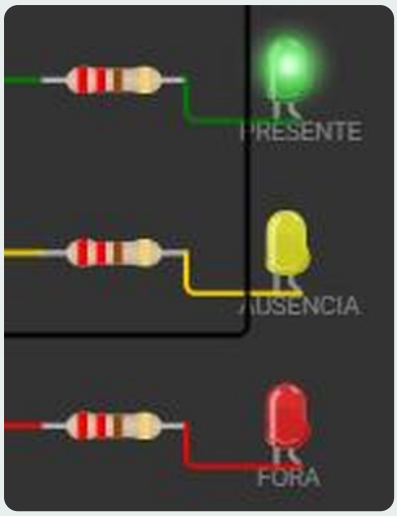


Dois botões: as falhas sob demanda

Ligados entre o pino e o GND, com o `INPUT_PULLUP` interno: solto lê `HIGH`, apertado lê `LOW`. O código age na **borda de descida** e ignora mudanças a menos de 50 ms da anterior (debounce), para que um aperto não conte como vários.

Azul, ANOMALIA. Cada aperto arma a próxima anomalia do ciclo duplicata → fora de ordem → salto, aplicada no heartbeat seguinte. É o teste adversarial da matrícula terminada em 8.

Vermelho, REDE OFF. Alterna uma queda de rede simulada. O aparelho continua medindo e guarda os heartbeats; ao voltar, entrega todos com `bufferizado: true`.



Três LEDs: a decisão do servidor, visível

Cada LED tem um resistor de 220 Ω em série: com 3,3 V no pino e cerca de 2 V de queda no LED, passam uns 6 mA, dentro do que o GPIO fornece com folga.

Comportamento seguro. Os LEDs mostram o estado que o servidor decidiu, e só enquanto essa resposta é recente (4 s). Se o servidor some ou a rede cai, **todos se apagam**: um verde velho aceso diria "você está presente" sem ninguém saber se é verdade. O OLED explica o motivo (`SEM RESP.` ou `REDE OFF`).



Buzzer: o aviso que fecha o laço

Um piezo entre o `GPIO18` e o GND. `tone(18, 1760, 400)` gera uma onda quadrada de 1760 Hz por 400 ms.

Ele representa a notificação do celular. É o ponto em que o sistema deixa de só medir e passa a **atuar**: o aviso chega enquanto o estudante ainda pode voltar, e voltar muda a grandeza que o sistema mede. Por isso ele toca só nos momentos previstos pelo projeto, e uma vez só por aviso.

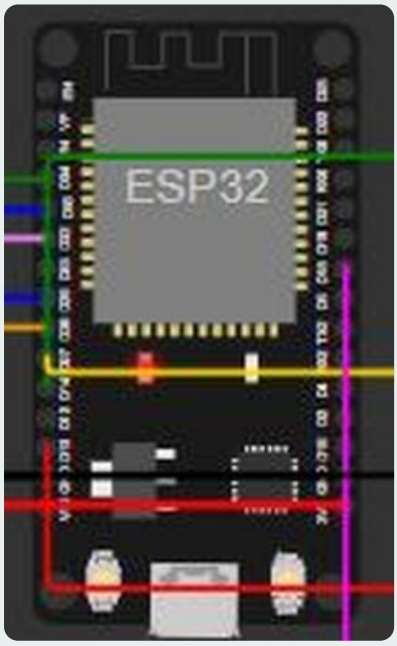


OLED: o que o estudante vê

SSD1306 de 128×64 pixels, pelo barramento I^2C (SDA no 21, SCL no 22, endereço 0x3C).

LINHA	CONTEÚDO
1	Pseudônimo do aluno e sequência atual
2 (grande)	Estado decidido pelo servidor, `REDE OFF` ou `SEM RESP.`
3	Acumulado (%) e orçamento restante (s), ou a fila retida
4	RSSI filtrado de A, B e C (`--` = não ouvida)
5	Região decidida no próprio aparelho

Durante um aviso, as linhas 3 a 5 dão lugar ao texto da mensagem por 6 s.



ESP32: o aparelho

Processador de dois núcleos com Wi-Fi. No Wokwi, conecta à rede `Wokwi-GUEST`, que tem saída real para a internet. É por isso que o MQTT funciona com um broker público de verdade.

Ele não guarda identidade: o aluno aparece só como o pseudônimo `tok-7f3a`. No sistema real, esse identificador vem da chave pública gerada no cofre de hardware no pareamento. Aqui é fixo, e não há assinatura.

05 Como o programa do aparelho funciona

Leitura e validação: âncora não ouvida é falta de dado

```
if (bruto < RSSI_NAO_OUVIDA) {         // abaixo de -100 dBm
  ouvida[i] = false;
  preenchida[i] = 0; indice[i] = 0;     // esvazia a janela desta âncora
  continue;
}
```

Uma implementação ingênua trataria −103 dBm como "muito longe" e jogaria o número na média. Mas −103 dBm, aqui, significa que o aparelho **não ouviu** a âncora. Não é uma medida baixa, é ausência de medida. O heartbeat leva `"rssi": null` e a janela é esvaziada, para que valores antigos não se misturem aos novos quando a âncora voltar. Leituras fisicamente impossíveis (acima de −30 dBm) contam em `amostrasInvalidas`.

Média móvel por âncora

Cinco amostras de 100 ms. Suaviza o ruído sem atrasar mais que meio segundo. A região só é avaliada com a janela cheia; antes disso o estado é `INDETERMINADA`, e o servidor não decide nada com base nele.

Regra local: histerese de região

```
if (nenhumaOuvida)                  nova = REG_FORA;
else if (melhor > LIMIAR_ENTRADA)   nova = REG_DENTRO;    // -70 dBm
else if (melhor < LIMIAR_SAIDA)     nova = REG_FORA;      // -85 dBm
// entre os dois: mantém
```

Dois limiares, não um. Quem senta na borda do alcance, com o sinal oscilando entre −72 e −78 dBm, não fica entrando e saindo: a faixa morta de 15 dB absorve a oscilação. Esta é a característica de estado que a Atividade 03 exige no próprio ESP32 (item 5.3).

O heartbeat e a sequência

`eventId = deviceId#bootId#sequence`. O `bootId` é sorteado com `esp_random()` a cada inicialização. Sem ele, reiniciar o aparelho zeraria a

sequência, e todo heartbeat novo teria a mesma identidade de um antigo: o servidor descartaria o boot inteiro como cópia. O problema foi visto rodando num protótipo anterior, e a correção vale para os dois.

A fila da queda de rede

Com a rede fora, o heartbeat vai para uma fila circular de 60 posições (30 s), guardando a sequência, o `eventTimeMs` original, os três RSSI e a região. Se a fila enche, o **mais antigo** é descartado e contado. O servidor verá um salto na sequência, e o intervalo aparece como sem evidência, e não como vazio. Ao voltar, a fila é drenada com `bufferizado: true`.

As três anomalias, feitas de verdade

ANOMALIA	COMO O APARELHO A PRODUZ
Duplicata	Reenvia o *último payload*, byte a byte: mesmo `eventId`, mesmo conteúdo. É o que um QoS 1 reentregando faria
Fora de ordem	Segura o heartbeat N e o publica logo depois do N+1. A mensagem é real e nova, só chega na ordem errada
Salto	Soma 5 à sequência. Esses cinco números nunca existem

Recebendo as respostas

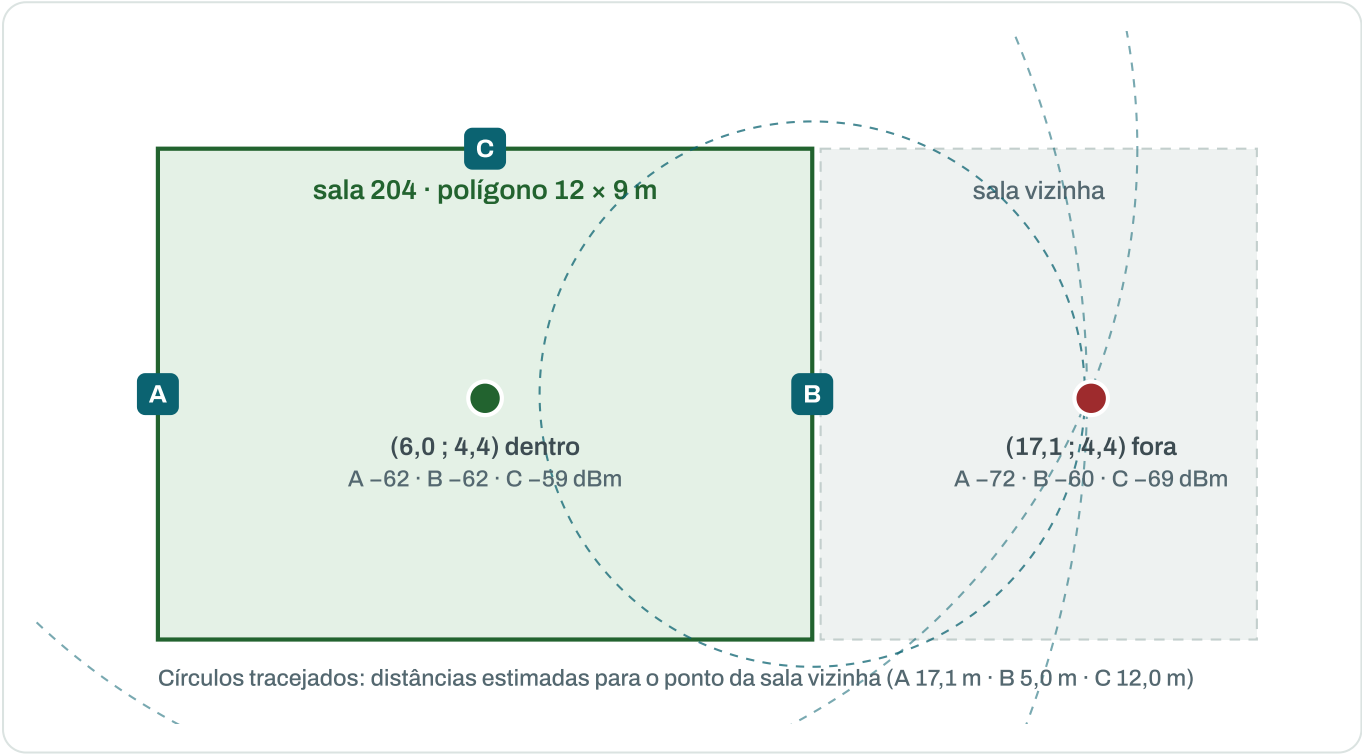
O parser procura `"campo":valor` na string recebida. É pequeno e não aloca árvore JSON, o que importa num microcontrolador. Duas proteções:

- **Estado velho não sobrescreve estado novo.** `estadoId` traz um número crescente; um número menor que o último aplicado é ignorado (a não ser que a queda seja grande, o que indica servidor reiniciado).
- **Aviso é idempotente.** Os últimos 8 `avisoId` exibidos ficam guardados; uma reentrega não toca o buzzer de novo.

06 Como o servidor decide

Onde o aparelho está: trilateração e polígono

Cada RSSI vira uma distância pelo modelo de propagação do projeto técnico (§5): `d = 10^((A − RSSI) / 10n)`, com `A = −45 dBm` a 1 m e `n = 2,2`. Três círculos, subtraídos dois a dois, viram um sistema linear em (x, y), resolvido direto. O ponto então passa por um teste de pertinência ao polígono (*ray casting*): conta quantas paredes uma semirreta a partir do ponto atravessa.

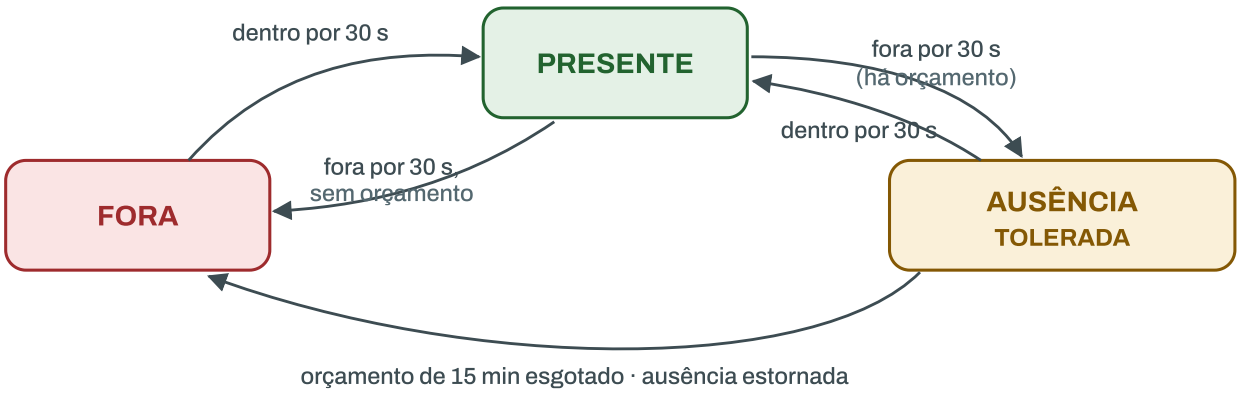


O ponto vermelho é o caso que derruba qualquer limiar em metros. O melhor sinal é de −60 dBm, forte o bastante para passar pela histerese do aparelho (`DENTRO_ALCANCE`). Um raio de alcance contaria presença. Mas os três círculos se cruzam a 5 m *do lado de fora* da parede leste, e o polígono diz não. Esse cenário foi executado no Wokwi (teste T2).

O que cada heartbeat diz

REGIÃO INFORMADA	POSIÇÃO	CLASSE
`FORA_ALCANCE`	qualquer, ou nenhuma	**fora**
`DENTRO_ALCANCE`	dentro do polígono	**dentro**
`DENTRO_ALCANCE`	fora do polígono	**fora**
`DENTRO_ALCANCE`	sem as três âncoras	**neutro**: não há base para decidir
`INDETERMINADA`	—	**neutro**

A máquina de estados



PAUSA_PROGRAMADA existe no contrato, mas depende da agenda e não está neste marco

O acumulado é refeito sobre o log, a cada heartbeat

O servidor nunca faz `acumulado += 0,5`. A cada heartbeat aceito, ele ordena o log inteiro pelo tempo do evento e percorre a sessão desde o início. Isso é o que torna possível cumprir três regras do projeto técnico ao mesmo tempo:

- **Entrada e saída valem desde o início real.** A saída só é confirmada depois de 30 s de sinal fora, mas ela começou 30 s antes. Esses 30 s deixam de ser presença e passam a ser ausência, retroativamente.
- **Evento atrasado ou fora de ordem não corrompe nada.** Ele entra na posição certa do log, e o resultado é idêntico ao da chegada em ordem. Há um teste que embaralha o log inteiro e compara.

- **O acumulado garantido nunca diminui.** O que ainda pode mudar (ausência em curso, ausência aguardando requalificação, saída ainda não confirmada) é publicado à parte, como `provisorioS`.

Orçamento, requalificação e estorno

SITUAÇÃO	O QUE ACONTECE COM O TEMPO
Sai e volta com orçamento	A ausência consome o orçamento e fica **provisória**. Só é creditada se o estudante permanecer `min(30 min, restante)` depois de voltar
Volta só por instantes e sai de novo	A ausência anterior é **estornada**, mas o orçamento continua consumido. Aparecer 20 s não reabre crédito
Orçamento acaba durante a ausência	Vai para `FORA` e a ausência **inteira** é estornada. O aviso informa quando o tempo parou de contar: no início real da ausência
Várias saídas curtas	O orçamento é da **sessão**, não do evento. Três saídas de 6 min estouram os 15
Sai no fim da aula	Soma o que assistiu. Ausência em curso no encerramento não é creditada

Os quatro avisos

Só existem os quatro momentos do §7 do projeto técnico: 10 min de ausência acumulada, 13 min, limite esgotado e fim da aula. Cada um é emitido **uma vez por sessão**, com `avisoId = sessão#tipo`. Se o estudante ignorar, o sistema não insiste.

Silêncio

Uma thread do servidor observa o relógio real. Se nenhum heartbeat chega por 6 s, o servidor escreve `SILENCIO` e **não decide nada**: silêncio pode ser rede, bateria ou aplicativo morto, e nenhuma dessas é evidência de que o estudante saiu. O que chegar depois entra no passado. Quando os heartbeats voltam, escreve `RETOMADO`.

O tempo do protótipo

O relógio do Wokwi anda entre 10 % e 80 % do tempo real, variando com a carga. Se o servidor medisse durações pelo próprio relógio, "30 s de ausência"

seria outra coisa a cada execução. Por isso a linha do tempo da máquina de estados é o `eventTimeMs` do aparelho, usado **só como diferença** dentro de um mesmo boot. Um boot novo é encaixado logo depois do último instante conhecido. O carimbo do servidor continua gravado em cada linha do log, para auditoria. Num aparelho real, com relógio sincronizado, a máquina usaria o carimbo do servidor.

07 Os contratos

Três schemas JSON (draft 2020-12) em `contrato/v1/`, todos com `additionalProperties: false`. O servidor lê o schema do disco e valida cada heartbeat: um campo inventado é rejeitado, não ignorado. Os exemplos em `contrato/v1/exemplos/` são mensagens **reais** da sessão gravada, e todos passam pela validação.

`presenca.heartbeat.v1` : aparelho → servidor

```
{
  "eventType": "presenca.heartbeat.v1",
  "eventId": "apar-01#a52e#14",           ← deviceId#bootId#sequence
  "deviceId": "apar-01",
  "entityId": "sala-204",
  "aluno": "tok-7f3a",                    ← pseudônimo, nunca MAC/IMEI/nome
  "bootId": "a52e",
  "eventTimeMs": 16371,                   ← desde o boot, não absoluto
  "sequence": 14,
  "value": -60.4,  "unit": "dBm",         ← melhor sinal
  "state": "DENTRO_ALCANCE",              ← histerese no aparelho
  "ancoras": [
    {"id": "anc-204-A", "rssi": -72.1},
    {"id": "anc-204-B", "rssi": -60.4},
    {"id": "anc-204-C", "rssi": -68.7}
  ],
  "amostrasInvalidas": 0,
  "bufferizado": false,
  "anomaliaInjetada": "nenhuma"           ← o servidor não lê este campo
}
```

Este é o heartbeat da sala vizinha: dentro do alcance e, mesmo assim, classificado como **fora**.

`presenca.estado.v1` : servidor → aparelho, com retain

```
{"eventType":"presenca.estado.v1","estadoId":"srv-204#62","aluno":"tok-7f3a",
 "entityId":"sala-204","estado":"AUSENCIA_TOLERADA","acumuladoS":7.4,"provisorioS"
 "percentual":7.4,"orcamentoRestanteS":5.5,"tempoAulaS":33.3,"classe":"fora",
 "posicao":[17.1,4.4],"dentroPoligono":false,"encerrada":false,"emitidoEmMs":17903
```

Publicado no tópico `…/estado/tok-7f3a` : cada aparelho só assina o próprio. Com `retain` , um aparelho que acabou de ligar recebe o último estado de imediato. O JSON sai compacto, sem espaços, porque o parser do ESP32 procura `"campo":valor` .

`presenca.aviso.v1` : servidor → aparelho, QoS 1

```
{"eventType":"presenca.aviso.v1","avisoId":"s31a2#AVISO_10_MIN","aluno":"tok-7f3a"
 "tipo":"AVISO_10_MIN","texto":"Fora da sala ha 20 s. Seu limite e de 30 s nesta a
 "percentual":7.4,"emitidoEmMs":1790310788000}
```

Por que a versão está no nome

Uma mudança incompatível cria `presenca.heartbeat.v2` , e a v1 continua válida enquanto os aparelhos antigos migram. Adicionar `bateria` ao heartbeat hoje faz o servidor responder `INVALIDO … Additional properties are not allowed` : há um teste só para isso. É a alteração ao vivo mais convincente para mostrar na banca.

08 Testes executados no Wokwi

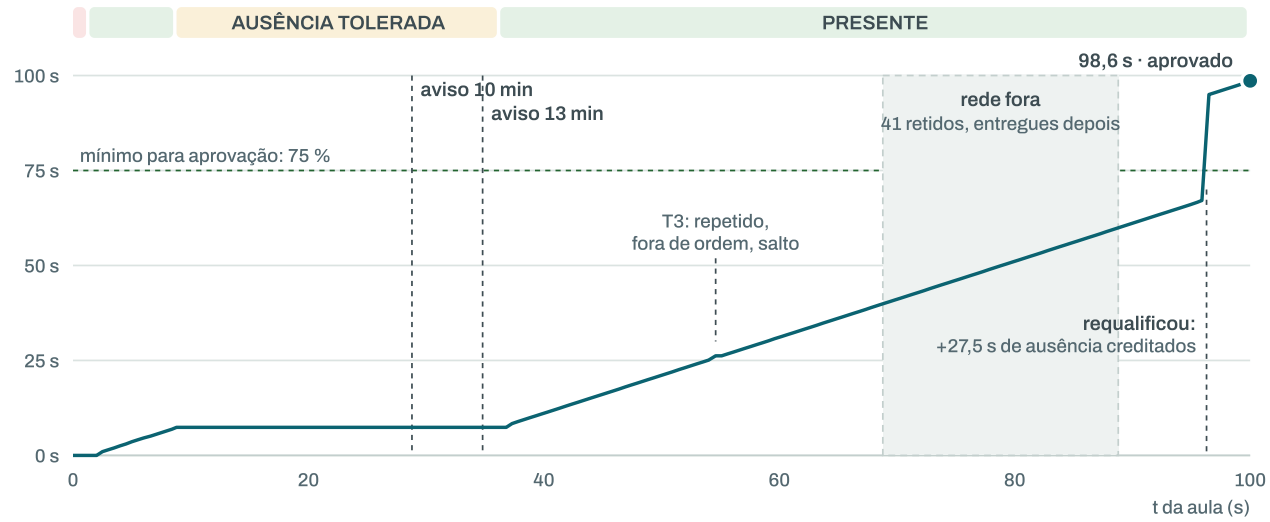
Tudo abaixo foi executado em 25/09/2026, com o aparelho no Wokwi e o servidor no notebook falando pelo `broker.hivemq.com:1883` . Não há simulação de resultado: as capturas são da tela do Wokwi, e os trechos de terminal foram copiados dos logs em `evidencias/presenca/` . Os potenciômetros e botões foram acionados pelo próprio circuito do Wokwi.

Parâmetros de todos os testes

Escala 1:30. Persistência 1 s · orçamento 30 s · avisos em 20 s e 26 s · requalificação 60 s · aula de 100 s · mínimo de 75 %. Heartbeat a cada 500 ms. Tempos em segundos da linha do tempo da sessão (`t`).

A sessão gravada, do início ao fim

Os testes T1 a T4 e T6 aconteceram numa mesma aula de 100 s, a sessão `s31a2`. O gráfico mostra o acumulado garantido que o servidor publicou para cada heartbeat, na ordem do tempo do evento.

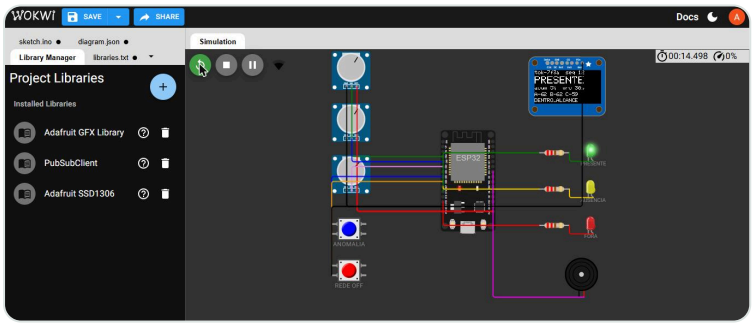


Três coisas para notar. Durante a ausência (faixa âmbar) o acumulado **para**, mas não cai. Na queda de rede, o servidor ficou em silêncio e depois recebeu 41 heartbeats atrasados que preencheram o trecho sem buraco. Em t = 96,3 s a requalificação se cumpre e os 27,5 s de ausência tolerada entram de uma vez no acumulado.

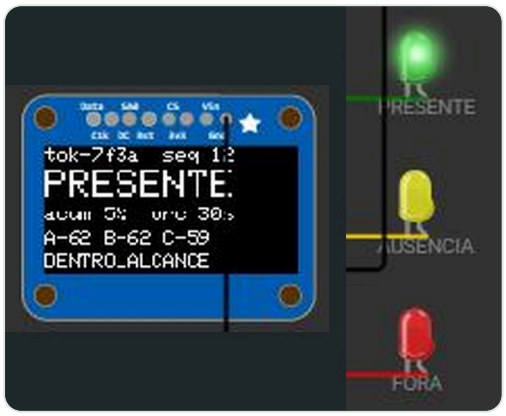
T1 Estado normal: dentro da sala

passou

Estado inicial	Simulação reiniciada, sessão nova `s31a2` aberta no servidor. Estado FORA, acumulado 0.
Entradas	Potenciômetros em A = 628, B = 628, C = 667 (de 1023), o centro da sala.
Valores e tempos	RSSI filtrado A −62,0, B −62,0, C −59,4 dBm. Posição trilaterada (6,0; 4,4). Região `DENTRO_ALCANCE`.
Esperado	Depois de 1 s de sinal dentro, `PRESENTE` desde o início real da condição; LED verde; acumulado crescendo; nenhum aviso.
Observado	`FORA → PRESENTE em t=1,4 s`, confirmado no heartbeat seguinte e datado no início da condição. LED verde aceso, OLED com `PRESENTE`, orçamento intacto em 30 s.
Por quê	Três heartbeats seguidos *dentro* cobrem a persistência. A entrada é datada no primeiro deles, e não no que a confirmou: é o cálculo retroativo.



Captura do Wokwi durante o T1.



OLED e LEDs ampliados.

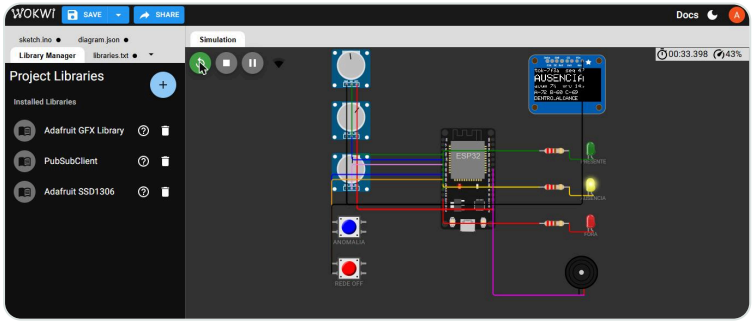
servidor · log-servidor.txt

```
01:30:53 t=  2.0s seq=3   A -62 B -62 C -59 DENTRO_ALCANCE ( 6.0, 4.4) dentro -> FORA
01:30:55 t=  2.5s seq=4   A -62 B -62 C -59 DENTRO_ALCANCE ( 6.0, 4.4) dentro -> PRESENTE  acum  1.
   >>> FORA -> PRESENTE em t=1.4 s (sinal na sala sustentado)
```

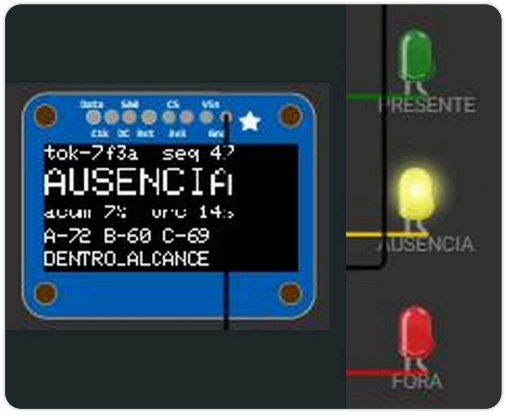
T2 Decisão e atuação: sala vizinha, avisos e retorno

passou

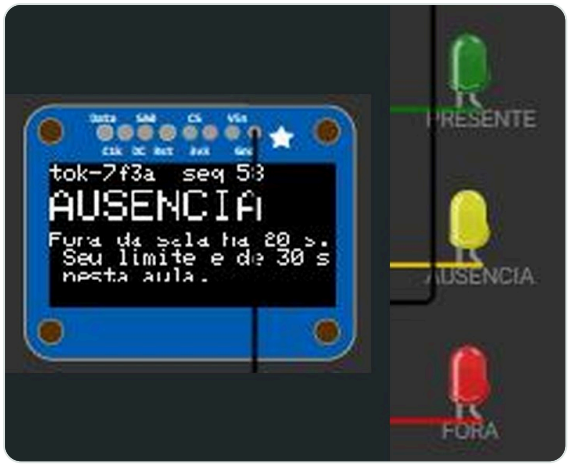
Estado inicial	`PRESENTE`, acumulado 7,4 s, orçamento 30 s.
Entradas	Potenciômetros para A = 481, B = 652, C = 531 (sala vizinha, colado à parede leste). Depois do segundo aviso, de volta a 628 / 628 / 667.
Valores e tempos	RSSI A −72, B −60, C −69 dBm: a região continua `DENTRO_ALCANCE`. Posição (17,1; 4,4), fora do polígono. Saída a partir de t = 8,8 s; volta em t = 36,3 s.
Esperado	`AUSENCIA_TOLERADA` com LED amarelo; aviso de 10 min em 20 s de ausência (t = 28,8); aviso de 13 min em 26 s (t = 34,8); retorno antes dos 30 s volta a `PRESENTE` sem punição; ausência provisória até a requalificação.
Observado	Exatamente isso. Buzzer e OLED com "Fora da sala ha 20 s. Seu limite e de 30 s nesta aula." e depois "Faltam 4 s do seu limite...". Retorno confirmado em t = 36,3 s com 2,5 s de orçamento sobrando. A ausência de 27,5 s foi creditada em t = 96,3 s, quando a requalificação se cumpriu.
Por quê	A histerese sozinha diria "presente": o sinal é forte. É o polígono que decide. Os avisos chegam enquanto ainda cabe reação, e a reação (voltar) é que impede a punição. É o laço ciber-físico fechado.



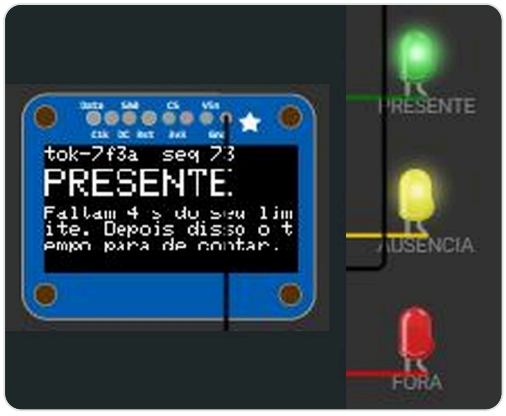
Sinal forte, mas na sala vizinha: **AUSENCIA**, com a região ainda `DENTRO_ALCANCE`.



OLED e LEDs ampliados.



Primeiro aviso (10 min reais, 20 s na escala).



Segundo aviso, já de volta: o LED verde acendeu.

servidor · log-servidor.txt (trechos)

```
01:32:36 t= 17.0s seq=30  A -72 B -60 C -69 DENTRO_ALCANCE ( 17.1,  4.4) fora  -> AUSENCIA_TOLERAD
 AVISO AO ESTUDANTE [AVISO_10_MIN] t=28.8 s
 "Fora da sala ha 20 s. Seu limite e de 30 s nesta aula."
 AVISO AO ESTUDANTE [AVISO_13_MIN] t=34.8 s
 "Faltam 4 s do seu limite. Depois disso o tempo para de contar."
01:33:26 t= 36.3s seq=68  A -62 B -62 C -59 DENTRO_ALCANCE (  6.0,  4.4) dentro -> AUSENCIA_TOLERA
01:33:27 t= 36.8s seq=69  ...                                dentro -> AUSENCIA_TOLERADA acum  7.4s orc 2.0
01:33:28 t= 37.3s seq=70  ...                                dentro -> PRESENTE       acum  8.4s orc 2.5s
   >>> AUSENCIA_TOLERADA -> PRESENTE em t=36.3 s (retornou dentro do orcamento)
01:36:37 t= 95.9s seq=182 ...                                 dentro -> PRESENTE       acum 67.1s
01:36:38 t= 96.5s seq=183 ...                                 dentro -> PRESENTE       acum 95.0s  ← requalific
```

Repare no orçamento em t = 36,8 s: 2,0 s. No heartbeat seguinte, quando o retorno se confirma, ele **volta a 2,5 s**. O servidor refez a sessão e descobriu que a ausência terminou em 36,3 s, e não em 36,8 s.

T3 Adversarial (matrícula final 8): repetido, fora de ordem, salto

passou

Estado inicial	`PRESENTE`, t ≈ 49 s, acumulado 20,5 s, sequência 91.
Entradas	Três apertos do botão azul, cada um segurado por 2 s, com cerca de 6 s entre eles.
Valores	Duplicata do seq 92 · seq 100 retido e entregue depois do 101 · salto de 106 para 112.
Esperado	Duplicata descartada sem recálculo; fora de ordem aceito na posição original, sem alterar o resultado; salto contado com 5 perdidos. Nenhuma mudança de estado.
Observado	`DUPLICADO` · `FORA DE ORDEM … inserido na posicao original` · `SALTO … 5 heartbeat(s) perdido(s)`. Estado `PRESENTE` o tempo todo. O fora de ordem chegou com o mesmo acumulado do heartbeat anterior (26,2 s), já que o resultado não depende da ordem de chegada.
Por que o ingênuo erra	Um contador que soma 0,5 s por heartbeat recebido contaria a duplicata duas vezes. Um que usa "o último que chegou" como estado atual retrocederia no tempo com o fora de ordem. Um que não olha a sequência não saberia que 5 heartbeats sumiram, e trataria o intervalo como observado.

aparelho · log-serial-aparelho.txt

```
### anomalia armada: duplicata (aplica no proximo heartbeat)
>>> ADVERSARIAL: reenviando o ultimo heartbeat (mesmo eventId)
### anomalia armada: fora-de-ordem (aplica no proximo heartbeat)
>>> ADVERSARIAL: segurando seq=100 para entregar depois de 101
>>> ADVERSARIAL: entregando agora o heartbeat retido (fora de ordem)
### anomalia armada: salto (aplica no proximo heartbeat)
>>> ADVERSARIAL: salto de 106 para 112 (5 numeros nunca enviados)
```

servidor · log-servidor.txt

```
01:35:06 t= 49.9s  seq=92   ... dentro -> PRESENTE  acum  21.0s
01:35:07 DUPLICADO     apar-01#a52e#92 ja processado - descartado, nada recalculado
01:35:17 SALTO         esperado 100, recebido 101 - 1 heartbeat(s) perdido(s); o intervalo fica sem evidencia
01:35:17 t= 55.1s  seq=101  ... dentro -> PRESENTE  acum  26.2s
01:35:17 FORA DE ORDEM  seq=100 chegou depois de 101 - inserido na posicao original e sessao recalcula
01:35:17 t= 54.6s  seq=100  ... dentro -> PRESENTE  acum  26.2s
01:35:24 SALTO         esperado 107, recebido 112 - 5 heartbeat(s) perdido(s); o intervalo fica sem evidencia
01:35:24 t= 58.1s  seq=112  ... dentro -> PRESENTE  acum  29.2s
```

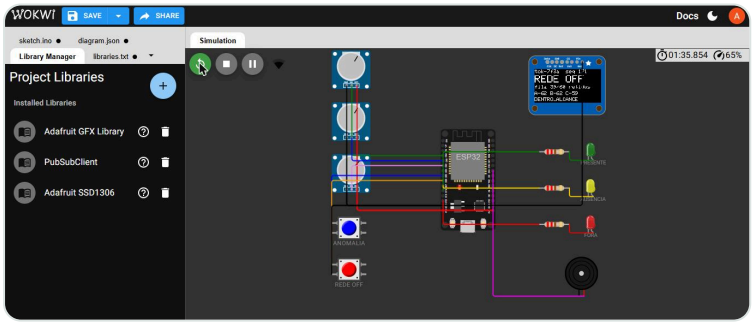
Defeito encontrado neste teste

Quando o 101 chega antes do 100, o servidor vê uma lacuna e conta "1 perdido". O 100 chega em seguida, mas a contagem não era desfeita: o relatório final dizia `6 perdidos`, quando só 5 se perderam. Corrigido depois da execução. O servidor agora guarda as lacunas e, quando um atrasado preenche uma, desconta e escreve `preenche a lacuna: nao estava perdido, so atrasado`. Há um teste para isso. O log acima é o da execução original, com o defeito.

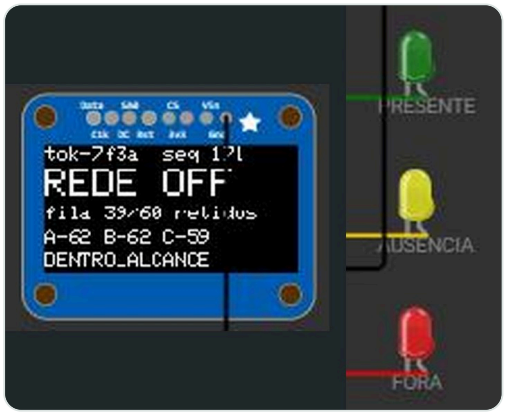
T4 Queda de rede: silêncio, retenção e reentrega

passou

Estado inicial	`PRESENTE`, t ≈ 68 s, acumulado 39,4 s.
Entradas	Botão vermelho uma vez; cerca de 25 s reais depois, de novo.
Valores e tempos	41 heartbeats retidos (seq 133 a 173, t de 68,8 a 88,8 s); a fila chegou a 39/60 na captura. Nenhum descarte por fila cheia.
Esperado	O aparelho continua medindo e mostra `REDE OFF` com a fila; LEDs de estado apagados. O servidor acusa silêncio e não decide nada. Na volta, recebe os retidos como atrasados, no tempo original, e o acumulado segue sem buraco.
Observado	`SILENCIO` 6 s depois do último heartbeat; `RETOMADO` e 41 linhas `ATRASADO … entra no passado e a sessao e recalculada`. O acumulado seguiu de 39,9 s a 59,9 s através do trecho retido. O estado continuou `PRESENTE`.
Por quê	Silêncio não é ausência. Um servidor ingênuo, vendo o aparelho sumir, poderia abrir uma ausência e cobrar orçamento. Este espera o dado: quando ele chega, mesmo atrasado, a sessão é refeita como se tivesse chegado a tempo.



Rede fora: o aparelho segue medindo e guardando. **Todos os LEDs apagados**, porque uma decisão velha não é exibida como atual.



OLED e LEDs ampliados.

aparelho · log-serial-aparelho.txt

```
### rede simulada: INDISPONIVEL
[REDE] indisponivel: heartbeat seq=133 retido (fila 1/60)
[REDE] indisponivel: heartbeat seq=134 retido (fila 2/60)
...
[REDE] indisponivel: heartbeat seq=173 retido (fila 41/60)
### rede simulada: RESTABELECIDA
[RETRY] 41 heartbeat(s) retidos entregues com atraso
```

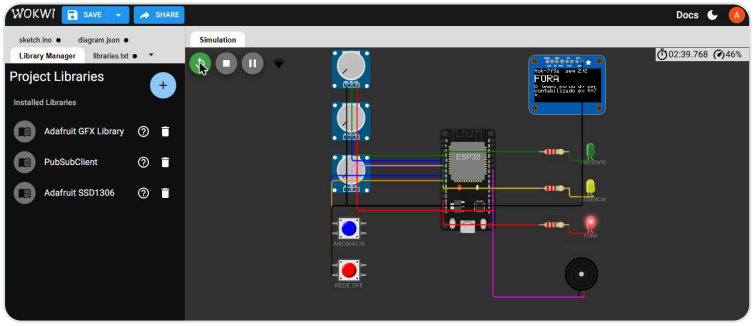
servidor · log-servidor.txt

```
01:35:47 t=  68.3s  seq=132  ... dentro -> PRESENTE  acum  39.4s
01:35:53 SILENCIO      nenhum heartbeat ha 6 s - rede, bateria ou app. Nada e decidido sem dado; o que c
01:36:25 RETOMADO      heartbeats voltaram a chegar
01:36:25 ATRASADO      seq=133 retido no aparelho, medido em 76335 ms - entra no passado e a sessao e r
01:36:25 t=  68.8s  seq=133  ... dentro -> PRESENTE  acum  39.9s
...  (40 linhas ATRASADO)
01:36:28 t=  88.8s  seq=173  ... dentro -> PRESENTE  acum  59.9s
```

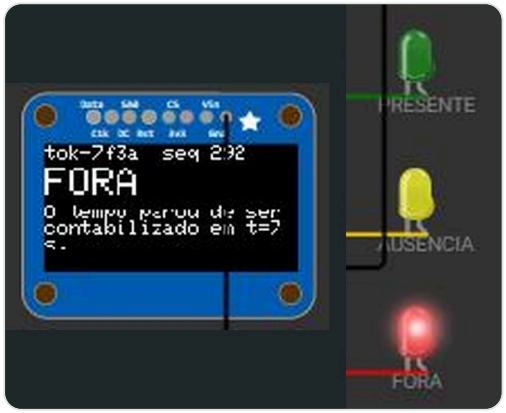
T5 Orçamento esgotado: aviso e estorno

passou

Estado inicial	Sessão nova `sb6a8`. PRESENTE desde t = 0, acumulado 6,6 s.
Entradas	Os três potenciômetros em 0: nenhuma âncora ouvida, fora do prédio. Mantidos assim por mais de 30 s.
Valores e tempos	RSSI `--` nas três; região `FORA_ALCANCE`. Ausência a partir de t = 6,6 s; avisos em 26,6 e 32,6 s; orçamento esgotado em t = 36,6 s.
Esperado	Os dois avisos e, ao fim do orçamento, `FORA` com LED vermelho, a mensagem de limite e a ausência inteira estornada: o acumulado volta a contar só a presença real.
Observado	`LIMITE_ESGOTADO` em t = 36,6 s: "O tempo parou de ser contabilizado em t=7 s." Estado `FORA`, LED vermelho, orçamento 0. Acumulado 6,6 s: nenhum segundo da ausência foi creditado.
Por quê	§6 do projeto técnico: sem retorno dentro do orçamento, o tempo desde o início real da ausência é estornado. A mensagem aponta esse início (t ≈ 6,6 s, arredondado no texto), e não o momento em que o limite estourou. É o horário que importa para o estudante contestar.



Limite esgotado: **FORA** e LED vermelho.



OLED e LEDs ampliados.

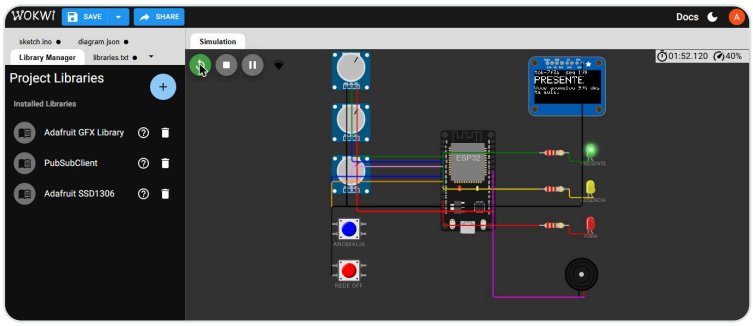
servidor · log-servidor.txt

```
01:37:31 t=  7.7s  seq=230  A -- B -- C -- FORA_ALCANCE  sem posicao fora  -> AUSENCIA_TOLERADA ac
   >>> PRESENTE -> AUSENCIA_TOLERADA em t=6.6 s (sinal fora da sala sustentado)
 AVISO AO ESTUDANTE  [LIMITE_ESGOTADO]  t=36.6 s
 "O tempo parou de ser contabilizado em t=7 s."
01:38:39 t=  37.2s  seq=288  A -- B -- C -- FORA_ALCANCE  sem posicao fora  -> FORA          acum  6.6s c
```

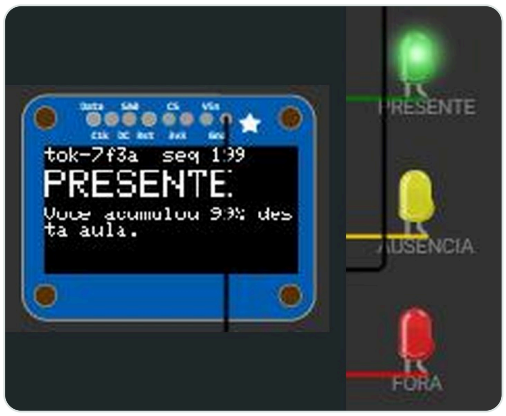
T6 Fim da aula: o resultado da sessão

passou

Estado inicial	Sessão `s31a2`, depois de T1 a T4.
Entradas	Nenhuma. A aula termina em t = 100 s.
Esperado	Aviso `FIM_DA_AULA` com o percentual; relatório final com o detalhamento; aprovação se ≥ 75 %.
Observado	"Voce acumulou 99% desta aula." Relatório: 71,1 s de presença + 27,5 s de ausência creditada = 98,6 s (98,6 %), **aprovado**. Orçamento usado 27,5 de 30 s.
Por quê	A presença perdida está toda no começo: a aula começa em t = 0 e a presença é confirmada em t = 1,4 s. Os outros 98,6 s estão cobertos por presença ou por ausência tolerada que se requalificou.



O aviso de fim de aula no aparelho.



OLED ampliado.

servidor · log-servidor.txt

```
------------------------------------------------------------------------
 RELATORIO FINAL DA SESSAO s31a2
 estado: PRESENTE   t=100.0 s de 100 s
 acumulado garantido: 98.6 s (98.6%)   provisorio: 0.0 s   -> APROVADO na aula (minimo 75%)
 presenca confirmada 71.1 s + ausencia creditada 27.5 s   orcamento usado 27.5 de 30 s
 contador ingenuo (ao vivo, sem dedup): 67.0 s
   t=   1.4s  FORA -> PRESENTE  (sinal na sala sustentado)
   t=   8.8s  PRESENTE -> AUSENCIA_TOLERADA  (sinal fora da sala sustentado)
   t=  36.3s  AUSENCIA_TOLERADA -> PRESENTE  (retornou dentro do orcamento)
 aceitos=188 duplicados=1 fora de ordem=1 saltos=2 (6 perdidos) atrasados=41 invalidos=0
------------------------------------------------------------------------
```

O "contador ingênuo" (0,5 s por heartbeat *dentro*, somado na chegada) daria 67,0 s, reprovando um estudante que só saiu com tolerância. Ele não enxerga a

ausência tolerada nem os intervalos irregulares entre heartbeats. O "6 perdidos" é o defeito do T3, já corrigido.

T7 Reinício do aparelho: silêncio e boot novo

passou

O que aconteceu	Não foi planejado: a simulação foi reiniciada logo depois de abrir a sessão `s31a2`, e a janela do Wokwi ficou oculta por alguns instantes (o navegador congela a simulação).
Observado	Dois heartbeats do boot anterior (seq 74, 75), `SILENCIO`, `RETOMADO` e o boot novo `a52e` começando em seq 1, **sem nenhuma duplicata**, encaixado em t = 1,0 s logo depois do último instante conhecido. O primeiro heartbeat do boot novo veio `INDETERMINADA` (janela da média ainda enchendo) e foi tratado como neutro.
Por quê	O `bootId` na identidade do evento e o encaixe da linha do tempo por boot.

servidor · log-servidor.txt

```
01:30:32 t=  0.5s  seq=75   ... DENTRO_ALCANCE (  6.0,  4.4) dentro -> FORA
01:30:38 SILENCIO      nenhum heartbeat ha 6 s - rede, bateria ou app. Nada e decidido sem dado; ...
01:30:51 RETOMADO      heartbeats voltaram a chegar
01:30:51 t=  1.0s  seq=1    ... INDETERMINADA  (  6.0,  4.4) neutro -> FORA
```

09 Testes automatizados

`python -m pytest servidor/testes -q` → **34 passed.** Não precisam de Wokwi nem de broker: o servidor é exercitado com um cliente MQTT falso, e cada resposta publicada é validada contra o próprio contrato.

GRUPO	O QUE GARANTE
Parâmetros	A escala comprime todos os tempos juntos (30 s → 1 s, 15 min → 30 s…)
Localização	A trilateração recupera o ponto em 4 posições; sinal forte na sala vizinha é *fora*; âncora não ouvida deixa neutro
Máquina de estados	Persistência na entrada; oscilação curta não derruba; neutro interrompe; entrada datada no início real
Orçamento	Ausência curta creditada só depois de requalificar; voltar por instantes não reabre crédito; avisos em 20 s e 26 s, uma vez só; esgotamento estorna tudo; orçamento por sessão; retorno a tempo não é punido pela demora da confirmação
Retroativo	Embaralhar o log não muda o resultado; lote atrasado corrige o passado; o acumulado nunca diminui (verificado a cada heartbeat de uma sessão inteira)
Fim da aula	Quem sai no fim soma o que assistiu
Servidor	Respostas válidas pelos schemas; estado retido e por aluno; campo desconhecido rejeitado; duplicata fora do log; fora de ordem na posição original; salto conta perdidos; atrasado que preenche lacuna não fica perdido; reboot não vira duplicata; lote atrasado recalcula; contador ingênuo infla com duplicatas

Um desses testes falhou na primeira execução e mostrou um problema real. Durante o segundo de confirmação de uma saída, o acumulado garantido subia meio segundo e depois voltava. A cauda ainda não confirmada passou a ser contada como provisória.

10 O que só apareceu rodando

ACHADO	CONSEQUÊNCIA	TRATAMENTO
Fora de ordem inflava os perdidos	Relatório com 6 perdidos quando eram 5	Lacunas por boot; o atrasado que preenche uma é descontado (T3)
O "fora de ordem" do protótipo de CO_2 nunca foi fora de ordem	Ele reenviava uma sequência *já processada*, e o gateway a classificava, corretamente, como duplicata. O teste adversarial daquela fronteira não exercitava o que dizia	A âncora de CO_2 agora segura o evento N e o entrega depois do N+1, como o aparelho. Compilado no Wokwi
Acumulado garantido não era monotônico por meio segundo	Subia e voltava durante a confirmação da saída	Cauda não confirmada vai para o provisório
Janela oculta congela a simulação	Relógio parado em 14,5 s; apertos de botão feitos nesse intervalo duram 0 ms simulados e não são vistos	Na apresentação, a janela do Wokwi fica em primeiro plano. O servidor mostrou `SILENCIO` nas quatro vezes em que isso aconteceu, sem decidir nada

11 O que foi validado e o que não foi

VALIDADO NO SIMULADOR	EXIGIRIA HARDWARE E AMBIENTE REAIS
Contrato, sequência, deduplicação, fora de ordem, salto, silêncio	Propagação de rádio real: multipercurso, corpo humano, parede, orientação do aparelho
Histerese de região, trilateração, polígono, persistência	Calibração dos limiares em dBm por levantamento na sala (cantos, porta, corredor, vizinha)
Orçamento, requalificação, estorno, avisos no momento certo	Cadência real: heartbeat de 60 a 90 s no Android e eventos de região no iOS
Retenção local e reentrega com recálculo retroativo	Fila sobrevivendo a reinício do aparelho (aqui é volátil)

Declaração obrigatória sobre a entrada

Os potenciômetros **não são rádio**. Não houve aquisição nem validação de RSSI real: o Wokwi não simula propagação, não há BLE, distância nem parede. São fontes de valor controláveis entre −105 e −35 dBm.

Fora do escopo deste marco: token rotativo da âncora, assinatura do heartbeat com a chave do aparelho, pareamento por QR, variância de IMU, motivo da ausência (`bateria` , `app_morto`) e `PAUSA_PROGRAMADA` , que depende da agenda.

12 Roteiro para a apresentação ao vivo

O roteiro completo, com comandos, posições dos potenciômetros pelo teclado, o que aparece e o que falar em cada teste, está no Guia do dia, G9. Os valores usados na execução registrada acima (628/628/667 e 481/652/531) foram aplicados por automação; para operar à mão, use as receitas de teclado do G6.

13 Perguntas prováveis

P. Por que o ESP32 é o celular, e não a âncora?

Porque a âncora nunca escuta aparelhos (decisão fechada nº 3). Quem mede o sinal é o celular. A fronteira que carrega a permanência é, portanto, celular → servidor. A âncora aparece como o sinal que o celular ouve.

P. Se o servidor trilatera, para que a histerese no aparelho?

Para não mandar ao servidor a oscilação de quem está na borda do alcance, e porque é assim que o iOS funciona: o sistema decide a entrada e a saída da região e acorda o app. O polígono fica no servidor porque o aparelho não conhece a planta.

P. O que acontece se chegar um heartbeat de 20 s atrás?

Ele entra na posição dele no log e a sessão é refeita. Se ele mostra que o estudante estava na sala, o que parecia ausência vira presença. O acumulado garantido nunca cai; o que muda é o provisório.

P. Por que não somar 0,5 s a cada heartbeat?

Duplicatas contariam duas vezes, atrasados cairiam fora do lugar, e a regra de que a ausência começa no início real, e não na confirmação, seria impossível. Na sessão gravada, esse contador daria 67 s; o correto é 98,6 s.

P. E se o celular morrer?

O servidor vê silêncio e não decide nada sem dado. Distinguir bateria de app morto (o atributo `motivo`) está fora deste marco.

P. O RSSI de um potenciômetro vale alguma coisa?

Como grandeza física, não. Ele exercita a lógica com valores controláveis e repetíveis. Os limiares reais sairiam de um levantamento na sala.

P. Por que o `eventTimeMs` é a linha do tempo, se o relógio do aparelho não é confiável?

Porque aqui o problema é o oposto: o relógio do Wokwi anda a 10 %–80 % do real, e medir 30 s pelo relógio do servidor daria outra coisa a cada execução. O aparelho só contribui com diferenças dentro do mesmo boot, e o carimbo do servidor fica gravado em cada linha do log.

Parte II · segunda fronteira

Âncora de CO_2 ↔ gateway: a defesa contra o conluio em massa

Construída antes da fronteira principal e mantida como complemento. A turma deixa os celulares na sala e vai embora: todas as defesas de rádio confirmam a presença. O CO_2 da sala não. Pastas `co2-ancora/` e `co2-gateway/`.

C1 O problema que este protótipo resolve

O sistema **Presença** mede a permanência de estudantes em sala por rádio: três âncoras transmitem, o celular ouve, e a posição sai por trilateração. Contra quase toda fraude isso funciona — token rotativo impede forjar sinal, assinatura impede falsificar identidade.

Existe uma fraude, porém, que **nenhuma defesa de rádio alcança**: a turma inteira deixa os celulares na sala e vai embora. Os aparelhos estão genuinamente ali. A trilateração confirma, o token confere, a assinatura bate. O rádio não tem como saber que não há ninguém.

> **A ideia**
>
> Pessoas respiram e produzem CO_2. Se o rádio declara 30 estudantes e o CO_2 diz que a sala está vazia, alguém está mentindo — e **o ar não mente**.

É isso que esta âncora faz: cruza duas fontes independentes de evidência e acusa a contradição. Nenhuma das duas decide sozinha.

O ciclo completo exigido pela atividade

O protótipo implementa o caminho inteiro, e vale saber apontar cada etapa no código:

ETAPA	ONDE ACONTECE
Fenômeno	Concentração de CO_2 na sala (representada pelo potenciômetro)
Sensor / entrada	`analogRead(34)` — ADC de 12 bits
Validação	Faixa física + rejeição de salto implausível + média móvel
Evento estruturado	`sala.ambiente.v1` em JSON, no serial e no MQTT
Estado e regra	Histerese, persistência, validade e verificação cruzada
Decisão	● COERENTE · ● ATENÇÃO · ● INCOERENTE
Atuação	LEDs e display na âncora; alerta no gateway

C2 A fronteira entre os dois componentes

O Marco 2 pede uma integração real entre *dois* componentes — não o sistema inteiro. A fronteira escolhida é a que existe de verdade no projeto: a âncora da sala falando com o gateway do prédio.



Repare que a seta **vai e volta.** O gateway não só recebe: ele devolve a contagem de estudantes que o subsistema de rádio declara. Sem esse dado a âncora não teria como fazer a verificação cruzada, porque ela não tem acesso ao rádio.

Quem decide o quê, e por quê

DECISÃO	ONDE	MOTIVO
Estado da sala (coerente ou não)	Âncora	Depende de histórico de segundos que não vale a pena trafegar pela rede
O que fazer com esse estado	Gateway	Depende de contexto que a âncora não tem: turma, horário, histórico do semestre
Qual horário vale para o registro	Gateway	O `eventTimeMs` do ESP32 é tempo desde o boot, e relógio de dispositivo em campo não é confiável

C3 Todos os arquivos, e por que cada um existe

```
marco-02/
├── GUIA.md                ← este guia em Markdown
│
├── contrato/              ← O IDIOMA que os dois componentes falam
│   └── v1/                    "v1" é a versão — e ela faz parte do nome
│       ├── sala.ambiente.v1.schema.json
│       ├── gateway.contagem.v1.schema.json
│       └── exemplos/          quatro instâncias válidas
│
├── co2-ancora/              ← O QUE RODA NO ESP32
│   ├── sketch.ino             o programa, ~530 linhas de C++
│   ├── diagram.json           o circuito descrito em texto
│   └── libraries.txt          as três bibliotecas externas
│
├── co2-gateway/           ← O QUE RODA NO NOTEBOOK
│   ├── consumidor.py          o gateway em Python
│   ├── requirements.txt       paho-mqtt e jsonschema
│   └── config.exemplo.json    modelo de configuração
│
└── evidencias/            ← as imagens deste guia
```

Por que o contrato tem uma pasta só para ele

Esta é a pasta mais importante conceitualmente, e a que mais gente esquece de fazer.

Quando dois programas diferentes conversam, eles precisam concordar **antes** sobre o formato das mensagens. Se o ESP32 mandar `"valor"` e o Python esperar `"value"`, nada funciona — e o erro só aparece em execução, no pior momento possível.

O contrato resolve isso sendo um **arquivo**, não um acordo verbal. E não é decorativo: o consumidor **lê o schema do disco em tempo de execução** e valida cada mensagem contra ele.

ARQUIVO	O QUE É	QUEM USA
`sala.ambiente.v1.schema.json`	Descrição formal do evento: campos, tipos, faixas, obrigatoriedade	O consumidor valida cada mensagem contra ele
`gateway.contagem.v1.schema.json`	Mesma coisa para o comando de volta	Documentação; o ESP32 valida de forma simplificada, por limite de memória
`exemplos/*.json`	Uma instância válida por situação	Para entender e para testar sem ligar nada

Por que a versão está no nome

No dia em que o formato mudar de um jeito incompatível, cria-se `sala.ambiente.`**`v2`** e a `v1` continua existindo. Os dispositivos antigos seguem funcionando enquanto migram. Se a versão não estivesse no nome, você quebraria todo mundo de uma vez.

Os três arquivos do produtor são o projeto do Wokwi

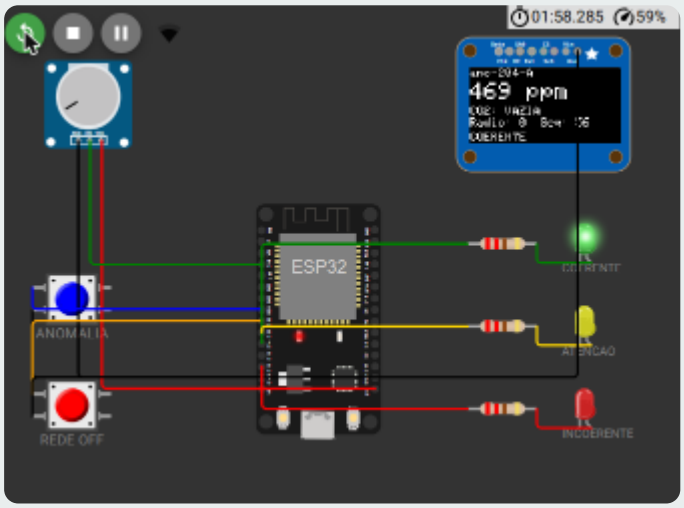
Se você abrir um Wokwi vazio e colar `sketch.ino`, `diagram.json` e `libraries.txt`, tem o protótipo inteiro de volta. São também três dos arquivos exigidos no ZIP da entrega individual.

O `diagram.json` merece atenção: ele é o **circuito descrito em texto**. Diz quais peças existem, onde ficam na tela e quais fios ligam o quê. O Wokwi desenha a partir dele — não existe um editor gráfico por trás guardando outra coisa.

Arquivos que nascem em execução

ARQUIVO	QUANDO APARECE	VERSIONADO?
`config.json`	Quando você copia o exemplo	Não — é seu, pode ter outro broker
`recebidos.log`	Toda execução do consumidor	Não — mas é a sua evidência de que funcionou

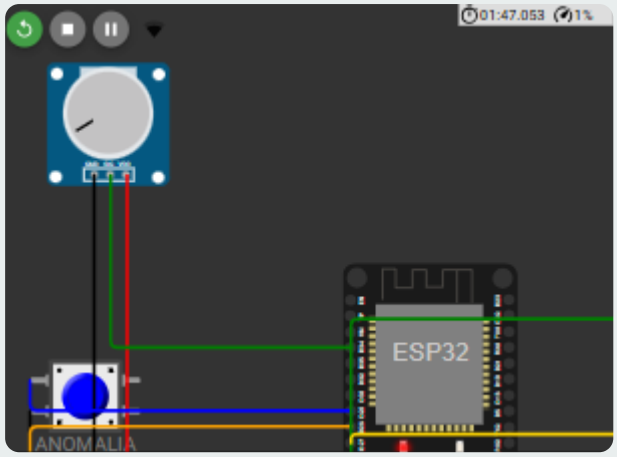
C4 O circuito, peça por peça



Visão geral

Onze componentes. Na foto, capturada com a simulação em execução, o display mostra `469 ppm` e o LED verde está aceso.

`GPIO34 potenciômetro` `GPIO25 botão anomalia` `GPIO26 botão rede` `GPIO14 LED verde` `GPIO27 LED amarelo` `GPIO13 LED vermelho` `GPIO21/22 display`



Potenciômetro — a entrada de CO_2

O que é: um divisor de tensão variável. Girando o botão, a tensão no pino `SIG` varia continuamente entre 0 V e 3,3 V.

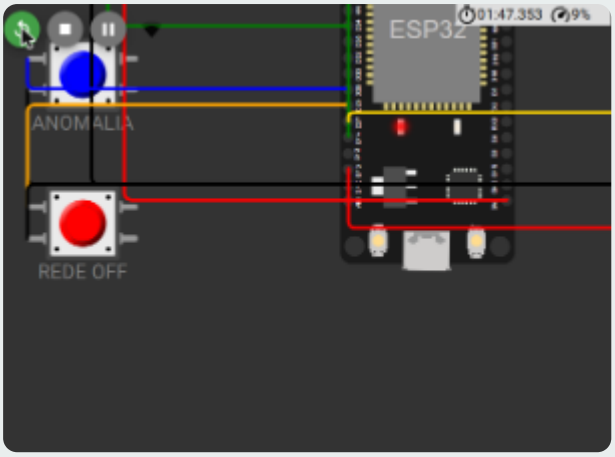
Como está ligado: `VCC` → `3V3`, `GND` → `GND`, `SIG` → `GPIO34`.

Como o código lê:

```
int cru = analogRead(PINO_POT);          // 0..4095 — ADC de 12 bits
return 380.0 + (cru / 4095.0) * (2200.0 - 380.0);
```

> **Declaração obrigatória**
>
> **Não é um sensor de CO_2.** Não há aquisição de grandeza física real, calibração nem resposta química. É uma fonte de valor controlável que eu declaro estar representando CO_2 entre 380 e 2200 ppm. O enunciado autoriza a substituição desde que ela seja declarada.

Por que o GPIO34: porque é entrada do **ADC1**. Os pinos do ADC2 param de funcionar quando o Wi-Fi está ligado — e este projeto vive de Wi-Fi. Usar um pino do ADC2 seria um bug silencioso: leituras zeradas assim que o rádio subisse.



Os dois botões — as condições de falha

Cada botão tem um lado no pino do ESP32 e o outro no `GND`. Não há resistor externo porque o código usa o **pull-up interno** do chip:

```
pinMode(PINO_BTN_ANOMALIA, INPUT_PULLUP);
```

Consequência que confunde na leitura do código: o pino lê `HIGH` quando o botão está *solto* e `LOW` quando está *pressionado*.

Botão azul — injeta anomalia de sequência

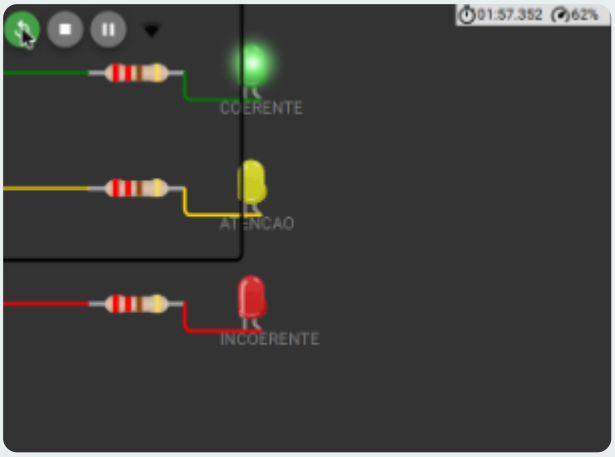
Cada aperto arma uma anomalia, aplicada na publicação seguinte, ciclando entre três. É o **teste adversarial obrigatório** da matrícula terminada em 8.

APERTO	ANOMALIA	O QUE A ÂNCORA FAZ
1º	`duplicata`	Publica de novo com a *mesma* sequência
2º	`fora-de-ordem`	Publica com sequência 3 números atrás
3º	`salto`	Pula 5 números na sequência

Botão vermelho — simula queda de rede

Com a rede fora, o ESP32 **continua amostrando e decidindo**, guarda os eventos numa fila de 20 posições e preserva o horário original de cada um. Ao apertar de novo, drena a fila marcando `bufferizado: true`.

Debounce: um botão físico não fecha o contato de forma limpa — nos primeiros milissegundos ele treme e gera dezenas de transições. Sem os 50 ms de debounce, um aperto viraria cinco anomalias armadas.



Os três LEDs e os resistores — a atuação

Cada LED segue o mesmo caminho: `GPIO → resistor 220 Ω → ânodo`, e o `cátodo → GND`.

LED	PINO	SIGNIFICA
Verde	`GPIO14`	CO_2 e contagem do rádio concordam
Amarelo	`GPIO27`	Dados insuficientes ou suspeita ainda não confirmada
Vermelho	`GPIO13`	O rádio declara ocupação que o CO_2 não sustenta

Por que 220 Ω: um LED é um diodo — ligado direto, conduz sem limite e queima, podendo levar junto o pino. O resistor limita a corrente a `(3,3 − 2,0) / 220 ≈ 6 mA`, confortavelmente abaixo dos 40 mA que o pino suporta.

Por que GPIO13 e não GPIO12: o GPIO12 é um **pino de strapping** (MTDI). Se estiver em nível alto no boot, o chip tenta reconfigurar a tensão da memória flash e pode não inicializar.



Display OLED — a observabilidade

Um SSD1306 de 128×64 pixels, monocromático, que comunica por **I²C** — um barramento de dois fios: `SDA` (dados) no `GPIO21` e `SCL` (relógio) no `GPIO22`. Responde no endereço `0x3C`.

Lendo a tela da foto, linha por linha:

`anc-204-A`	Identificação da âncora
`469 ppm`	CO_2 depois da média móvel
`CO2: VAZIA`	Ocupação inferida só pelo CO_2
`Radio: 0 · Seq: 51`	Contagem declarada pelo gateway e nº do evento
`COERENTE`	Estado final da verificação cruzada

> **Entenda esta tela**
>
> O estado é **COERENTE mesmo com a sala vazia** porque o rádio está declarando **zero** estudantes. Não há contradição entre "não tem ninguém" e "o rádio não vê ninguém". A incoerência só nasce quando o rádio afirma 10 ou mais *e* o CO_2 desmente.

Degradação: o código guarda se a inicialização do display deu certo e só desenha se deu. Display ausente ou com defeito não trava o resto — é decisão de projeto, não acaso.

O ESP32 — o cérebro

CARACTERÍSTICA	POR QUE IMPORTA AQUI
Dois núcleos a 240 MHz	Sobra folga para o filtro e o MQTT rodarem juntos
Wi-Fi integrado	É o que permite falar MQTT sem hardware adicional
ADC de 12 bits	Lê o potenciômetro com 4096 níveis
ADC2 inutilizável com Wi-Fi	Por isso o potenciômetro está no GPIO34
Pinos de strapping (0, 2, 12, 15)	Evitados para não atrapalhar o boot
`millis()` desde o boot	Não há relógio de tempo real: por isso `eventTimeMs` não é tempo absoluto

No Wokwi o ESP32 conecta na rede virtual `Wokwi-GUEST`, que tem saída real para a internet. É assim que ele alcança o broker MQTT público.

C5 Como o programa do ESP32 funciona

Amostragem e validação — a cada 400 ms

São **duas rejeições diferentes**, e vale saber distinguir:

```
// Rejeição 1 — leitura impossível: fora da faixa física do sensor
if (bruto < 350.0 || bruto > 5000.0) { amostrasInvalidas++; return false; }

// Rejeição 2 — ruído: gás não muda 300 ppm em 400 ms
if (temLeituraValida && fabs(bruto - ppmFiltrado) > 300.0) {
  amostrasInvalidas++; return false;
}
```

A primeira pega um valor que *nenhum sensor real produziria*. A segunda pega interferência elétrica: a concentração de um gás num ambiente fechado não muda centenas de ppm em quatro décimos de segundo — fisicamente não dá tempo.

A amostra aceita entra numa **média móvel de 5 posições**, que suaviza o que restou de ruído.

Camada 1 — ocupação pelo CO_2

Três mecanismos, todos dependentes de estado. Nenhum é comparação instantânea:

Histerese — dois limiares, não um

```
const float LIMIAR_OCUPADA = 800.0;   // sobe para OCUPADA acima disso
const float LIMIAR_VAZIA   = 600.0;   // desce para VAZIA abaixo disso
```

Com um limiar único de 700 ppm, um valor oscilando em torno dele faria o estado piscar dezenas de vezes por minuto. Com dois, existe uma **faixa morta** entre 600 e 800 onde o estado simplesmente não muda. É o mesmo princípio do termostato.

Persistência — a condição precisa se sustentar

Não basta cruzar o limiar: é preciso permanecer do outro lado por 6 a 8 segundos. O código acumula o tempo enquanto a condição vale e **zera assim que ela se quebra**.

Validade — o dado expira

Passados 10 segundos sem nenhuma leitura válida, a ocupação volta para `INDETERMINADA`. O sistema **esquece** o que sabia, em vez de continuar decidindo com base em dado velho.

Camada 2 — a verificação cruzada

```
bool condicaoSuspeita = (contagemRadio >= 10) && (ocupacao == OCUP_VAZIA);
```

Aqui estão as **duas entradas combinadas**: o CO_2, do sensor local, e a contagem do rádio, que chega pelo MQTT vinda do gateway. E a suspeita ainda precisa persistir por 10 segundos antes de virar acusação — antes disso o estado é ● **ATENÇÃO**, não ● **INCOERENTE**.

Rearme — evitando oscilação na atuação

Mesmo depois de tudo isso, o estado não pode mudar mais de uma vez a cada 3 segundos.

Montagem e publicação do evento

O JSON é montado com `snprintf`, sem biblioteca, para economizar memória — e vai para **dois lugares ao mesmo tempo**: o monitor serial (requisito da entrega individual) e o broker MQTT (a fronteira do Marco 2).

> **Armadilha que custa uma tarde**
>
> `mqtt.setBufferSize(512)` — o padrão do PubSubClient é **256 bytes**, e o nosso payload passa disso. Sem essa linha o `publish` falha *silenciosamente*: nenhum erro, nenhuma mensagem, simplesmente não chega.

Recebimento do comando — idempotência

```
if (cmdId.length() > 0 && cmdId == ultimoComandoId) return;   // já aplicado
```

O MQTT com QoS 1 garante entrega **ao menos uma vez** — a mesma mensagem pode chegar duas. Guardando o último `commandId` aplicado, uma reentrega não altera o estado. É o mesmo princípio que o consumidor usa com o `eventId`.

C6 Como o consumidor funciona

Validação contra o contrato, em tempo de execução

O consumidor **lê o arquivo de schema do disco** e valida cada mensagem contra ele. O contrato não é decorativo: se você acrescentar um campo ao evento no ESP32, o consumidor **rejeita** a mensagem, porque o schema declara `additionalProperties: false`.

> **A melhor alteração ao vivo**
>
> O professor pede uma pequena alteração durante a verificação. Acrescente um campo ao evento, mostre o consumidor rejeitando, e então versione para `v2`. Prova que o contrato está sendo validado de verdade, não apenas documentado.

As três detecções de anomalia

Esta é a parte que corresponde à responsabilidade individual declarada (**comunicação e resiliência**) e ao teste adversarial da matrícula.

DETECÇÃO	COMO FUNCIONA	O QUE O CONSUMIDOR FAZ
Duplicata	`eventId` já está no conjunto de vistos	Descarta sem alterar o estado
Fora de ordem	`sequence` menor que a maior já vista	Aceita e registra, mas não retrocede o estado
Salto	`sequence > maior + 1`	Conta quantos eventos faltaram e sinaliza

O conjunto de vistos é limitado a 500 entradas, descartando as mais antigas. Sem esse limite, um sistema rodando por semanas consumiria memória sem parar.

> **Detalhe que vale destacar**
>
> O consumidor **não usa** o campo `anomaliaInjetada` para detectar nada. Ele detecta pela sequência, exatamente como faria em produção. O campo existe só para deixar a demonstração legível.

O carimbo de tempo autoritativo

O `eventTimeMs` que vem do ESP32 é tempo *desde o boot*, não tempo absoluto — e o relógio de um dispositivo em campo não é confiável de qualquer forma. O tempo que vale para o registro é o do gateway, aplicado no instante da recepção.

O comando de volta

A cada 10 segundos o consumidor publica a contagem declarada pelo rádio. Durante a execução você digita um número no terminal para mudá-la — é assim que se provoca o estado ● INCOERENTE na demonstração.

C7 Um evento inteiro, do potenciômetro ao log

Vale decorar esta sequência — é a resposta para “explique como funciona”.

01 **Você**

Gira o potenciômetro para 470 ppm

02 **ESP32**

A cada 400 ms lê o ADC: `analogRead(34)` → 0..4095 → 470 ppm

03 **ESP32 · validação**

Está entre 350 e 5000? sim. Saltou mais de 300 ppm? não

04 **ESP32 · filtro**

Entra na média móvel de 5 amostras → `ppmFiltrado = 469`

05 **ESP32 · camada 1**

469 < 600 sustentado por 8 s → `ocupacao = VAZIA`

06 **Gateway → ESP32**

Havia publicado `contagem = 25`

07 **ESP32 · camada 2**

25 ≥ 10 **e** ocupação VAZIA → condição suspeita

08 **ESP32 · persistência**

A condição se mantém por 10 s → `estado = INCOERENTE`

09 **ESP32 · atuação local**

Apaga o LED verde, acende o vermelho, atualiza o display

10 **ESP32 · evento**

A cada 2 s monta o JSON com `eventId: "anc-204-A♯63"`

11 **ESP32 · saída**

Escreve no monitor serial **e** publica no tópico MQTT

12 **Broker**

Entrega ao consumidor

13 **Gateway · contrato**

Valida contra o schema → ok

14 **Gateway · dedup**

Verifica o `eventId` no conjunto de vistos → novo

15 **Gateway · sequência**

Verifica lacuna e ordem → sem salto

16 Gateway · tempo

Carimba o horário de recepção — este é o que vale

17 Gateway · atuação remota

Detecta a mudança para INCOERENTE → **alerta na tela**

18 Gateway · registro

Grava a linha em `recebidos.log`

C8 Perguntas prováveis, com resposta pronta

P. Por que o potenciômetro e não um sensor de verdade?

Porque o Wokwi não simula química de gás, e o enunciado autoriza a substituição desde que declarada. O que estou validando é a **lógica de estado, o contrato e a resiliência** — não a aquisição do sinal. Isso está no cabeçalho do código e na seção de limitações do README.

P. O que acontece se chegarem dois eventos iguais?

O consumidor descarta o segundo, porque o `eventId` já está no conjunto de vistos, e o estado não muda. Posso demonstrar agora apertando o botão azul.

P. E se um evento chegar fora de ordem?

É aceito e registrado, mas não retrocede o estado. Retroceder seria pior que ignorar — significaria voltar para uma leitura que já sei estar desatualizada.

P. Por que a regra não é só `if (co2 < 600)`?

Porque um valor instantâneo oscila e geraria acusações falsas. A regra usa histerese de dois limiares, persistência de 6 a 10 segundos, média móvel de 5 amostras, expiração do dado após 10 segundos e um tempo mínimo de 3 segundos entre mudanças de estado.

P. Onde está o estado guardado?

Em três lugares, cada um com um motivo. No **ESP32**: a janela da média, os acumuladores de tempo e a fila de eventos não enviados. No **consumidor**: o conjunto de `eventId` vistos, a maior sequência e os contadores. No **contrato**: nada — contrato não guarda estado, ele descreve formato.

P. O que acontece se a rede cair no meio da aula?

O ESP32 continua amostrando e decidindo, e guarda os eventos numa fila de 20 posições com o horário original preservado. Quando volta, drena a fila marcando `bufferizado: true`, e o consumidor os trata como eventos *atrasados*, não como novos. Se a fila encher, descarta o mais antigo **e registra o descarte** — assim o intervalo aparece como incompleto, em vez de silenciosamente vazio.

P. Isso escala para um prédio inteiro?

O volume é de cerca de meio evento por segundo por sala. Sessenta salas dão 30 eventos por segundo, o que qualquer broker aguenta. O gargalo não é o dado, é o número de conexões Wi-Fi — e é por isso que, no projeto completo, as três âncoras de cada sala formam uma malha ESP-NOW e só a líder fala com o gateway.

Projeto **Presença** · Marco 2 e Atividade 03 · Software para Sistemas Ubíquos · Instituto de Informática, UFG · Prof. Dr. Otávio Calaça Xavier
Alany Gabrielly, matrícula 202105018 · capturas e logs da execução real em `marco-02/evidencias/`